When the Scheme Is the Signal: Collection Artifacts, Domain Leakage and Cross-Corpus Failure in Lexical Phishing-URL Detection

Research paper — full draft with literature survey, results and appendices

[Author names]

[Department, Institution, City, Country]

[Corresponding author e-mail]

Document generated 2026-10-12 from the repository branch research-v2.

Status of the results. Sections 1–5, the literature survey and all appendices are complete. Section 6 reports only experiment runs that have finished; every unfinished experiment is marked “pending” and no number is extrapolated. The original project's dataset (raw_data.csv) is not available; results use documented public corpora. Abstract, discussion and conclusion are provisional until the experiment matrix completes.

Abstract

Lexical URL classifiers are routinely reported with 95–99 % accuracy on public phishing benchmarks, yet they fail when moved to differently collected data. We audit a typical detector (28 lexical features and a weighted soft-voting ensemble of logistic regression, random forest and XGBoost) and six public corpora with raw URLs, plus three live feeds. In one widely used corpus, every benign URL is an `https://www.` homepage; across corpora, the scheme, the `www.` prefix and URL length change in strength and even in direction of association with phishing. A linear probe identifies the source corpus of a benign URL with 83 % accuracy (chance 17 %); dropping artifact columns barely changes this, while canonicalising the URL string lowers it to 62 %. Models trained on PhiUSIIL reach ROC-AUC 0.998–0.999 with or without registered-domain-grouped splitting, yet with a threshold set for a 1 % false-positive budget they flag 99–100 % of benign URLs in four other corpora, and their ROC-AUC falls to 0.34–0.78, below chance on two corpora. Training on five pooled corpora restores ranking on a held-out corpus (ROC-AUC 0.77–0.98) but leaves validation-chosen thresholds badly miscalibrated, under-flagging one unseen corpus and over-flagging another. We release a tested, reproducible pipeline and argue that URL detectors should be evaluated across corpora, at fixed false-positive budgets, with recording format canonicalised.

Keywords — *phishing detection; malicious URL detection; lexical features; dataset bias; shortcut learning; cross-dataset generalisation; domain-grouped evaluation; calibration; machine learning for security*

Contents

Abstract 2

1. Introduction 4

2. Related Work 4

3. Problem Definition and Hypotheses 5

4. Method 6

4.1 Pipeline overview 6

4.2 Data handling 6

4.3 URL parsing 6

4.4 Representations 6

4.5 Models 7

4.6 Operating thresholds and calibration 7

4.7 Evaluation 7

5. Experimental Setup 8

5.1 Corpora 8

5.2 Splits 8

5.3 Environment 8

6. Results 9

6.1 Baseline reproduction 9

6.2 Collection artifacts differ in size and direction across corpora (RQ1, RQ2) 9

6.3 Random versus registered-domain-grouped splits on corpus A (RQ1, H1b) 10

6.4 External corpora: near-total false-positive rates (RQ2, H1a, H2) 12
6.5 Leave-one-corpus-out: training on diverse corpora (RQ3) 14
6.6 Pending analyses 15
7. Discussion 16
8. Threats to Validity and Limitations 16
9. Conclusion 17
References 17
Appendix A. Literature Survey Table 23
Appendix B. Literature Review 35
Appendix C. Research Gaps, Hypotheses and Selected Direction 41
Appendix D. Repository Audit of the Original Implementation 45
Appendix E. Dataset Audit 50
Appendix F. Experiment Protocol 53
Appendix G. Complete Result Tables 56
Appendix H. Additional Figures 59
Appendix I. Decision Log 66
Appendix J. Reproducibility Guide 67

1. Introduction

Phishing remains one of the most damaging and highest-volume attack vectors on the web, and most phishing campaigns live for hours rather than days [1]. Blacklists react too slowly to protect users during that window [2], which motivates *proactive* detectors that judge a URL from its string alone, before the page is fetched. Lexical URL classifiers are attractive for this role: they need no network access, add sub-millisecond latency, and are routinely reported with accuracies of 95–99 % [3, 4, 5, 6, 7, 8].

These numbers are hard to reconcile with what happens when such models leave the dataset they were trained on. Cross-dataset accuracy of URL classifiers has been reported at 51–59 % [9] and 57 % [10]; detectors "work effectively only when the test sets are split from the same training dataset" [11]; and models trained against homepage-only benign sets lose 27–39 points on legitimate login URLs [12]. The security-ML literature has a name for the likely cause — *spurious correlations from sampling bias* [13] — and phishing researchers have long warned that benign URLs drawn from popularity lists are structurally unlike phishing URLs [14, 15, 16, 17]. What is missing is a measurement: *which* properties of public URL corpora carry the reported performance, whether they point the same way across corpora, and whether removing them helps.

This study began as an audit of a course-project detector (28 lexical features, a 1:2:2 soft-voting ensemble of logistic regression, random forest and XGBoost, reported test accuracy 96.8 %). Its two most influential features by SHAP were the URL scheme (`https`) and a subdomain count driven by the `www.` prefix, and it classified `https://google.com/` as malicious with 88 % probability. We show that this is not an implementation accident but a property of the data the field evaluates on.

Research questions.

- **RQ1.** How much of the in-distribution performance of lexical URL classifiers on public phishing corpora is carried by nameable *collection artifacts* (scheme, `www.` prefix, homepage-only benign URLs) and by registered-domain memorisation?
- **RQ2.** Do these artifacts point in the same direction across corpora, and does that explain cross-corpus failure?
- **RQ3.** Does an artifact-aware recipe — string canonicalisation, domain-grouped model selection, validation-chosen operating thresholds — restore performance on unseen corpora, and do probabilities and thresholds remain reliable under shift?

Contributions. (Stated as *provisional* until every run in §6 is complete; each is tied to a table.)

1. A per-class artifact audit of six public phishing-URL corpora and three live feeds, showing that the dominant artifacts reverse sign between corpora (§5.1, Table T1, DATASET_AUDIT).
2. A dataset-identification probe showing that the source corpus of a benign URL is predictable from lexical features far above chance, that dropping artifact columns barely changes this, and that canonicalising the string does (§6.2, Table T10).
3. A controlled comparison of random versus registered-domain-grouped splits, in-distribution versus external and leave-one-corpus-out evaluation, for three feature representations and four model families including a character CNN (§6.3–6.5).
4. An operating-point analysis: calibration and the transfer of validation-chosen false-positive thresholds to unseen corpora (§6.6).
5. An open, tested implementation (parser, features, splits, metrics, runner, 90+ unit tests) and full provenance of every dataset and number.

2. Related Work

Lexical features and classical learners. The URL string alone was shown to separate phishing from benign pages in early measurement work [18, 19], and bag-of-token and host features reached 95–99 %

accuracy [3, 20, 21]. Later work added obfuscation-resistant and online features [4, 22], distributional character statistics [23], word-level NLP features [6], search-engine relatedness [24] and benchmark corpora [5, 16, 25]. Tree ensembles are consistently strongest among classical learners, and stacking or voting adds about one point on random splits [7, 8, 26, 27].

Learned representations. Character CNNs and LSTMs [28, 29, 30, 31, 32, 33, 34] and transformers or URL-pretrained encoders [35, 36, 37, 38, 39, 40] improve in-distribution scores, especially at low false-positive rates on production telemetry. TransURL is the only representation paper we found with a substantial cross-dataset test, where all models degrade sharply [39]. Notably, Aljofey et al. strip `http://`, `https://` and `www.` before modelling without measuring the effect [32].

Evaluation realism. Arp et al. identify sampling bias, spurious correlations and the base-rate fallacy as pervasive pitfalls of security ML [13]; TESSERACT shows that temporally inconsistent splits inflate malware F1 [41]; and pragmatic assessment calls for repeated trials over several datasets [42]. For phishing, guidelines require diverse benign URLs with paths, realistic prevalence and temporal tests [14]; balanced, Alexa-derived benchmarks are criticised [15, 43, 44]; benign index pages are said to "create a bias in the data" [17]; and leakage and base rates inflate web-page benchmarks [45]. Cross-dataset studies of URL detectors report collapse [9, 11, 10] but operate on pre-computed feature tables or do not isolate the mechanism. The closest work is a single-author preprint that names the `https://` and `www.` shortcut and tests canonicalisation on one external benign set [46], and a study of phishing *e-mail* that uses a corpus-identification probe and artifact-removal ablations [47].

Robustness, explanation and calibration. URL detectors are brittle to cheap adversarial URLs [17, 48, 49], and real attackers favour such cheap manipulations [50]. SHAP [51, 52] is widely used for feature selection in phishing work [53, 54] but rarely to diagnose artifacts [9]. Calibration under dataset shift has been studied for Android malware [55] building on [56, 57], but we found no study for phishing URLs.

Positioning. We work on raw URLs from six public corpora, name and quantify specific artifacts per class and per corpus, separate artifact effects from domain memorisation with grouped splits, test the recipe under leave-one-corpus-out transfer, and measure whether operating thresholds survive the shift. A full comparison table is in `research/LITERATURE_SURVEY_TABLE.md`.

3. Problem Definition and Hypotheses

Given a URL string u, a detector outputs $p(u) \approx \text{P}(\text{malicious} \mid u)$, and flags u when $p(u) \geq \tau$. We call a property $a(u)$ a **collection artifact** if (i) it is determined by how a source recorded or sampled URLs rather than by the URL's intent, and (ii) its class-conditional distribution differs within a corpus. The scheme prefix (absent, `http`, `https`), a leading `www.` label, and the restriction of a benign source to bare homepages satisfy both conditions in the corpora studied (§5.1).

Hypotheses (pre-registered in `research/RESEARCH_GAPS.md`; H1c and canonicalisation were added post hoc as described in `EXPERIMENT_PROTOCOL.md`):

- **H1a** Removing artifact features changes in-distribution ROC-AUC only marginally but changes false-positive rates on external corpora substantially.
- **H1b** Random splits over-estimate grouped-split performance, most on corpora with many URLs per registered domain.
- **H1c** The source corpus of a URL is identifiable from lexical features well above chance.
- **H2** Thresholds selected for a 1 % false-positive budget on validation data do not hold on external corpora; post-hoc calibration does not repair this.
- **H3** Recall on timestamped phishing feeds changes with submission year as the share of https phishing rises.

- **H4** A character CNN shares the artifact dependence of lexical models unless its input is canonicalised.

4. Method

4.1 Pipeline overview

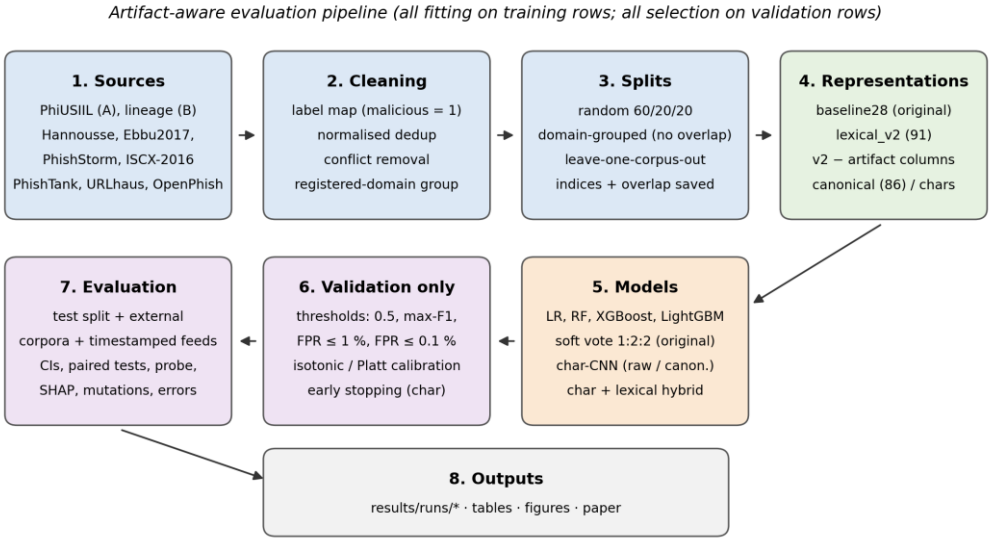


Fig. 1. Pipeline of the study. All fitting uses training rows only and all model, threshold and calibration selection uses validation rows only; test, external corpora and feeds are scored once.

4.2 Data handling

Each source is mapped to `url, label` with malicious = 1. URLs are stripped of surrounding whitespace; a *normalised key* (lower-case scheme and host, default port and fragment removed, empty path collapsed) is used only for duplicate detection; all normalised keys carrying both labels are removed. Features are always computed on the original string. The grouping key is the registered domain — the label directly below the public suffix, computed with the offline Public Suffix List snapshot of `tldextract` 5.3.1 — or the literal address for IP hosts.

4.3 URL parsing

The parser accepts any string, prepends `http://` when no scheme is present (recording the fact), and never raises: malformed authorities (e.g. unmatched IPv6 brackets) and invalid ports set a `parse_error` flag. It extracts scheme, userinfo, host, port, path, query, fragment, subdomain, registered domain and public suffix, IP and IPv6 status, and the IDNA-decoded host. Tests cover missing schemes, IPv4/IPv6 literals, ports, userinfo, punycode, Unicode hosts, percent-encoding (including double encoding), 15 kB strings and non-URL strings (`tests/test_feature_extractor.py`).

4.4 Representations

- **baseline28** — the original project's 28 features computed by its unmodified code (lengths and character counts, digit-to-letter ratio, entropy, subdomain count, last host label as a categorical `tld_type`, IP host, `https`, shortener and suspicious-TLD lists, keyword and brand substrings, hyphen and randomness heuristics). `tld_type` is one-hot encoded with the encoder fitted on training rows only and TLDs with fewer than 20 training occurrences pooled.
- **lexical_v2** — 91 numeric features computed from the parsed components: whole-URL shape (length, entropy, character-class counts and ratios, longest digit run, token statistics, hex tokens, double encoding); scheme and authority (`https`, explicit scheme, `www.` prefix, userinfo, ports);

hostname (length, entropy, label counts with and without the public suffix, registered-domain length and entropy, digit and hyphen counts, consonant runs, IP/punycode/non-ASCII, abused suffix, free-hosting suffix, shortener); brand tokens *placed* in subdomain, registered domain or path rather than matched as raw substrings; path, query and fragment structure (depth, extensions, embedded URLs or e-mail addresses, keyword placement).

- **lexical_v2 − artifacts** — `lexical_v2` without the five columns that encode recording format: `https_scheme`, `had_explicit_scheme`, `has_www_prefix`, `host_num_labels`, `num_subdomain_labels_psl`. The analogous ablation of baseline28 removes `https_usage` and `num_subdomains`.
- **lexical_v2_canon** — the string is first *canonicalised* by removing the scheme and one leading `www.`/`wwwN.` label (regular expression `^(?:[a-zA-Z][a-zA-Z0-9+.-]*://)?(?:www\d?\.)?`); `lexical_v2` is computed on the result and the five artifact columns are dropped, leaving 86 features. By construction every feature is invariant to the presence or form of the scheme and `www.` prefix, which is unit-tested.
- **char** — the first 160 characters as ASCII codes (others mapped to an unknown symbol), optionally canonicalised.

No representation uses network lookups.

4.5 Models

Logistic regression (standardised inputs, C = 1, balanced class weights), random forest (200 trees, balanced class weights) and XGBoost (200 rounds, depth 6, learning rate 0.1, row and column subsampling 0.8) keep the original project's settings so that the reproduction is faithful; LightGBM (400 rounds, 63 leaves, learning rate 0.05) is added as a second boosting family; the **soft vote** averages LR, RF and XGBoost probabilities with weights 1:2:2 as in the original project. The **char-CNN** follows eXpose/URLNet [28, 30] at small scale: a 32-dimensional character embedding, three parallel 1-D convolutions (kernel widths 3, 5, 7; 128 filters each) with ReLU and global max-pooling, dropout 0.3, a 128-unit dense layer and a sigmoid output (115,329 parameters). The **hybrid** concatenates standardised `lexical_v2 − artifacts` features to the pooled convolutional vector before the dense layer. Char models are trained with Adam (learning rate 10^{-3}, batch 256), class-balanced binary cross-entropy, at most 5 epochs or 20 minutes, keeping the epoch with the best validation ROC-AUC (patience 2). No hyper-parameter was tuned on test or external data.

4.6 Operating thresholds and calibration

For each model and seed, four thresholds are fixed on the validation split: $\tau = 0.5$; the τ maximising validation F1; and the smallest τ whose validation false-positive rate is at most 1 % (respectively 0.1 %), i.e. the most sensitive threshold within the false-positive budget. The same τ is applied unchanged to the test split and to every external corpus. Post-hoc calibration (isotonic regression and Platt scaling) is fitted on validation scores only.

4.7 Evaluation

Positive class: malicious. Reported: accuracy, precision, recall, specificity, F1, FPR, FNR, balanced accuracy, MCC, ROC-AUC, PR-AUC (average precision), TPR at FPR 1 % and 0.1 %, log-loss, Brier score and expected calibration error (15 equal-width bins). On positive-only feeds only recall is defined. Uncertainty: mean ± standard deviation over seeds; 95 % percentile-bootstrap intervals (1,000 resamples) for single-seed point estimates; paired bootstrap of metric differences and McNemar's exact test for comparisons on identical test URLs. A difference is called significant only when the paired 95 % interval excludes zero and its sign is the same for all seeds.

Probe. For each class separately, 8,000 URLs per corpus (fewer where unavailable) are pooled from six corpora, and a multinomial logistic regression predicts the source corpus from a representation under 5-fold cross-validation grouped by registered domain. Chance is 1/6.

Explanations. TreeExplainer SHAP values [52] for XGBoost on 3,000 test URLs.

Label-preserving variants. Tier A transformations are equivalent under RFC 3986 and preserve the label by construction: upper-casing scheme and host, adding an empty-path slash, adding the default port, percent-encoding one unreserved path character, adding a fragment. Tier B transformations change only presentation and are *assumed* label-preserving: toggling a leading `www.`, toggling `http`/`https`, appending a tracking parameter. We report the share of predictions that flip at $\tau = 0.5$ and the mean absolute change in probability.

5. Experimental Setup

5.1 Corpora

TABLE I. Corpora after cleaning (malicious = 1). Shares are over all URLs of the corpus.

corpus	raw	kept	malicious	benign	registered domains	share https	share www.	timestamps
ebbu2017	73575	54721	24811	29910	21606	0.207	0.442	no
hannousse2021	11430	11416	5701	5715	6634	0.389	0.44	no
iscx2016	45343	45324	9946	35378	4304	0.047	0.096	no
kaggle651k_bp	522214	514031	90015	424016	145482	0.017	0.095	no
lineage253k	253098	252394	123095	129299	130453	0	0.156	no
openphish	300	300	300	0	144	0.713	0.19	no
phishstorm	96018	95962	47972	47990	30894	0.018	0.523	no
phishtank	67621	67326	67326	0	12971	0.919	0.042	2011-02→2026-10
phiusiil	235795	234687	99838	134849	175508	0.781	0.753	no
urlhaus	72849	72847	72847	0	17585	0.379	0.005	2018-06→2026-10

Corpus A is PhiUSIIL [25] (234,687 URLs after cleaning). Corpus B is a 253,098-URL stratified sample of the Kaggle/HuggingFace lineage (`Mitake/PhishingURLsANDBenignURLs`), sized and balanced like the original project's unavailable dataset (129,420 / 123,678); 252,394 remain after cleaning. External labelled corpora: Hannousse & Yahiouche 2021 [16], Ebbu2017 [6], PhishStorm [24], ISCX-URL-2016 benign and phishing lists [5], and (two early runs only) the 651k Kaggle corpus [58]. Positive-only feeds: PhishTank verified-online dump with submission times (67,326 URLs, 2011–2026; only 64 submitted before 2021), URLhaus malware URLs (72,847; scored to check out-of-task behaviour), and an OpenPhish snapshot (300).

The original project's dataset could not be obtained; its provenance analysis (`research/DATASET_AUDIT.md` §1) indicates a scheme-normalised compilation of the same Kaggle lineage with a newer phishing source.

5.2 Splits

Random: stratified 60/20/20. Grouped: `StratifiedGroupKFold` with 10 folds on registered domain; two folds for test, two for validation, six for training; no registered domain occurs in two partitions. LOCO: for each of six labelled corpora, the other five are pooled (each capped at 50,000 URLs, stratified), split by registered domain into train/validation/test, and the whole held-out corpus is the evaluation set. Split indices, class balance and domain overlap are saved for every seed.

5.3 Environment

Intel Core i7-1355U (10 cores, 12 threads), 16 GB RAM, no GPU; Windows 11; Python 3.10.11, scikit-learn 1.7.2, XGBoost 3.2.0, LightGBM 4.7.0, PyTorch 2.13.0 (CPU), SHAP 0.49.1, NumPy 1.26.4, pandas 2.3.3, tldextract 5.3.1. Seeds 42–46 (tabular), 42–44 (LOCO and char models on corpus A), 42 (char models on corpus B). Every run directory contains `config.json` and `environment.json`.

6. Results

Status of the evidence on 2026-10-12: complete for §6.1, §6.2 and the baseline-feature runs on corpus A (random split, 5 seeds; grouped split, 5 seeds); partial for leave-one-corpus-out (2 of 6 held-out corpora); **pending** for corpus B, `lexical_v2` and canonicalised runs, character models, calibration, SHAP, mutation and error analyses. Pending items are listed explicitly and no number is extrapolated.

6.1 Baseline reproduction

The original repository's code path — its unmodified feature extractor, data-splitting function, model factories, ensemble and metric functions — runs end to end on both substitute corpora (Table II). On PhiUSIIL it scores higher than the project reported on its own unavailable data: the 1:2:2 soft vote reaches test accuracy 0.9970, F1 0.9965 and ROC-AUC 0.9988, against the stored accuracy of 0.9678 on the original data. The four models are within 0.0018 accuracy of each other, and the ensemble exceeds XGBoost by only 0.0002 ROC-AUC. This matches the audit's observation that the ensemble did not beat random forest on the original data either.

On corpus B the same code reaches only 0.8484 accuracy and 0.9317 ROC-AUC. The cause is a defect, not a property of the corpus. 99.8 % of corpus-B URLs carry no scheme, and for such input the original extractor parses no host at all, so domain length, subdomain count and TLD collapse to constants (verified on single examples; regression test in the repository). Corpus-B results that use `baseline28` therefore measure this defect as well as the model. `lexical_v2` uses a parser that handles scheme-less input.

TABLE II. Re-execution of the original pipeline code (random 60/20/20 split, seed 42, threshold 0.5), test split. The first four rows are the stored results of the original project on its unavailable dataset, shown for reference only.

data	model	accuracy	precision	recall	f1	roc_auc
original raw_data.csv (stored result, not re-run)	Logistic Regression	0.8864	0.903	0.8598	0.8809	not stored
original raw_data.csv (stored result, not re-run)	Random Forest	0.9679	0.9771	0.9568	0.9668	not stored
original raw_data.csv (stored result, not re-run)	XGBoost	0.9671	0.9828	0.9492	0.9657	not stored
original raw_data.csv (stored result, not re-run)	Weighted Soft-Voting Ensemble	0.9678	0.9824	0.9512	0.9665	not stored
phiusiil (original code re-run)	Logistic Regression	0.9952	0.9985	0.9903	0.9944	0.9983
phiusiil (original code re-run)	Random Forest	0.9968	0.9984	0.994	0.9962	0.9984
phiusiil (original code re-run)	XGBoost	0.9969	0.9992	0.9936	0.9964	0.9986
phiusiil (original code re-run)	Weighted Soft-Voting Ensemble	0.997	0.9994	0.9935	0.9965	0.9988
lineage253k (original code re-run)	Logistic Regression	0.7025	0.7403	0.6009	0.6633	0.805
lineage253k (original code re-run)	Random Forest	0.8455	0.8465	0.8346	0.8405	0.9239
lineage253k (original code re-run)	XGBoost	0.8406	0.8542	0.8117	0.8324	0.9265
lineage253k (original code re-run)	Weighted Soft-Voting Ensemble	0.8484	0.8529	0.8328	0.8427	0.9317

6.2 Collection artifacts differ in size and direction across corpora (RQ1, RQ2)

Fig. 2 and Appendix E report the class-conditional shares of three recording properties. In PhiUSIIL, **every** benign URL is an `https://www.` homepage with an empty path, against 48 % https, 42 % `www.` and 72 % homepages among phishing URLs. A single binary indicator therefore separates the classes well: the single-feature ROC-AUC is 0.21 for the `www.` prefix and 0.24 for the scheme, where values far below 0.5 mean "more frequent in benign URLs".

The same indicators carry little or opposite information elsewhere. The `www.` prefix has single-feature AUC 0.27–0.34 in Hannousse 2021, Ebbu2017 and PhishStorm, but 0.71 in ISCX-2016 and 0.69 in the Kaggle corpus, where it marks *phishing*. URL length points to phishing in PhiUSIIL and PhishStorm (0.72–0.73) and to benign in ISCX-2016 (0.24), whose 35,378 benign URLs are deep pages from only about 300 sites. The present-day phishing population differs again: 92 % of PhishTank's verified-online URLs are https and only 4 % start with `www.`.

The dataset-identification probe (Table III) shows that these differences are learnable. With `lexical_v2`, a linear model identifies which of six corpora a **benign** URL comes from with accuracy 0.830 (chance 0.167), and a malicious URL with accuracy 0.606. Removing the five artifact columns barely changes this (0.814 and 0.578), because the scheme still leaks through slash, colon and length counts; the two scheme-less corpora stay identifiable at 0.97 or higher. Canonicalising the string before computing features lowers the accuracies to 0.617 and 0.428 and makes the scheme-less corpus B nearly indistinguishable among benign URLs (0.184). PhiUSIIL's benign URLs remain identifiable at 0.981 even after canonicalisation: being a bare homepage is a sampling artifact that no string transformation removes. The original 28 features give 0.803 and 0.577. H1c is supported.

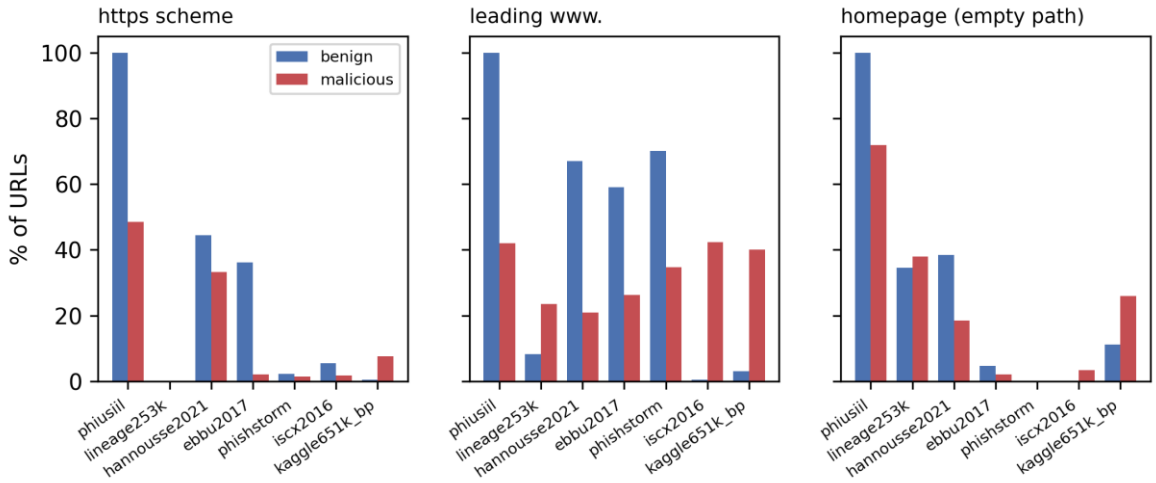


Fig. 2. Share of URLs with an https scheme, a leading www. label and an empty path (homepage), by class and corpus. Corpus B and the Kaggle corpus store URLs without a scheme.

TABLE III. Dataset-identification probe: accuracy of a 6-way logistic regression predicting the source corpus of a URL from its representation (grouped 5-fold CV; chance = 0.167), overall and recall per corpus.

class	representation	accuracy	phiusiil	lineage253k	hannousse2021	ebbu2017	phishstorm	iscx2016
benign	baseline28	0.803	1	0.999	0.367	0.544	0.841	0.945
malicious	baseline28	0.577	0.872	0.999	0.18	0.506	0.3	0.489
benign	lexical_v2	0.83	1	1	0.402	0.67	0.82	0.967
malicious	lexical_v2	0.606	0.865	0.999	0.277	0.535	0.385	0.481
benign	lexical_v2_canon	0.617	0.981	0.184	0.319	0.553	0.703	0.874
malicious	lexical_v2_canon	0.428	0.748	0.344	0.107	0.505	0.37	0.402
benign	lexical_v2 − artifact cols	0.814	1	0.99	0.383	0.661	0.766	0.961
malicious	lexical_v2 − artifact cols	0.578	0.839	0.972	0.179	0.544	0.369	0.45

6.3 Random versus registered-domain-grouped splits on corpus A (RQ1, H1b)

Random splitting leaks domains asymmetrically. With seed 42, 59.4 % of phishing test URLs share a registered domain with a training URL, against 2.4 % of benign test URLs (26.7 % overall; seed 43: 58.7 % and 2.3 %). Grouped splitting removes the overlap by construction (Fig. 3; full results in Appendix G, Table T3).

Despite this leakage, grouping hardly changes in-distribution performance on PhiUSIIL with the baseline features (5 seeds each; Table IV):

TABLE IV. In-distribution test performance on corpus A with the original 28 features: random vs registered-domain-grouped split (mean ± sd over 5 seeds).

Model	ROC-AUC random	ROC-AUC grouped	F1@0.5 random	F1@0.5 grouped
Logistic regression	0.998 ± 0.000	0.998 ± 0.000	0.995 ± 0.001	0.993 ± 0.003
Random forest	0.998 ± 0.000	0.998 ± 0.000	0.996 ± 0.001	0.994 ± 0.003
XGBoost	0.999 ± 0.000	0.999 ± 0.000	0.996 ± 0.001	0.995 ± 0.003
Soft vote 1:2:2	0.999 ± 0.000	0.999 ± 0.000	0.996 ± 0.001	0.995 ± 0.003

H1b is **not supported on corpus A**. The models do not need memorised domains because the recording artifacts already separate the classes. Whether grouping matters on corpora with many URLs per domain (corpus B; ISCX-2016 within LOCO) is **pending**.

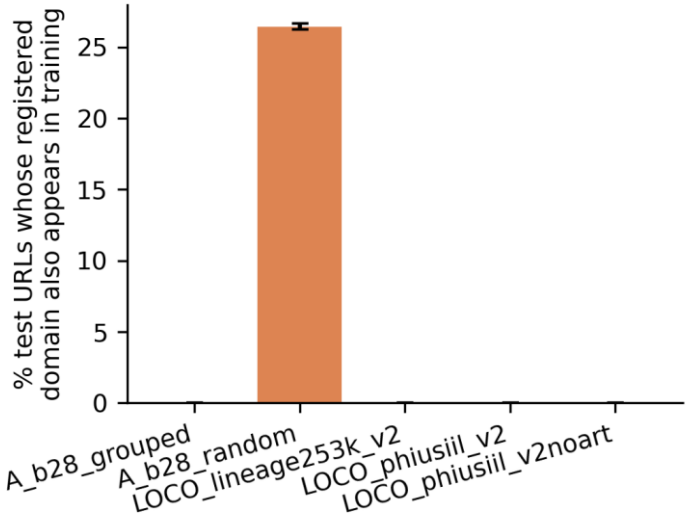


Fig. 3. Share of test URLs whose registered domain also occurs in the training partition, random vs grouped splits (mean ± sd over seeds).

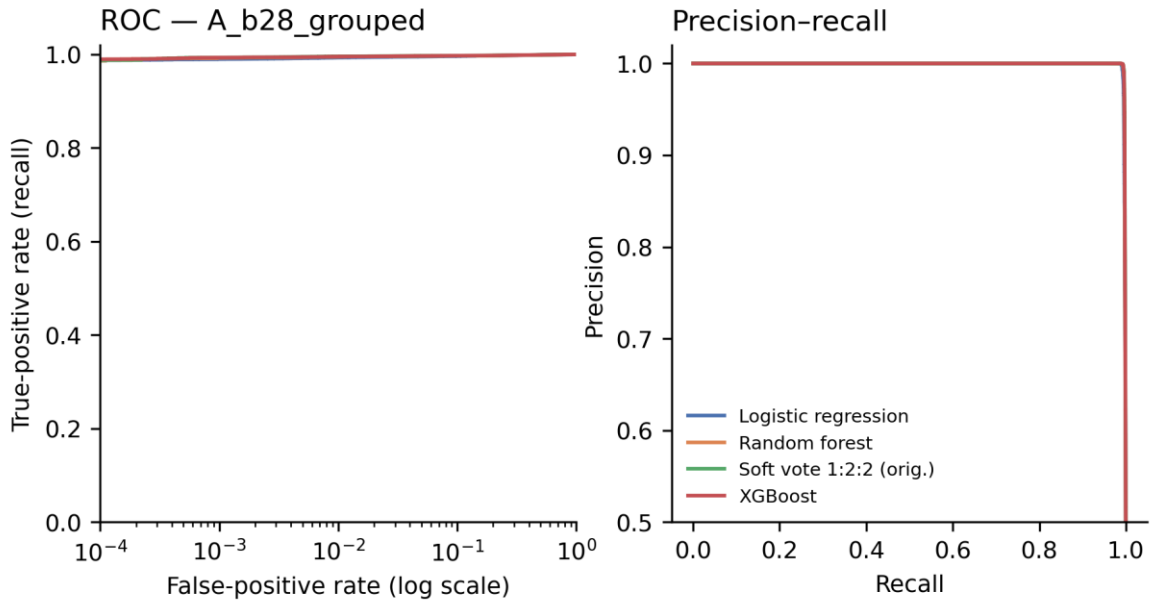


Fig. 4. ROC (log-scaled false-positive axis) and precision–recall curves on the corpus-A grouped test split, seed 42.

6.4 External corpora: near-total false-positive rates (RQ2, H1a, H2)

Models trained on corpus A were applied unchanged to every other corpus, using the threshold chosen on corpus-A validation data for a 1 % false-positive budget (Fig. 5; Table V; full results in Appendix G, Table T4). In-distribution the threshold holds: realised test FPR is 0.009–0.010 at recall 0.992–0.995.

On external corpora it fails for every model. **99.4–100 % of the benign URLs of Hannousse 2021, Ebbu2017, PhishStorm and ISCX-2016 are flagged.** On corpus B, logistic regression flags 74 % and every tree model and the soft vote flag 100 %. Ranking quality collapses as well. External ROC-AUC is 0.56–0.76 on Hannousse, Ebbu2017 and PhishStorm and falls **below chance** on ISCX-2016 (0.42–0.47) and the Kaggle corpus (0.35–0.48), the two corpora where `www.` marks phishing rather than benign URLs (§6.2).

The in-distribution model ranking (all ≥ 0.998) bears no relation to the external ranking. Random forest, which ties for best in-distribution, has the lowest external AUC on Hannousse, Ebbu2017 and PhishStorm (0.56–0.62).

Recall on the positive-only feeds is 0.99–1.00 (PhishTank 0.992–0.996, OpenPhish 1.000, URLhaus malware URLs 1.000). This is not evidence of good detection: a detector that flags every URL not shaped like an `https://www.` homepage has near-perfect recall and near-total FPR. At an operational prevalence of 1 % malicious URLs, recall 0.995 with FPR 0.994 gives a precision of about 1 %, no better than flagging at random.

The random-split runs give the same picture (FPR 0.994–1.000 on the four independent labelled corpora, 0.74–1.00 on corpus B; external AUC 0.34–0.78; Appendix G, Table T4). H2 is supported for the baseline representation.

Preliminary (1 of 5 seeds): removing the two artifact columns of the original features (`https_usage`, `num_subdomains`) barely changes in-distribution ROC-AUC (XGBoost 0.999, soft vote 0.999, random forest 0.997, logistic regression 0.987) and does not repair external behaviour: 94–100 % of the benign URLs of Hannousse 2021, Ebbu2017, PhishStorm and ISCX-2016 are still flagged, and XGBoost's ROC-AUC on ISCX-2016 falls further, to 0.10. This is consistent with the probe (§6.2): the shortcut survives column removal because the remaining length and path features still encode the homepage-only benign class. The remaining seeds, `lexical_v2` and the canonicalised representation are **pending** (runs `A_b28noart_grouped`, `A_v2noart_grouped`, `A_v2canon_grouped`).

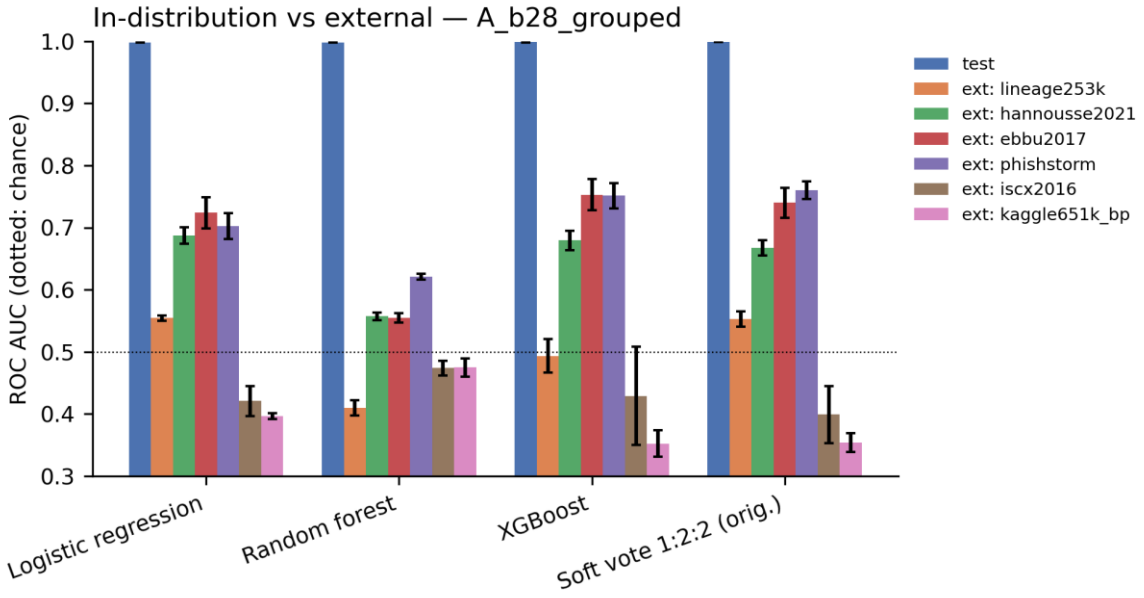


Fig. 5. ROC-AUC on the corpus-A grouped test split and on the external labelled corpora for models trained on corpus A with the original features (mean ± sd over 5 seeds). Dotted line: chance.

TABLE V. External evaluation of corpus-A models (original features, grouped split, 5 seeds): ROC-AUC and the share of benign URLs flagged (FPR) at the threshold chosen on corpus-A validation data for FPR ≤ 1 %; PhishTank is positive-only (recall).

model	in-dist AUC	hannousse2021 AUC	hannousse2021 FPR	ebbu2017 AUC	ebbu2017 FPR	phishstorm AUC	phishstorm FPR	iscx2016 AUC	iscx2016 FPR	lineage253k AUC	lineage253k FPR	phishtank recall
lr	0.998 ± 0.000	0.688 ± 0.013	0.994 ± 0.000	0.724 ± 0.025	1.000 ± 0.000	0.703 ± 0.021	1.000 ± 0.000	0.421 ± 0.024	1.000 ± 0.000	0.554 ± 0.004	0.743 ± 0.001	0.992 ± 0.001
rf	0.998 ± 0.000	0.557 ± 0.006	0.994 ± 0.000	0.555 ± 0.008	1.000 ± 0.000	0.621 ± 0.005	1.000 ± 0.000	0.474 ± 0.012	1.000 ± 0.000	0.410 ± 0.012	1.000 ± 0.000	0.995 ± 0.000
soft vote	0.999 ± 0.000	0.667 ± 0.011	0.994 ± 0.000	0.738 ± 0.021	1.000 ± 0.000	0.758 ± 0.013	1.000 ± 0.000	0.425 ± 0.071	1.000 ± 0.000	0.553 ± 0.011	1.000 ± 0.000	0.995 ± 0.000
xgb	0.999 ± 0.000	0.679 ± 0.016	0.994 ± 0.000	0.753 ± 0.025	1.000 ± 0.000	0.752 ± 0.020	1.000 ± 0.000	0.429 ± 0.079	1.000 ± 0.000	0.493 ± 0.027	1.000 ± 0.000	0.995 ± 0.000

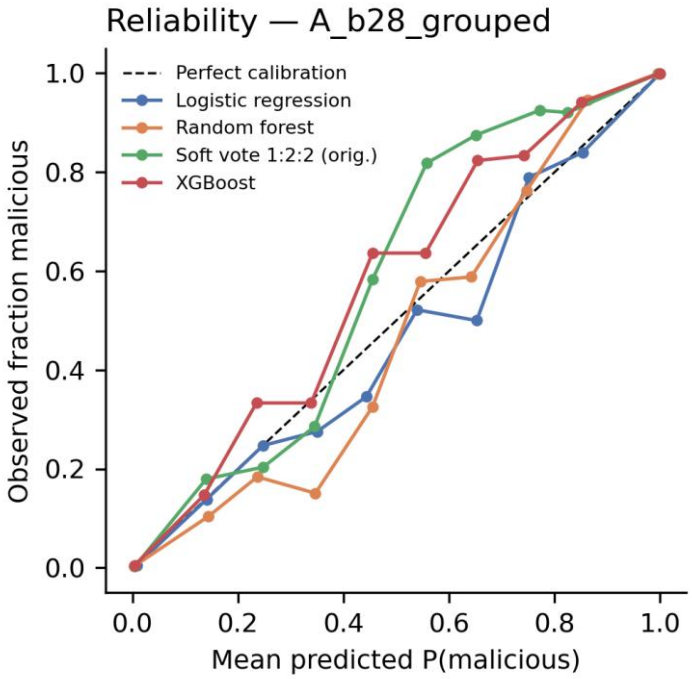


Fig. 6. Reliability diagram on the corpus-A grouped test split (seed 42). Expected calibration error is at most 0.004 because almost all scores lie near 0 or 1; the few mid-range scores deviate visibly from the diagonal.

6.5 Leave-one-corpus-out: training on diverse corpora (RQ3)

So far two held-out corpora have been evaluated. With PhiUSIIL held out (3 seeds), using models trained on a pool of the other five corpora with grouped train/validation/test splits. Training on diverse data changes the failure mode. Ranking transfers well to the held-out corpus: with `lexical_v2`, ROC-AUC is 0.977 (XGBoost), 0.966 (LR) and 0.891 (RF), and FPR at the validation-chosen 1 % threshold is 0.000. Recall at that threshold, however, falls to 0.42 (XGBoost), 0.25 (RF) and 0.15 (LR) on PhiUSIIL phishing, and to 0.16–0.44 on PhishTank. The held-out corpus's score distribution has shifted, so a threshold that is right for the pool is far too conservative for the new corpus.

Removing the five artifact columns *lowers* held-out ROC-AUC (XGBoost 0.977 → 0.878; LR 0.966 → 0.888; RF 0.891 → 0.866), while recall at the validation threshold stays similar (0.32–0.42). In this direction the artifact columns carried information that transferred, so artifact removal is not uniformly beneficial.

With corpus B held out (`lexical_v2`, 3 seeds), ranking transfers less well: held-out ROC-AUC is 0.874 ± 0.004 (XGBoost), 0.865 ± 0.004 (LR) and 0.772 ± 0.003 (RF), against 0.934–0.980 on the pool's own test split. Here the validation threshold errs in the *opposite* direction from the PhiUSIIL case: at the threshold chosen for a 1 % false-positive budget, the models flag 16.0 % (XGBoost), 17.6 % (RF) and 94.6 % (LR) of corpus-B benign URLs, at recall 0.71, 0.42 and 1.00. The same recipe thus under-flags one unseen corpus and over-flags another.

Removing the five artifact columns (3 seeds) changes held-out ROC-AUC on corpus B only slightly (XGBoost 0.874 → 0.849; LR 0.865 → 0.883; RF 0.772 → 0.780) but removes logistic regression's over-flagging: its FPR at the validation threshold falls from 0.946 ± 0.034 to 0.152 ± 0.113, with recall 0.72 instead of 1.00. For the tree models FPR stays at 0.15–0.19. Corpus B stores URLs without a scheme, so a linear model that gives weight to the scheme columns shifts all corpus-B scores at once; without those columns the shift disappears. The remaining four held-out corpora and the canonicalised representation are **pending**.

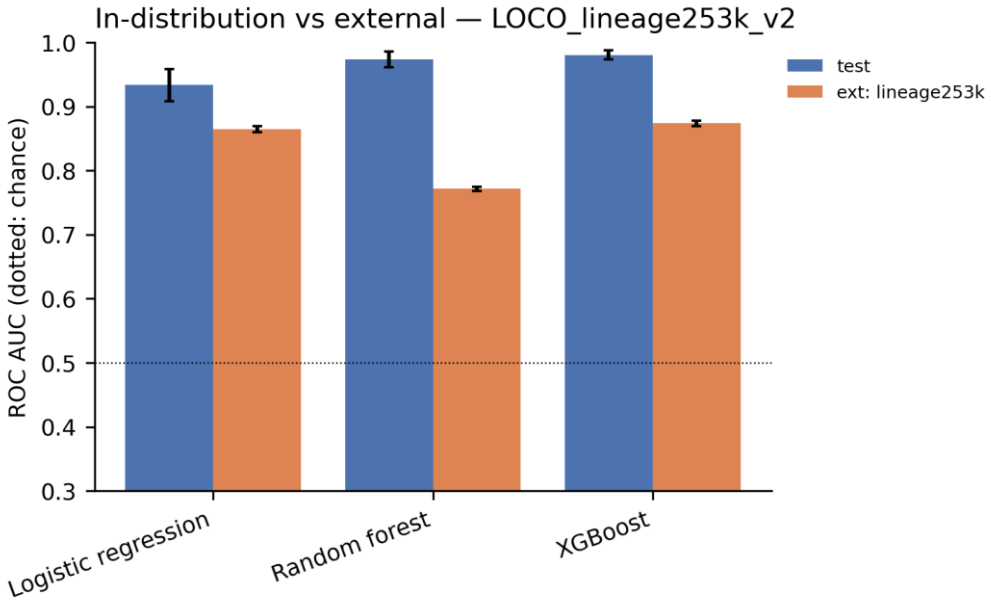


Fig. 7. Leave-one-corpus-out with corpus B held out: ROC-AUC on the pool's own test split and on the held-out corpus (lexical_v2, 3 seeds).

TABLE VI. Leave-one-corpus-out results available so far: models trained on the other five corpora and tested on the held-out corpus (mean ± sd over 3 seeds). Recall and FPR use the threshold chosen on pool validation data for FPR ≤ 1 %.

held_out	representation	model	seeds	pool test AUC	held-out AUC	held-out F1@0.5	held-out recall@valFPR1%	held-out FPR@valFPR 1%
phiusiil	v2	lr	3	0.921 ± 0.012	0.966 ± 0.006	0.750 ± 0.015	0.147 ± 0.122	0.000 ± 0.000
phiusiil	v2	rf	3	0.960 ± 0.001	0.891 ± 0.027	0.762 ± 0.009	0.251 ± 0.054	0.000 ± 0.000
phiusiil	v2	xgb	3	0.968 ± 0.002	0.977 ± 0.017	0.784 ± 0.004	0.422 ± 0.047	0.000 ± 0.000
phiusiil	v2noart	lr	3	0.917 ± 0.016	0.888 ± 0.007	0.766 ± 0.007	0.321 ± 0.031	0.000 ± 0.000
phiusiil	v2noart	rf	3	0.958 ± 0.001	0.866 ± 0.013	0.784 ± 0.010	0.321 ± 0.052	0.000 ± 0.000
phiusiil	v2noart	xgb	3	0.965 ± 0.003	0.878 ± 0.013	0.800 ± 0.001	0.424 ± 0.084	0.000 ± 0.000
lineage253k	v2	lr	3	0.934 ± 0.025	0.865 ± 0.004	0.660 ± 0.004	0.997 ± 0.001	0.946 ± 0.034
lineage253k	v2	rf	3	0.973 ± 0.012	0.772 ± 0.003	0.755 ± 0.001	0.421 ± 0.060	0.176 ± 0.021
lineage253k	v2	xgb	3	0.980 ± 0.007	0.874 ± 0.004	0.769 ± 0.003	0.709 ± 0.058	0.160 ± 0.091
lineage253k	v2noart	lr	3	0.936 ± 0.009	0.883 ± 0.003	0.710 ± 0.023	0.723 ± 0.126	0.152 ± 0.113
lineage253k	v2noart	rf	3	0.970 ± 0.013	0.780 ± 0.009	0.761 ± 0.003	0.377 ± 0.035	0.149 ± 0.022
lineage253k	v2noart	xgb	3	0.972 ± 0.010	0.849 ± 0.005	0.779 ± 0.005	0.694 ± 0.050	0.188 ± 0.027

6.6 Pending analyses

These are implemented and will run on the completed matrix; no result is claimed for them yet:

- random vs grouped splits on corpus B;
- `lexical_v2` and artifact ablations on both corpora;
- canonicalised features, in-corpus and LOCO (H1a);
- character CNN, canonicalised character CNN and hybrid (H4);
- paired bootstrap and McNemar comparisons (Appendix G, T6);
- post-hoc calibration under shift (Appendix G, T7; H2);
- label-preserving mutation flip rates (Appendix G, T8);
- temporal recall on PhishTank by year (H3; only 64 URLs predate 2021, so the analysis covers 2021–2026);
- SHAP and error analysis;
- computational cost (Appendix G, T9; two timings from 2026-10-11 are inflated by a system standby and are excluded).

7. Discussion

Provisional; restricted to the completed evidence.

What the high scores measure. On PhiUSIIL the original pipeline reaches ROC-AUC 0.999 whether or not registered domains are shared between training and test, and it then flags essentially all benign URLs of four independently collected corpora. With the probe results, the simplest explanation is that these models learn *how each class was recorded* — benign as `https://www.` homepages, phishing as everything else — rather than properties of phishing URLs. This is the shortcut-learning pitfall described by Arp et al. [13], here with named, measurable artifacts. It matches the benign-homepage effect reported for login URLs [12] and the `https://www.` shortcut reported in a recent preprint [46]. The original project's misclassification of `https://google.com/` is the same phenomenon seen through one URL.

Why cross-corpus results vary so much. The artifacts reverse sign across corpora (§6.2), so a model's external performance depends on whether the target corpus happens to share the training corpus's recording conventions. This explains the below-chance AUC on ISCX-2016 and the Kaggle corpus. It also explains why cross-dataset numbers in the literature range from near chance [9, 10] to moderate [39]: the outcome is largely a property of the dataset pair.

Model choice is secondary. In-distribution, the soft vote, XGBoost, random forest and logistic regression differ by at most 0.001 ROC-AUC. Externally they differ by up to 0.2, in an order unrelated to their in-distribution ranking. A single random split therefore says little about which model, ensemble or otherwise, to deploy.

Thresholds are the operational failure. Even where ranking transfers (LOCO with PhiUSIIL held out, AUC up to 0.977), a threshold chosen for a 1 % FPR budget on the training distribution transfers badly: it over-flags in one direction and under-flags in the other. Deployment would need threshold recalibration on traffic from the target environment, a step the public-benchmark protocol never exercises.

What is not established yet. Until the pending runs finish, it remains open whether canonicalised features or character models on canonical input reduce the external failure, whether random splits inflate results on domain-heavy corpora, and whether calibration repairs threshold transfer. All conclusions above come from balanced public corpora and the baseline representation.

8. Threats to Validity and Limitations

- **Original data unavailable.** The project's own 253,098-URL dataset could not be obtained. Conclusions about it rest on analogy with PhiUSIIL, whose benign class has the same `https://www.` form, and with a size-matched sample of its probable source lineage (corpus B).
- **"External" is not "real traffic".** Every external corpus is itself a public compilation with its own artifacts. A model that transferred perfectly between them would still not be validated on real user traffic, so these results show failure modes, not deployment performance.
- **Balanced prevalence.** All labelled corpora are roughly balanced; precision at realistic prevalence is derived analytically, not measured.
- **Temporal coverage.** Timestamped data are positive-only (PhishTank, URLhaus) and concentrated in 2021–2026. No temporally split benchmark with benign URLs was available.
- **Grouping granularity.** Registered-domain grouping does not remove near-duplicates that share a phishing kit across domains, or benign URLs of one organisation under several domains.
- **Label noise.** Labels are inherited from the sources (blacklists, curated lists); no URL was relabelled.
- **Mutations.** Tier B variants (scheme and `www.` toggles) are assumed, not proven, to preserve the label.
- **Model coverage and budget.** No transformer baseline was trained (CPU only), and character models use a short training budget and fewer seeds.

- **Post-hoc additions.** Canonicalised features and the leave-one-corpus-out design were added after the dataset probe and after inspecting the first baseline run. They are reported as such and use the same fixed hyper-parameters.
- **Incomplete runs.** Part of the experiment matrix is still running; §6 marks every pending item.

9. Conclusion

Provisional. Lexical phishing-URL detectors evaluated on a public corpus can reach ROC-AUC 0.999 while flagging 99–100 % of the benign URLs of four other public corpora. The cause we can name and measure is how each corpus recorded its classes — scheme, `www.` prefix and homepage-only benign URLs — whose predictive direction reverses between corpora, so the source corpus of a URL is predictable from lexical features far above chance. Training on several corpora restores much of the ranking ability on an unseen corpus but leaves operating thresholds unreliable.

Future work should evaluate URL detectors on benign traffic sampled the way deployed systems see it, canonicalise recording format before learning, report results across corpora and at fixed false-positive budgets, and treat threshold recalibration as part of deployment. The pending runs will establish whether canonicalisation and character-level models change these conclusions.

References

Numbered in order of first citation. All entries were verified against an authoritative page (DOI resolver, publisher, arXiv or proceedings site); see research/paper_metadata.csv.

[1] A. Oest et al., “Sunrise to Sunset: Analyzing the End-to-end Life Cycle and Effectiveness of Phishing Attacks at Scale,” in 29th USENIX Security Symposium (USENIX Security 20), 2020. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/oest-sunrise.

[2] A. Oest et al., “PhishTime: Continuous Longitudinal Measurement of the Effectiveness of Anti-phishing Blacklists,” in 29th USENIX Security Symposium (USENIX Security 20), pp. 379–396, 2020. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/oest-phishtime.

[3] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, “Beyond blacklists: learning to detect malicious web sites from suspicious URLs,” in Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '09), pp. 1245-1254, 2009, doi: 10.1145/1557019.1557153.

[4] A. Le, A. Markopoulou, and M. Faloutsos, “PhishDef: URL names say it all,” in 2011 Proceedings IEEE INFOCOM, pp. 191-195, 2011, doi: 10.1109/INFCOM.2011.5934995.

[5] M. S. I. Mamun, M. A. Rathore, A. H. Lashkari, N. Stakhanova, and A. A. Ghorbani, “Detecting Malicious URLs Using Lexical Analysis,” in Network and System Security (NSS 2016), LNCS 9955, pp. 467-482, Springer, 2016, doi: 10.1007/978-3-319-46298-1_30.

[6] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, “Machine learning based phishing detection from URLs,” Expert Systems with Applications, 2019, doi: 10.1016/j.eswa.2018.09.029.

[7] S. Sankaranarayanan, A. T. Sivachandran, A. S. Mohd Khairuddin, K. Hasikin, and A. R. Wahab Sait, “An ensemble classification method based on machine learning models for malicious Uniform Resource Locators (URL),” PLOS ONE, 2024, doi: 10.1371/journal.pone.0302196.

[8] H. A. Khan et al., “Explainable AI for phishing URL detection: a Bayesian-optimized stacking ensemble framework with SHAP-guided feature learning,” Frontiers in Artificial Intelligence, 2026, doi: 10.3389/frai.2026.1854934.

[9] M. Mia, D. Derakhshan, and M. M. A. Pritom, “Can Features for Phishing URL Detection Be Trusted Across Diverse Datasets? A Case Study with Explainable AI,” in Proceedings of the 11th International Conference on Networking, Systems, and Security (NSysS 2024), pp. 137–145, arXiv:2411.09813, 2024, doi: 10.1145/3704522.3704532.

[10] S. Henry, B. U.-A. Stephen, and P. Asuquo, “Ensemble transfer learning for cross-dataset phishing detection,” Franklin Open, 2026, doi: 10.1016/j.fraope.2026.100579.

[11] F. Rashid, B. Doyle, S. C. Han, and S. Seneviratne, “Phishing URL detection generalisation using Unsupervised Domain Adaptation,” Computer Networks, 2024, doi: 10.1016/j.comnet.2024.110398.

[12] M. Sanchez-Paniagua, E. Fidalgo Fernandez, E. Alegre, W. Al-Nabki, and V. Gonzalez-Castro, “Phishing URL Detection: A Real-Case Scenario Through Login URLs,” IEEE Access, 2022, doi: 10.1109/ACCESS.2022.3168681.

[13] D. Arp et al., “Dos and Don'ts of Machine Learning in Computer Security,” in 31st USENIX Security Symposium (USENIX Security 22), pp. 3971–3988, arXiv:2010.09470, 2022. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/arp.

[14] S. Marchal, and N. Asokan, “On Designing and Evaluating Phishing Webpage Detection Techniques for the Real World,” in 11th USENIX Workshop on Cyber Security Experimentation and Test (CSET 18), 2018. [Online]. Available: https://www.usenix.org/conference/cset18/presentation/marchal.

[15] A. E. Aassal, S. Baki, A. Das, and R. M. Verma, “An In-Depth Benchmarking and Evaluation of Phishing Detection Research for Security Needs,” IEEE Access, pp. 22170–22192, 2020, doi: 10.1109/ACCESS.2020.2969780.

[16] A. Hannousse, and S. Yahiouche, “Towards benchmark datasets for machine learning based website phishing detection: An experimental study,” Engineering Applications of Artificial Intelligence, arXiv:2010.12847, 2021, doi: 10.1016/j.engappai.2021.104347.

[17] B. Sabir, M. A. Babar, R. Gaire, and A. Abuadbba, “Reliability and Robustness analysis of Machine Learning based Phishing URL Detectors,” IEEE Transactions on Dependable and Secure Computing, arXiv:2005.08454, 2022, doi: 10.1109/TDSC.2022.3218043.

[18] D. K. McGrath, and M. Gupta, “Behind Phishing: An Examination of Phisher Modi Operandi,” in USENIX Workshop on Large-Scale Exploits and Emergent Threats (LEET '08), 2008. [Online]. Available: https://www.usenix.org/legacy/event/leet08/tech/full_papers/mcgrath/mcgrath.pdf.

[19] S. Garera, N. Provos, M. Chew, and A. D. Rubin, “A framework for detection and measurement of phishing attacks,” in Proceedings of the 2007 ACM Workshop on Recurring Malcode (WORM '07), pp. 1-8, 2007, doi: 10.1145/1314389.1314391.

[20] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, “Identifying suspicious URLs: an application of large-scale online learning,” in Proceedings of the 26th International Conference on Machine Learning (ICML '09), pp. 681-688, 2009, doi: 10.1145/1553374.1553462.

[21] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, “Learning to detect malicious URLs,” ACM Transactions on Intelligent Systems and Technology, 2011, doi: 10.1145/1961189.1961202.

[22] A. Blum, B. Wardman, T. Solorio, and G. Warner, “Lexical feature based phishing URL detection using online learning,” in Proceedings of the 3rd ACM Workshop on Artificial Intelligence and Security (AISec '10), pp. 54-60, 2010, doi: 10.1145/1866423.1866434.

[23] R. Verma, and K. Dyer, “On the Character of Phishing URLs: Accurate and Robust Statistical Learning Classifiers,” in Proceedings of the 5th ACM Conference on Data and Application Security and Privacy (CODASPY '15), pp. 111-122, 2015, doi: 10.1145/2699026.2699115.

[24] S. Marchal, J. Francois, R. State, and T. Engel, “PhishStorm: Detecting Phishing With Streaming Analytics,” IEEE Transactions on Network and Service Management, 2014, doi: 10.1109/TNSM.2014.2377295.

[25] A. Prasad, and S. Chandra, “PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning,” Computers & Security, 2024, doi: 10.1016/j.cose.2023.103545.

[26] R. S. Rao, T. Vaishnavi, and A. R. Pais, “CatchPhish: detection of phishing websites by inspecting URLs,” Journal of Ambient Intelligence and Humanized Computing, 2020, doi: 10.1007/s12652-019-01311-4.

[27] K. L. Chiew, C. L. Tan, K. Wong, K. S. C. Yong, and W. K. Tiong, “A new hybrid ensemble feature selection framework for machine learning-based phishing detection system,” Information Sciences, 2019, doi: 10.1016/j.ins.2019.01.064.

[28] J. Saxe, and K. Berlin, “eXpose: A Character-Level Convolutional Neural Network with Embeddings For Detecting Malicious URLs, File Paths and Registry Keys,” arXiv preprint arXiv:1702.08568, 2017. [Online]. Available: https://arxiv.org/abs/1702.08568.

[29] A. C. Bahnsen, E. C. Bohorquez, S. Villegas, J. Vargas, and F. A. Gonzalez, “Classifying phishing URLs using recurrent neural networks,” in 2017 APWG Symposium on Electronic Crime Research (eCrime), 2017, doi: 10.1109/ecrime.2017.7945048.

[30] H. Le, Q. Pham, D. Sahoo, and S. C. H. Hoi, “URLNet: Learning a URL Representation with Deep Learning for Malicious URL Detection,” arXiv preprint arXiv:1802.03162, 2018. [Online]. Available: https://arxiv.org/abs/1802.03162.

[31] F. Tajaddodianfar, J. W. Stokes, and A. Gururajan, “Texception: A Character/Word-Level Deep Learning Model for Phishing URL Detection,” in ICASSP 2020 - IEEE International Conference on Acoustics, Speech and Signal Processing, pp. 2857-2861, 2020, doi: 10.1109/ICASSP40776.2020.9053670.

[32] A. Aljofey, Q. Jiang, Q. Qu, M. Huang, and J.-P. Niyigena, “An Effective Phishing Detection Model Based on Character Level Convolutional Neural Network from URL,” Electronics, 2020, doi: 10.3390/electronics9091514.

[33] A. S. Bozkir, F. C. Dalgic, and M. Aydos, “GramBeddings: A New Neural Network for URL Based Identification of Phishing Web Pages Through N-gram Embeddings,” Computers & Security, 2023, doi: 10.1016/j.cose.2022.102964.

[34] O. K. Sahingoz, E. Buber, and E. Kugu, “DEPHIDES: Deep Learning Based Phishing Detection System,” IEEE Access, 2024, doi: 10.1109/ACCESS.2024.3352629.

[35] P. Maneriker, J. W. Stokes, E. G. Lazo, D. Carutasu, F. Tajaddodianfar, and A. Gururajan, “URLTran: Improving Phishing URL Detection Using Transformers,” in MILCOM 2021 - IEEE Military Communications Conference, pp. 197-204, 2021, doi: 10.1109/MILCOM52596.2021.9653028.

[36] E. M. Rudd, and A. Abdallah, “Training Transformers for Information Security Tasks: A Case Study on Malicious URL Prediction,” arXiv preprint arXiv:2011.03040, 2020. [Online]. Available: https://arxiv.org/abs/2011.03040.

[37] Y. Wang, W. Zhu, H. Xu, Z. Qin, K. Ren, and W. Ma, “A Large-Scale Pretrained Deep Model for Phishing URL Detection,” in ICASSP 2023 - IEEE International Conference on Acoustics, Speech and Signal Processing, pp. 1-5, 2023, doi: 10.1109/ICASSP49357.2023.10095719.

[38] Y. Li, Y. Liu, P. Li, Y. Jia, and Y. Wang, “Continuous Multi-Task Pre-training for Malicious URL Detection and Webpage Classification (URLBERT),” Computer Networks, 2025, doi: 10.1016/j.comnet.2025.111513.

[39] R. Liu et al., “TransURL: Improving malicious URL detection with multi-layer Transformer encoding and multi-scale pyramid features,” Computer Networks, 2024, doi: 10.1016/j.comnet.2024.110707.

[40] S. S. Roy, and S. Nilizadeh, “PhishLang: A Real-Time, Fully Client-Side Phishing Detection Framework Using MobileBERT,” arXiv:2408.05667 (v3, Apr 2025); CC BY 4.0, 2024. [Online]. Available: https://arxiv.org/abs/2408.05667.

[41] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, “TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time,” in 28th USENIX Security Symposium (USENIX Security 19), pp. 729–746, arXiv:1807.07838, 2019. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/pendlebury.

[42] G. Apruzzese, P. Laskov, and J. Schneider, “SoK: Pragmatic Assessment of Machine Learning for Network Intrusion Detection,” in 2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P), pp. 592–614, arXiv:2305.00550, 2023, doi: 10.1109/EuroSP57164.2023.00042.

[43] A. Das, S. Baki, A. E. Aassal, R. Verma, and A. Dunbar, “SoK: A Comprehensive Reexamination of Phishing Research From the Security Perspective,” IEEE Communications Surveys & Tutorials, arXiv:1911.00953, 2020, doi: 10.1109/COMST.2019.2957750.

[44] M. Alam et al., “SoK: PHILTER: Uncovering Security and Functional Gaps in AI-based Phishing Website Detection Literature via an LLM-based Reasoning Framework,” in 35th USENIX Security Symposium (USENIX Security 26), prepublication, 2026. [Online]. Available: https://www.usenix.org/system/files/conference/usenixsecurity26/sec26_prepub_alam.pdf.

[45] T. Dalton et al., “PhreshPhish: A Real-World, High-Quality, Large-Scale Phishing Website Dataset and Benchmark,” arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2507.10854.

[46] G. R. Giri, “ShieldPhish: Evaluating Representation Robustness and Cross-Source Generalization in Phishing URL Detection,” Zenodo preprint (submitted to IEEE), 2026, doi: 10.5281/zenodo.23141305.

[47] I. Bhuiyan, and T. Bhuiyan, “The Fragility of Phishing Detection Models: Evidence from Cross-Corpus Transfer, Prevalence Shift, Artifact Learning, and Evasion Risk,” Big Data and Cognitive Computing, 2026, doi: 10.3390/bdcc10070211.

[48] G. Apruzzese, M. Conti, and Y. Yuan, “SpacePhish: The Evasion-space of Adversarial Attacks against Phishing Website Detectors using Machine Learning,” in Proceedings of the 38th Annual Computer Security Applications Conference (ACSAC 2022), pp. 171–185, arXiv:2210.13660, 2022, doi: 10.1145/3564625.3567980.

[49] A. AlEroud, and G. Karabatis, “Bypassing Detection of URL-based Phishing Attacks Using Generative Adversarial Deep Neural Networks,” in Proceedings of the Sixth International Workshop on Security and Privacy Analytics (IWSPA '20), pp. 53-60, 2020, doi: 10.1145/3375708.3380315.

[50] G. Apruzzese, H. S. Anderson, S. Dambra, D. Freeman, F. Pierazzi, and K. A. Roundy, “"Real Attackers Don't Compute Gradients": Bridging the Gap Between Adversarial ML Research and Practice,” in 2023 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML), arXiv:2212.14315, 2023, doi: 10.1109/SaTML54575.2023.00031.

[51] S. M. Lundberg, and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” in Advances in Neural Information Processing Systems 30 (NeurIPS 2017), 2017. [Online]. Available: https://arxiv.org/abs/1705.07874.

[52] S. M. Lundberg et al., “From local explanations to global understanding with explainable AI for trees,” Nature Machine Intelligence 2(1):56-67, 2020, doi: 10.1038/s42256-019-0138-9.

[53] M. Al-Fayoumi, B. Alhijawi, Q. A. Al-Haija, and R. Armoush, “XAI-PhD: Fortifying Trust of Phishing URL Detection Empowered by Shapley Additive Explanations,” International Journal of Online and Biomedical Engineering (iJOE) 20(11):80-101, 2024, doi: 10.3991/ijoe.v20i11.49533.

[54] T. Kehkashan et al., “Explainable phishing website detection for secure and sustainable cyber infrastructure,” Scientific Reports 15, article 41751, 2025, doi: 10.1038/s41598-025-27984-w.

[55] D. Li, T. Qiu, S. Chen, Q. Li, and S. Xu, “Can We Leverage Predictive Uncertainty to Detect Dataset Shift and Adversarial Examples in Android Malware Detection?,” in Annual Computer Security Applications Conference (ACSAC '21), pp. 596-608, 2021, doi: 10.1145/3485832.3485916.

[56] A. Niculescu-Mizil, and R. Caruana, “Predicting good probabilities with supervised learning,” in Proceedings of the 22nd International Conference on Machine Learning (ICML '05), 2005, doi: 10.1145/1102351.1102430.

[57] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On Calibration of Modern Neural Networks,” in Proceedings of the 34th International Conference on Machine Learning (ICML 2017), PMLR 70, 2017. [Online]. Available: https://arxiv.org/abs/1706.04599.

[58] Manu Siddhartha (Kaggle user sid321axn), “Malicious URLs dataset (Kaggle: 'Huge dataset of 6,51,191 Malicious URLs'),” Kaggle dataset, 2021. [Online]. Available: https://www.kaggle.com/datasets/sid321axn/malicious-urls-dataset.

[59] R. Jordaney et al., “Transcend: Detecting Concept Drift in Malware Classification Models,” in 26th USENIX Security Symposium (USENIX Security 17), pp. 625–642, 2017. [Online]. Available: https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/jordaney.

[60] F. Barbero, F. Pendlebury, F. Pierazzi, and L. Cavallaro, “Transcending TRANSCEND: Revisiting Malware Classification in the Presence of Concept Drift,” in 2022 IEEE Symposium on Security and Privacy (S&P), arXiv:2010.03856, 2022, doi: 10.1109/SP46214.2022.9833659.

[61] Y. Chen, Z. Ding, and D. Wagner, “Continuous Learning for Android Malware Detection,” in 32nd USENIX Security Symposium (USENIX Security 23), pp. 1127–1144, arXiv:2302.04332, 2023. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/chen-yizheng.

[62] A. Ejaz, A. N. Mian, and S. Manzoor, “Life-long phishing attack detection using continual learning,” Scientific Reports, 2023, doi: 10.1038/s41598-023-37552-9.

[63] Z. Kan et al., “TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time (Extended Version),” arXiv preprint, 2024, doi: 10.48550/arXiv.2402.01359.

[64] S. Krishnan, and M. N. S, “An Efficient Framework for Phishing URL Detection Using Adaptive Drift-Aware Feature Monitoring and Online Learning,” Security and Privacy (Wiley), 2025, doi: 10.1002/spy2.70057.

[65] W. Fernando, and N. Komninos, “Evaluating and Combating the Impact of Concept Drift on the Performance of Machine Learning-Based Phishing Detection Systems,” arXiv preprint, 2026. [Online]. Available: https://arxiv.org/abs/2606.11471.

[66] M.-Y. Kan, and H. O. N. Thi, “Fast webpage classification using URL features,” in Proceedings of the 14th ACM International Conference on Information and Knowledge Management (CIKM '05), pp. 325-326, 2005, doi: 10.1145/1099554.1099649.

[67] P. Kolari, T. Finin, and A. Joshi, “SVMs for the Blogosphere: Blog Identification and Splog Detection,” in AAAI Spring Symposium on Computational Approaches to Analyzing Weblogs (2006), 2006. [Online]. Available: https://www.semanticscholar.org/paper/b54518d807f05ebe5ce39fb7e22f943cd70cb4f1.

[68] D. Sahoo, C. Liu, and S. C. H. Hoi, “Malicious URL Detection using Machine Learning: A Survey,” arXiv preprint arXiv:1701.07179 (v3, 2019), 2017. [Online]. Available: https://arxiv.org/abs/1701.07179.

[69] A. C. Bahnsen, I. Torroledo, L. D. Camacho, and S. Villegas, “DeepPhish: Simulating Malicious AI,” Black Hat Europe 2018 (talk, 5 Dec 2018, slides); also presented at DEF CON 26 AI Village, 2018. [Online]. Available: https://i.blackhat.com/eu-18/Wed-Dec-5/eu-18-CorreaBahnsen-DeepPhish-Simulating-Malicious-AI.pdf.

[70] A. Oest, Y. Safei, A. Doupe, G.-J. Ahn, B. Wardman, and G. Warner, “Inside a phisher's mind: Understanding the anti-phishing ecosystem through phishing kit analysis,” in 2018 APWG Symposium on Electronic Crime Research (eCrime), 2018, doi: 10.1109/ECRIME.2018.8376206.

[71] J. Lee, P. Ye, R. Liu, D. M. Divakaran, and M. C. Chan, “Building Robust Phishing Detection System: an Empirical Analysis,” in NDSS Workshop on Measurements, Attacks, and Defenses for the Web (MADWeb 2020), 2020. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2020/02/23007-paper.pdf.

[72] H. Hu, S. T. K. Jan, Y. Wang, and G. Wang, “Assessing Browser-level Defense against IDN-based Phishing,” in 30th USENIX Security Symposium, pp. 3739-3756, 2021. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/hu-hang.

[73] E. Nowroozi, Abhishek, M. Mohammadi, and M. Conti, “An Adversarial Attack Analysis on Malicious Advertisement URL Detection Framework,” arXiv:2204.13172 (preprint), 2022. [Online]. Available: https://arxiv.org/abs/2204.13172.

[74] P. Kintis et al., “Hiding in Plain Sight: A Longitudinal Study of Combosquatting Abuse,” in Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17), pp. 569-586, 2017, doi: 10.1145/3133956.3134002.

[75] K. Tian, S. T. K. Jan, H. Hu, D. Yao, and G. Wang, “Needle in a Haystack: Tracking Down Elite Phishing Domains in the Wild,” in Proceedings of the Internet Measurement Conference 2018 (IMC '18), Boston, MA, 2018, doi: 10.1145/3278532.3278569.

[76] Q. Scheitle et al., “A Long Way to the Top: Significance, Structure, and Stability of Internet Top Lists,” in Proceedings of the Internet Measurement Conference 2018 (IMC '18), arXiv:1805.11506, 2018, doi: 10.1145/3278532.3278574.

[77] V. L. Pochat, T. V. Goethem, S. Tajalizadehkhoob, M. Korczyński, and W. Joosen, “Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation,” in Network and Distributed System Security Symposium (NDSS 2019), arXiv:1806.01156, 2019, doi: 10.14722/ndss.2019.23386.

[78] M. Botacin, F. Ceschin, R. Sun, D. Oliveira, and A. Grégio, “Challenges and pitfalls in malware research,” Computers & Security, 2021, doi: 10.1016/j.cose.2021.102287.

[79] S. Dambra et al., “Decoding the Secrets of Machine Learning in Malware Classification: A Deep Dive into Datasets, Feature Extraction, and Model Performance,” in Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23), arXiv:2307.14657, 2023, doi: 10.1145/3576915.3616589.

[80] A. Kulkarni, V. Balachandran, D. M. Divakaran, and T. Das, “Mitigating Bias in Machine Learning Models for Phishing Webpage Detection,” arXiv preprint, 2024. [Online]. Available: https://arxiv.org/abs/2401.08363.

[81] M. Sameen, K. Han, and S. O. Hwang, “PhishHaven-An Efficient Real-Time AI Phishing URLs Detection System,” IEEE Access, 2020, doi: 10.1109/ACCESS.2020.2991403.

[82] S. Jalil, M. Usman, and A. Fong, “Highly accurate phishing URL detection based on machine learning,” Journal of Ambient Intelligence and Humanized Computing, 2023, doi: 10.1007/s12652-022-04426-3.

[83] L. Yi, A. Omotosho, and H. Balogun, “Phishing URL Detection and Interpretability With Machine Learning: A Cross-Dataset Approach,” Security and Privacy (Wiley), 2026, doi: 10.1002/spy2.70175.

[84] A. Aljofey et al., “An effective detection approach for phishing websites using URL and HTML features,” Scientific Reports, 2022, doi: 10.1038/s41598-022-10841-5.

[85] Y. Huang, Q. Yang, J. Qin, and W. Wen, “Phishing URL Detection via CNN and Attention-Based Hierarchical RNN,” in 2019 18th IEEE International Conference on Trust, Security and Privacy in Computing and Communications / 13th IEEE International Conference on Big Data Science and Engineering (TrustCom/BigDataSE), pp. 112-119, 2019, doi: 10.1109/TrustCom/BigDataSE.2019.00024.

[86] W. Wang, F. Zhang, X. Luo, and S. Zhang, “PDRCNN: Precise Phishing Detection with Recurrent Convolutional Neural Networks,” Security and Communication Networks, 2019, doi: 10.1155/2019/2595794.

[87] W. Wei, Q. Ke, J. Nowak, M. Korytkowski, R. Scherer, and M. Wozniak, “Accurate and fast URL phishing detector: A convolutional neural network approach,” Computer Networks, 2020, doi: 10.1016/j.comnet.2020.107275.

[88] Z. Alshingiti, R. Alaqel, J. Al-Muhtadi, Q. E. U. Haq, K. Saleem, and M. H. Faheem, “A Deep Learning-Based Phishing Detection System Using CNN, LSTM, and LSTM-CNN,” Electronics, 2023, doi: 10.3390/electronics12010232.

[89] A. Ozcan, C. Catal, E. Donmez, and B. Senturk, “A hybrid DNN-LSTM model for detecting phishing URLs,” Neural Computing and Applications, 2023, doi: 10.1007/s00521-021-06401-z.

[90] H. Ghalechyan, E. Israyelyan, A. Arakelyan, G. Hovhannisyan, and A. Davtyan, “Phishing URL detection with neural networks: an empirical study,” Scientific Reports, 2024, doi: 10.1038/s41598-024-74725-6.

[91] K. Haynes, H. Shirazi, and I. Ray, “Lightweight URL-based phishing detection using natural language processing transformers for mobile devices,” in Procedia Computer Science, 191, pp. 127-134 (FNC/MobiSPC 2021), 2021, doi: 10.1016/j.procs.2021.07.040.

[92] F. Trad, and A. Chehab, “Prompt Engineering or Fine-Tuning? A Case Study on Phishing Detection with Large Language Models,” Machine Learning and Knowledge Extraction, 2024, doi: 10.3390/make6010018.

[93] I. Ali, and B. Subba, “PhishURLDetect: A parameter efficient fine-tuning of LLMs using LoRA for detection of phishing URLs,” in Proceedings of the 26th International Conference on Distributed Computing and Networking (ICDCN 2025), pp. 278-279, 2025, doi: 10.1145/3700838.3703658.

[94] H. Kibriya, R. Amin, S. S. Alshamrani, S. Rehman, M. Hassan, and F. S. Alsubaei, “Lightweight malicious URL detection using deep learning and large language models,” Scientific Reports, 2025, doi: 10.1038/s41598-025-26653-2.

[95] R. Liu et al., “PMANet: Malicious URL detection via post-trained language model guided multi-level feature attention network,” Information Fusion, 2025, doi: 10.1016/j.inffus.2024.102638.

[96] Y. Tian, Y. Yu, J. Sun, and Y. Wang, “From Past to Present: A Survey of Malicious URL Detection Techniques, Datasets and Code Repositories,” arXiv preprint arXiv:2504.16449 (v2, under review at Computer Science Review), 2025. [Online]. Available: https://arxiv.org/abs/2504.16449.

[97] X. Chen, H. Liu, T. Yuan, M. Kafai, P. Habas, and X. Zhang, “MemoPhishAgent: Memory-Augmented Multi-Modal LLM Agent for Phishing URL Detection,” arXiv preprint arXiv:2602.21394 (v3; arXiv comment states ACL 2026 Industry Track camera-ready - venue not independently verified), 2026. [Online]. Available: https://arxiv.org/abs/2602.21394.

[98] H. Trikilis, P. Marasinghe, F. Rashid, and S. Seneviratne, “Eliciting Least-to-Most Reasoning for Phishing URL Detection,” arXiv preprint arXiv:2601.20270 (short paper), 2026. [Online]. Available: https://arxiv.org/abs/2601.20270.

[99] Tarun Tiwari (Kaggle user taruntiwarihp), “Phishing Site URLs (Kaggle: 'Malicious and Phishing attacks ulrs'),” Kaggle dataset, 2020. [Online]. Available: https://www.kaggle.com/datasets/taruntiwarihp/phishing-site-urls.

[100] A. Internet), “Alexa.com Top Sites retirement (1 May 2022),” Alexa.com end-of-service notice; press coverage, 2022. [Online]. Available: https://www.alexa.com/minisiteinfo/.

[101] C. C. Foundation, “Common Crawl web corpus and URL index (benign URL source),” https://commoncrawl.org/overview, 2026. [Online]. Available: https://commoncrawl.org/overview.

[102] OpenPhish, “OpenPhish Community Feed,” https://openphish.com/phishing_feeds.html, 2026. [Online]. Available: https://openphish.com/phishing_feeds.html.

[103] Cisco Talos / OpenDNS (community), “PhishTank verified online phishing URL database,” https://phishtank.org/developer_info.php, 2026. [Online]. Available: https://phishtank.org/developer_info.php.

[104] a. Spamhaus, “URLhaus (abuse.ch) malware URL feed,” https://urlhaus.abuse.ch/api/, 2026. [Online]. Available: https://urlhaus.abuse.ch/api/.

[105] S. Axelsson, “The base-rate fallacy and the difficulty of intrusion detection,” ACM Transactions on Information and System Security, 2000, doi: 10.1145/357830.357849.

[106] R. Sommer, and V. Paxson, “Outside the Closed World: On Using Machine Learning for Network Intrusion Detection,” in 2010 IEEE Symposium on Security and Privacy, 2010, doi: 10.1109/SP.2010.25.

[107] V. Zeng, S. Baki, A. E. Aassal, R. M. Verma, L. F. T. D. Moraes, and A. Das, “Diverse Datasets and a Customizable Benchmarking Framework for Phishing,” in Proceedings of the Sixth International Workshop on Security and Privacy Analytics (IWSPA '20), 2020, doi: 10.1145/3375708.3380313.

[108] R. Verma, and A. Das, “What's in a URL: Fast Feature Extraction and Malicious URL Detection,” in Proceedings of the 3rd ACM International Workshop on Security and Privacy Analytics (IWSPA '17), pp. 55-63, 2017, doi: 10.1145/3041008.3041016.

[109] L. Tang, and Q. H. Mahmoud, “A Survey of Machine Learning-Based Solutions for Phishing Website Detection,” Machine Learning and Knowledge Extraction, pp. 672–694, 2021, doi: 10.3390/make3030034.

[110] N. Q. Do, A. Selamat, O. Krejcar, E. Herrera-Viedma, and H. Fujita, “Deep Learning for Phishing Detection: Taxonomy, Current Challenges and Future Directions,” IEEE Access, pp. 36429–36463, 2022, doi: 10.1109/ACCESS.2022.3151903.

[111] R. Zieni, L. Massari, and M. C. Calzarossa, “Phishing or Not Phishing? A Survey on the Detection of Phishing Websites,” IEEE Access, pp. 18499–18519, 2023, doi: 10.1109/ACCESS.2023.3247135.

Appendix A. Literature Survey Table

One row per verified source (111 sources after removing three duplicate records of the same DOI). Numbers in the 'Metrics reported' column are the source authors' own and are not comparable across rows (datasets, splits, class ratios and metrics differ). Access: F = full text read, A = abstract or publisher page only, P = paywalled (metadata only). Full notes for every source are kept in the repository under research/reading_notes/.

TABLE A1. Survey of verified literature (sorted by theme and year).

Ref.	Author (year); venue	Research objective	Method	Dataset	Evaluation protocol	Metrics reported	Main findings	Limitations	Research gap	Rel. / access
[59]	Jordaney (2017); 2017 / 26th USENIX Security Symposium	Deployed classifiers silently age under concept drift; how can a model know that its prediction is no longer supported by its training data?	Conformal-evaluator framework: non-conformity measures and p-value-based credibility/confidence to detect when a deployed classifier's decisions are no longer supported by trainin…	Android (Drebin and Marvin datasets) and Windows malware families (Sec. 5)	Train on older data, test on newer; measures how rejection thresholds trade off kept-accuracy vs rejected fraction	Shows that rejecting low-credibility predictions restores high precision on drifted Android/Windows data (Sec. 5 tables; full text).	Statistical assessment of prediction quality can flag aging models before accuracy visibly collapses; ties drift detection to a decision to abstain.	**Authors' stated:** Calibration set requirements and computational cost of conformal p-values; threshold selection needs validation data. **Our caveats:** Not evaluated on URLs; conformal credibilit…	Abstention/rejection as an answer to drift is almost absent from phishing-URL papers.	3 / F
[41]	Pendlebury (2019); 2019 / 28th USENIX Security Symposium	Reported malware-classifier accuracy is inflated by unrealistic class ratios (spatial bias) and by train/test splits that leak future knowledge (temporal bias).	Evaluation framework; constraints C1 (temporal training consistency), C2 (temporal goodware/malware window consistency), C3 (realistic malware-to-goodware ratio in testing); new m…	Android apps: ~129k samples (AndroZoo) spanning 2014-2016, labelled via VirusTotal (details Sec. 2.2); ~10% malware in realistic setting	Time-aware splits with monthly test periods over 24 months; compares 10-fold CV (biased) vs temporally consistent evaluation; varies malwar…	With temporal bias removed, AUT(F1,24m) = 0.58 (ALG1), 0.32 (ALG2), 0.64 (DL), all lower than the 10-fold F1 reported in the original papers (Fig. 5, Sec. 4; p. 736 of proceedings). Realistic 10% mal…	Random k-fold splits leak future knowledge (temporal bias) and unrealistic class ratios (spatial bias) jointly inflate reported malware-detection performance; sound evaluation needs time-ordered spli…	**Authors' stated:** Android-only evaluation; labelling via VirusTotal thresholds is noisy; ground-truth timestamps can be unreliable; dataset ends 2016. **Our caveats:** Phishing URLs have much shor…	No TESSERACT-style constraint set exists for phishing URL datasets; most public URL corpora carry no first-seen timesta…	5 / F
[2]	Oest (2020); 2020 / 29th USENIX Security Symposium	How fast, complete and consistent are anti-phishing blacklists, and how easily are they evaded?	Controlled, longitudinal experiments measuring blacklist speed, coverage and consistency; reproduces evasion techniques seen in the wild	2,862 researcher-deployed, innocuous PayPal-branded phishing sites across six deployments over nine months	Time-to-blacklist measurement per browser/platform; evasion vs. non-evasion arms	2,862 sites launched; identifies a JavaScript/behaviour-based evasion that blacklists failed to detect; mobile protections lag desktop (abstract; Sec. 6-7).	Ground-truth sources such as blacklists are slow, inconsistent, and evasion-sensitive; the labels researchers harvest from them are therefore delayed and biased toward unevasive phish.	**Authors' stated:** Artificial sites; single brand; nine-month window. **Our caveats:** Implication for us: PhishTank-style labels over-represent the easy-to-detect phish, so our test set is easier …	Label-source bias in phishing URL datasets is rarely quantified.	3 / F
[1]	Oest (2020); 2020 / 29th USENIX Security Symposium	The end-to-end timeline of real phishing campaigns, and how much harm occurs before defences react, was unknown.	Large-scale measurement of the phishing life cycle: deployment, email lure, victim visits, detection by the ecosystem, account compromise	One year of victim traffic to phishing pages impersonating a major financial brand (PayPal), via the 'Golden Hour' passive monitoring framework; 4.8M victim vi…	Longitudinal measurement; timing analysis of campaign phases relative to blacklist detection	4.8 million victims recorded in one year; average campaign lasts 21 hours from start to last victim; at least 7.42% of visitors submit credentials; a small set of campaigns causes 89.13% of victims (…	Phishing sites are extremely short-lived and most harm happens before blacklists react ('golden hour'); detection must be fast and must work on never-before-seen URLs/domains.	**Authors' stated:** Single brand/ecosystem vantage point; cloaking limits visibility. **Our caveats:** Not about ML evaluation per se, but its timing data undermines any evaluation that lets near-du…	Evaluation protocols rarely simulate the 'first hours' regime where the model is the only defence.	4 / F
[60]	Barbero (2022); 2022 / 2022 IEEE Symposium on Security and Privacy	Conformal rejection (Transcend) was empirically useful but lacked a formal basis and was computationally expensive; how to reject drifting samples efficiently …	TRANSCENDENT: formal treatment of conformal evaluation; two new conformal evaluators (inductive, cross-conformal) that are cheaper than the original transductive one; classificati…	Android malware over time (AndroZoo-derived, time-stamped) and other drift settings (Sec. VI)	Time-aware train/test with rejection; compares rejection quality against baselines (probability thresholds, etc.)	Reported rejection-vs-performance curves show TRANSCENDENT outperforms the original Transcend and probability-based rejection (Sec. VII; full text).	Rejection via conformal evaluation is a principled, efficient way to cope with drift; naive softmax/probability thresholds are weaker.	**Authors' stated:** Requires periodic calibration; cost of rejected samples must be absorbed by analysts. **Our caveats:** Benefit for lightweight URL models is untested.	Conformal rejection for URL classifiers remains open.	3 / F
[61]	Chen (2023); 2023 / 32nd USENIX Security Symposium	Android malware classifiers decay rapidly after deployment; how to keep them accurate with a limited labelling budget?	Hierarchical contrastive learning plus a new active-learning sample-selection strategy to maintain a drifting classifier with limited analyst labels	Android apps over a 7-year period (AndroZoo-based, time-stamped)	Train on one year, deploy over subsequent months/years with a monthly labelling budget; compares against prior active-learning baselines	F1 of a static classifier drops from 0.99 to 0.76 after six months on new samples (Sec. 1); proposed method lowers FNR from 14% to 9% and FPR from 0.86% to 0.48% versus prior active learning (abstrac…	Even strong classifiers decay fast under drift; the choice of which new samples to label matters as much as the model.	**Authors' stated:** Relies on timely analyst labels; Android-only. **Our caveats:** Transferring to URLs needs a time-stamped phishing feed; our dataset has none.	No equivalent labelling-budget study for phishing URLs.	3 / F
[62]	Ejaz (2023); 2023 / Scientific Reports	Static phishing classifiers trained on historical pages lose accuracy on future pages; can continual learning sustain performance over years?	Continual-learning (CL) algorithms on a vanilla neural network over FastText embeddings of HTML content, compared with static training and transfer learning	Self-collected phishing and benign web pages for 2018, 2019, 2020, split into six temporally ordered datasets	Train on earlier period, test on later periods; measures deterioration over time	CL keeps accuracy over time with 2.45% deterioration, versus 20.65% (static VNN) and 8% (transfer learning) (abstract).	A static phishing classifier loses over 20 percentage points across three years; incremental adaptation reduces but does not eliminate the loss.	**Authors' stated:** HTML-based, relatively small yearly samples; CL hyper-parameters. **Our caveats:** Content-based, not URL-based; still one of the few phishing papers with a genuine temporal spli…	Equivalent temporal study for lexical URL models.	3 / F
[63]	Kan (2024); 2024 / arXiv preprint	Same as TESSERACT, extended to Windows PE and PDF domains and to drift-mitigation strategies.	Extended TESSERACT: constraints, AUT metric, training-data tuning, plus delay-decay mitigation strategies (active learning, rejection, incremental retraining)	Android: 259,230 samples over five years (abstract); additional case studies on Windows PE and PDF malware	Time-aware evaluation with AUT over multi-year windows; cross-domain (Android, PE, PDF)	Abstract states reported F1 'upwards of 0.99' in prior work are inflated under realistic settings; per-domain AUT curves in Sec. 6-7 (full text available).	Spatial and temporal bias generalise beyond Android; periodic tuning and drift-mitigation strategies delay but do not remove decay.	**Authors' stated:** Preprint, not yet peer-reviewed in this extended form; malware-focused. **Our caveats:** Still no URL/phishing case study; the mitigation comparison assumes	Extending the PE/PDF case-study methodology to phishing URL feeds is open.	3 / F

Ref.	Author (year); venue	Research objective	Method	Dataset	Evaluation protocol	Metrics reported	Main findings	Limitations	Research gap	Rel. / access
			compared across doma…					labelled fresh data i…		
[64]	Krishnan (2025); 2025 / Security and Privacy	Phishing URL feature distributions drift, and frequent full retraining is costly.	Kolmogorov-Smirnov drift detection on feature distributions plus online SGD updates	Real-time phishing URLs from PhishTank (abstract); benign source not stated in abstract	Not specified in abstract beyond 'validation using real-time phishing URLs from PhishTank'	99.85% accuracy, 0.067% false-positive rate (abstract). (Full text not accessed; numbers restricted to the abstract/landing page.)	Claims that drift-aware online learning sustains near-perfect accuracy.	**Authors' stated:** Not visible from abstract. **Our caveats:** A 99.85%/0.067% result on a PhishTank-centred set is exactly the kind of figure TESSERACT/Arp et al. warn is inflated; no cross-source…	Independent, source-disjoint validation of drift-aware URL detectors.	2 / A
[65]	Fernando (2026); 2026 / arXiv preprint	ML phishing (email) detectors degrade as attacks evolve; how to quantify drift and reduce the degradation?	Statistical framework (adapted from the FeSAD ransomware drift framework) to quantify similarity of phishing vs benign samples and flag drifted samples; feature-robustness analysi…	Public phishing/spam email corpora (the paper is email-centric, not URL-centric; see PDF Sec. 2-4)	Temporal evaluation across dataset periods with drift scoring	Full text available, but headline numbers are not summarised in the abstract; treat quantitative claims cautiously (preprint).	Argues that phishing detectors degrade with evolution of attacks and that a drift score can be used to decide re-training.	**Authors' stated:** Preprint; limited datasets. **Our caveats:** Email modality; relevance to URL lexical models is indirect. Included to show that 2026 work still frames phishing drift mainly throu…	URL-level drift quantification with public time-stamped data.	2 / F
[66]	Kan (2005); 2005 / Proceedings of the 14th ACM International Conference on Inf…	Classify web pages by topic using only the URL, avoiding fetching page content.	Segments URLs into meaningful tokens, adds component (which URL part), sequential (order) and orthographic features; supervised maximum-entropy modelling.	Two standardized webpage-classification domains (topical classification); sizes not in abstract	Standard topical-classification benchmarks; split details not in abstract	Abstract states URL-only methods 'approach the performance' of full-text/link-based methods in certain scenarios; no numbers available without full text. (Only abstract/publisher-page information was…	Early evidence that URL tokens carry strong signal for page classification; introduced the token-segmentation idea reused by later phishing-URL work.	- Stated by authors: Not available (abstract only). - Our own caveats: Topic classification, not security; 2-page poster; no adversarial setting.	Pre-dates phishing-specific URL work; no temporal or adversarial evaluation.	2 / A
[67]	Kolari (2006); 2006 / AAAI Spring Symposium on Computational Approaches to Analyz…	Identify blogs and detect spam blogs (splogs).	SVM classifiers over textual and URL-derived feature sets; URL tokens shown useful.	Blog/splog samples (sizes not retrieved)	Not retrieved	Not retrieved. (Only abstract/publisher-page information was available; no additional numbers should be attributed to this paper without checking the full text.)	URL-token features contribute to spam-blog detection (often cited as early use of URL lexical features).	- Stated by authors: Not retrieved. - Our own caveats: Peripheral to phishing; included only for lineage.	n/a	1 / A
[19]	Garera (2007); 2007 / Proceedings of the 2007 ACM Workshop on Recurring Malcode	Tell phishing URLs from benign ones without page content; measure phishing prevalence at scale.	Taxonomy of URL obfuscation (IP host, redirection, brand in host/path, etc.); features incl. page rank and host-name/path heuristics; logistic regression filter; used for large-sc…	Google-internal URL data; 'several million URLs' measured (abstract); labelled set size not in abstract	Not stated in abstract (Ma et al. KDD 2009 summarise it as 97.3% accuracy over 2,500 URLs)	Accuracy 97.3% over 2,500 URLs is how Ma et al. (KDD 2009, related-work section) summarise this paper; the abstract itself gives no number. (Only abstract/publisher-page information was available; no…	Phishing URLs are often identifiable from their structure alone; brand names embedded in host/path are strong cues.	- Stated by authors: Not available (abstract only). - Our own caveats: Small labelled set; proprietary Google features (page rank) not reproducible; 2007 URL ecosystem.	No public dataset; no robustness analysis.	3 / A
[18]	McGrath (2008); 2008 / USENIX Workshop on Large-Scale Exploits and Emergent Threat…	Characterise how phishers construct URLs/domains and host sites.	Compares phishing URL feeds against DMOZ: URL/domain length distributions, character/vowel distributions, brand-name inclusion, subdomain use, IP hosts, shorteners; domain registr…	PhishTank and MarkMonitor real-time phishing feeds (>2 months, 2007-08), MarkMonitor-2006 archive; DMOZ URLs as benign comparison; whois/zone files	Descriptive statistics; no classification	URL length modes: 22 chars (DMOZ) vs 67 (PhishTank) vs 107 (MarkMonitor) (Sec. 3, Fig. 1); domain length mode 7 vs 10 chars (Fig. 2); 50-75% of phishing URLs contain target brand (Table 3); >3/4 cont…	Length, vowel ratio, character-diversity and brand-in-URL are strongly discriminative; phishing domains are short-lived.	- Stated by authors: Preliminary work; feeds biased toward reported phishing. - Our own caveats: 2007-era statistics; DMOZ as 'benign' is homepage-heavy (same shortcut risk as later datasets).	No learning or temporal-drift evaluation.	4 / F
[3]	Ma (2009); 2009 / Proceedings of the 15th ACM SIGKDD International Conference…	Classify URLs as malicious (phishing/spam) vs benign from URL-derived features without visiting the page.	Automatically extracts tens of thousands of binary lexical tokens (hostname and path tokens, delimiters) plus host-based properties; compares feature groups; L1-LR for sparse inte…	Benign: 15,000 URLs each from Yahoo random-URL feed and DMOZ; malicious: 5,500 PhishTank + Spamscatter URLs; 4 benign/malicious pairings, 20k-30k URLs each (Se…	10 random 50/50 splits per dataset; also cross-source training/testing (Table 4)	Overall accuracy 95-99% (abstract); error rates 1.2-6% for full feature set and 1.1-3.4% without WHOIS/blacklist features (Sec. 4.2, Fig. 2); lexical-only about 15% error at best vs 10% for host-only…	Lexical and host features are complementary; automatically generated token features beat hand-chosen ones; models trained on one data source generalise poorly to another.	- Stated by authors: Features assume infrastructure lookups (WHOIS/DNS) are available; redirection and short URLs not handled; distribution shift between sources acknowledged (Sec. 4.4). - Our own ca…	Cross-source generalisation and temporal robustness left largely open.	5 / F
[20]	Ma (2009); 2009 / Proceedings of the 26th International Conference on Machine…	Scale URL classification to continuous, non-stationary streams where batch retraining is infeasible.	Real-time feature collection system; sparse high-dimensional lexical+host features (grows to millions); compares online algorithms with continuous retraining to batch models retra…	Live feed from a large web-mail provider: about 20,000 URLs/day (6-7k malicious, about 2:1 benign:malicious) over 100 days, nearly 2 million URLs (Sec. 4.1)	Temporal: train on days up to t, test on day t+1; cumulative error over 100 days (Fig. 3-4)	Up to 99% accuracy on balanced data (abstract); CW online learning tracks batch accuracy with cumulative error about 1-2% while stale batch models degrade (Fig. 3-4; per-day false negatives up to 15%…	Continuous retraining matters: feature distribution of malicious URLs drifts daily; online CW matches batch at a fraction of cost.	- Stated by authors: Depends on a proprietary labelled feed; host features require live lookups. - Our own caveats: Day-ahead protocol is strong, but labels come from the same provider's filters (pot…	Public reproducible temporal benchmarks absent.	5 / F
[22]	Blum (2010); 2010 / Proceedings of the 3rd ACM Workshop on Artificial Intellige…	Detect present and emerging phishing domains quickly using URL lexical features and online updates.	Lexical bag-of-tokens features (host/path/binary indicators) fed to a confidence-weighted (CW) online classifier so the model adapts as new phishing campaigns appear.	UAB phishing URLs and benign URLs (sizes not in abstract)	Not stated in abstract	Not in abstract (often cited as about 97% accuracy; unverified here). (Only abstract/publisher-page information was available; no additional numbers should be attributed to this paper without checkin…	Lexical-only online learning is viable for zero-hour phishing detection.	- Stated by authors: Not available. - Our own caveats: Small, non-public dataset; no cross-dataset test.	Reproducibility; feature drift analysis.	3 / A
[4]	Le (2011); 2011 /	Client-side, proactive	Selects lexical features resistant	PhishTank (4,082 verified	Online cumulative error	Lexical-only accuracy 96-98%,	Lexical features suffice for	- Stated by authors: Small	Robustness to	4 / F

Ref.	Author (year); venue	Research objective	Method	Dataset	Evaluation protocol	Metrics reported	Main findings	Limitations	Research gap	Rel. / access
	2011 Proceedings IEEE INFOCOM, pp. 191-195	phishing-URL classification resilient to URL obfuscation and noisy training labels.	to obfuscation (host/path tokens, lengths, counts), compares lexical-only vs full (host/WHOIS) features, proposes AROW online learning; PhishDef…	phishing, June 2010), MalwarePatrol (2,001), Yahoo random (4,143) and DMOZ random (4,012) URLs; 5 pairings (Table I)	over batches of 400 URLs; label-noise injection experiments	within about 1% of full features (Sec. V, Table II); AROW reduces error up to 27% vs CW on DMOZ-Malware (Table III); about 95% accuracy with 5-10% label noise (Sec. V-C)…	practical purposes; online AROW is robust to mislabelled data.	datasets; obfuscation list may be incomplete. - Our own caveats: Benign classes are directory/random homepages; no temporal split.	adversarial token manipulation beyond listed obfuscations.	
[21]	Ma (2011); 2011 / ACM Transactions on Intelligent Systems and Technology, 2(3…	Journal synthesis: large-scale, real-time URL classification with lexical and host features.	Unifies the batch (KDD'09) and online (ICML'09) studies; adds analysis of feature importance and system considerations.	Consolidates the KDD/ICML datasets: about 2M-URL web-mail feed over 100 days plus directory/blacklist pairings	Random splits and day-ahead temporal evaluation (as in the conference versions)	Abstract-level only here; conference versions report 95-99% accuracy and about 1-2% cumulative error online. (Only abstract/publisher-page information was available; no additional numbers should be a…	Same conclusions as the conference papers with extended analysis.	- Stated by authors: See conference versions. - Our own caveats: Content assessed via metadata and the conference papers; article itself paywalled.	As above.	4 / A
[24]	Marchal (2014); 2014 / IEEE Transactions on Network and Service Management, 11(4),…	Real-time phishing URL rating for proxies/mail servers using URL-word relatedness.	Splits URL into registered domain (mld.ps) vs rest; computes relatedness among words using search-engine query logs; features on word counts, brand presence, Alexa rank; ML classi…	96,018 URLs: 48,009 phishing (PhishTank, from 53,089 unique) + 48,009 legitimate (DMOZ); released as urlset.csv (Sec. VI-A)	10-fold cross-validation, 100 runs (Sec. VI-B)	Accuracy 94.91% with 1.44% FPR after threshold tuning (abstract; Sec. VI-C), 95.66% at default threshold; features 1-9 alone 93.48% (Sec. VI-C). (Numbers taken from the full text; section/table refer…	Legitimate URLs show coherent word relatedness across domain and path; phishing URLs do not.	- Stated by authors: Depends on search-engine query APIs; relatedness changes over time. - Our own caveats: The released 'PhishStorm' dataset is widely reused (e.g., in Kaggle mixes); DMOZ benign cla…	External-API dependence conflicts with offline/lexical deployment.	3 / F
[23]	Verma (2015); 2015 / Proceedings of the 5th ACM Conference on Data and Applicati…	Improve accuracy/robustness of phishing URL classification with character-level statistical features.	Adds distributional features: KS statistic between URL character distribution and reference distributions, plus classic lexical features; evaluates several learners.	Phishing (PhishTank) and legitimate URLs; sizes not in abstract	Not in abstract	99.3% accuracy with FPR < 0.4% (abstract). (Only abstract/publisher-page information was available; no additional numbers should be attributed to this paper without checking the full text.)	Character-level statistics boost accuracy over token-only lexical features.	- Stated by authors: Not available. - Our own caveats: Very high accuracy likely dataset-dependent; no cross-dataset evidence.	Robustness claims untested over time.	4 / A
[68]	Sahoo (2017); 2017 / arXiv preprint arXiv:1701.07179	Structure the literature on ML-based malicious URL detection.	Defines the learning problem, surveys feature families and learning algorithms, discusses system design and open challenges.	Survey (no new data); reviews datasets up to 2.4M URLs (Ma et al.)	n/a (survey)	n/a (Numbers taken from the full text; section/table references given where identified.)	Lexical BoW features scale best; online learning suits drift; deep representation learning was emerging (precursor to URLNet).	- Stated by authors: Points out lack of large public benchmarks, concept drift, adversarial evasion, and short-lived URLs (open-challenges section). - Our own caveats: Pre-dates transformers; benchma…	Standardised benchmarks and temporal evaluation protocols.	4 / F
[6]	Sahingoz (2019); 2019 / Expert Systems with Applications, 117, pp. 345-357	Real-time, language-independent, third-party-free phishing URL detection.	Word-level tokenisation of URLs; 'NLP features' (brand/keyword, random-word, digit/length statistics), word vectors and hybrid sets; compares seven classifiers.	Ebbu2017: 73,575 URLs (36,400 legitimate via Yandex queries, 37,175 phishing from PhishTank) - figures as reported by Sanchez-Paniagua 2022 and DEPHIDES 2024, …	Not confirmed from full text	97.98% accuracy with RF on NLP features (abstract). (Only abstract/publisher-page information was available; no additional numbers should be attributed to this paper without checking the full text.)	Random Forest on NLP-style lexical features performs best; dataset released and widely reused.	- Stated by authors: Not available. - Our own caveats: Legitimate URLs from Yandex search results (deeper pages) vs PhishTank; later work (Sanchez-Paniagua 2022) shows models trained on this era degr…	Temporal validity; artifact analysis.	4 / A
[69]	Bahnsen (2018); 2018 / Black Hat Europe 2018	Can attackers use AI to craft URLs that evade AI-based phishing detectors?	Cluster PhishTank URLs by threat actor, train LSTM per actor on URLs that bypassed the detector, generate new URLs, measure evasion.	PhishTank-derived; sizes on slides.	Attack 'effectiveness' = fraction of generated URLs classified benign	Press/DEF CON summaries report attack effectiveness rising from 0.69% to 20.9% and 4.91% to 36.28% for two actors (secondary sources; slide numbers not re-verified here).	Generative evasion is feasible against character-level URL detectors.	- Authors' stated limitations: Not peer reviewed. - My assessment: No code/dataset release; vendor talk; numbers hard to reproduce.	Need reproducible, open mutation benchmarks instead of vendor demonstrations.	3 / F
[49]	AlEroud (2020); 2020 / Proceedings of the Sixth International Workshop on Security…	Can generative models create phishing URLs that bypass URL classifiers?	Train a GAN on phishing URLs; feed generated URLs to ML detectors (including PhishStorm-style intra-URL similarity features).	Not detailed in abstract.	Evasion rate against simple and advanced detectors	Abstract: the approach 'fooled both simple and advanced detection models' (no numbers in abstract).	GAN-generated URLs evade detectors.	- Authors' stated limitations: n/a (abstract only). - My assessment: Generated URLs may be unregistrable/unrealistic; unclear labels.	Realism constraints on generated adversarial URLs.	3 / A
[70]	Oest (2018); 2018 / 2018 APWG Symposium on Electronic Crime Research	How do phishers evade researchers/blacklists and structure URLs?	Analyse phishing kits' .htaccess cloaking and propose a URL classification scheme.	Phishing kits and URLs 2016-2017 (industry-sourced; not public).	Descriptive measurement	None quantitative in abstract.	Phishers track and evade anti-phishing infrastructure; URL type correlates with compromised infrastructure.	- Authors' stated limitations: n/a (abstract only). - My assessment: Tangential to URL classifiers.	n/a	2 / A
[71]	Lee (2020); 2020 / NDSS Workshop on Measurements, Attacks, and Defenses for th…	Phishing classifiers can be evaded by mimicking benign feature values.	Train multiple models, each on a random feature subset with injected noise; majority vote; evaluate against evasion strategies that alter feature values.	Benign: Alexa top 300k domains ranked Feb 2019, 110,090 pages crawled May-Aug 2019; phishing: PhishTank daily feed (paper Sec. IV). No license stated.	Native model vs. proposed ensemble under several evasion strategies (feature mimicry)	Ensemble 'performs close to native model' without attack and is 'more robust against evasion attacks' (abstract); numeric tables in Sec. V.	Feature-subset noisy ensembles raise evasion cost at small clean-accuracy cost.	- Authors' stated limitations: Evasion strategies are simulated by feature manipulation. - My assessment: Not URL-only; small-scale; no calibration or latency reporting.	Robust ensembles for lexical-only URL models with explicit mutation operators.	4 / F
[72]	Hu (2021); 2021 /	How effective are browser	Generate homograph IDNs for	Test cases generated from	Policy bypass rates per	All 7 browsers have rule	Punycode/IDN homographs	- Authors' stated limitations:	ML URL detectors'	4 / F

Ref.	Author (year); venue	Research objective	Method	Dataset	Evaluation protocol	Metrics reported	Main findings	Limitations	Research gap	Rel. / access
	30th USENIX Security Symposium, pp. 3739-3756	defenses against IDN homograph phishing domains?	popular domains, test display behaviour across browser versions, run a user study.	popular domains; not a URL classifier dataset.	browser/version; user deception rates	weaknesses; Chrome reversed some rules over time; bypassing homographs remain highly deceptive to users (abstract; detailed rates in Sec. 5-6).	still reach users; defenses inconsistent.	Snapshot to April 2020. - My assessment: Not about ML detection; relevant as threat model for homoglyph mutations.	handling of punycode (xn--) vs. Unicode rendering is rarely tested.	
[73]	Nowroozi (2022); 2022 / arXiv:2204.13172	Detect fraudulent advertisement URLs and assess vulnerability of tree ensembles to adversarial attacks.	Six feature groups (lexical + scraped), four boosting/bagging models, K-Means visual analysis, Zeroth-Order-Optimization attack.	Twelve dataset variants built by the authors (paper Sec. IV); no license statement seen.	Accuracy, FNR; mismatched and unlabelled datasets; limited-knowledge exploratory attack	Best FNR 0.0037 and accuracy 99.63% (abstract).	Tree ensembles are accurate but can be attacked in a black-box setting.	- Authors' stated limitations: Preprint; ZOO attack operates on feature vectors. - My assessment: Feature-space attack realism unclear (cf. SpacePhish).	Problem-space URL attacks on gradient-boosted lexical models.	3 / F
[74]	Kintis (2017); 2017 / Proceedings of the 2017 ACM SIGSAC Conference on Computer a…	Characterise combosquatting, a squatting form not captured by typo-distance.	Identify combosquatting domains for popular trademarks in large DNS datasets; study registration, longevity, abuse.	Passive DNS + active DNS over ~6 years (paper Sec. 3); not released.	Prevalence, lifetime, abuse categories	~468B DNS records analysed; ~60% of abusive combosquatting domains active for >1,000 days (per abstract/summary).	Combosquatting is more prevalent than typosquatting and long-lived.	- Authors' stated limitations: Trademark list limited; attribution of abuse is heuristic. - My assessment: No ML detection; lexical cues (brand token + hyphen/keyword) are what our model should captu…	Combosquatting-specific lexical features in URL classifiers are rarely isolated or explained.	3 / F
[75]	Tian (2018); 2018 / Proceedings of the Internet Measurement Conference 2018	Find high-quality 'elite' phishing pages that impersonate both content and domain.	Enumerate five squatting types for 702 brands, scan 224M DNS records, classify pages with visual+OCR features, verify manually.	DNS zone data and crawled pages (2017-2018); not released as a URL benchmark.	Precision via manual verification; blacklist evasion measured over time	657K candidate squatting domains; 1,175 verified squatting phishing pages; >90% evaded popular blacklists for at least a month (abstract).	Squatting phishing is long-lived and under-blacklisted; domain-level impersonation is a core evasion tactic.	- Authors' stated limitations: Brand list limited to 702; manual verification bottleneck. - My assessment: Content-based detector; our lexical model must rely on domain string cues.	Squatting-aware lexical features (brand-in-domain edit distance) under controlled mutation.	4 / F
[76]	Scheitle (2018); 2018 / Proceedings of the Internet Measurement Conference 2018	Research results depend on which top list (and which day) is used, yet lists differ structurally and churn heavily.	Structural and stability analysis of top lists and their effect on research results	Alexa, Umbrella, Majestic lists; survey of list use in measurement/security papers	Repeats several published measurements with different lists/dates to show result sensitivity	Demonstrates that list choice and date materially change research conclusions (Sec. 6).	Top lists differ strongly in structure (e.g., Umbrella contains many non-web hosts) and churn; results that depend on them are fragile.	**Authors' stated:** Measurement scope. **Our caveats:** Relevance is indirect: it quantifies how arbitrary the 'legitimate' class becomes when sourced from a list.	N/A	2 / F
[77]	Pochat (2019); 2019 / Network and Distributed System Security Symposium	Top-site lists used as benign ground truth are unstable and manipulable.	Measurement of top-list instability and manipulability (demonstrates cheap insertion into Alexa); proposes Tranco aggregate list	Alexa, Cisco Umbrella, Majestic and Quantcast top lists over time; 133 security papers surveyed for top-list usage	Longitudinal list comparison; manipulation experiments	Shows large day-to-day churn in Alexa and that a domain can be inserted into the top 1M cheaply (Sec. 4-5); many security papers (incl. phishing) use Alexa as benign ground truth (Sec. 3).	Using Alexa as 'legitimate' ground truth is unreliable and manipulable; Tranco is proposed for reproducibility.	**Authors' stated:** Any list is a proxy for popularity, not legitimacy. **Our caveats:** Popularity lists contain only registrable domains/hostnames, which is the root of the 'www./index page' artef…	Legitimate-URL sampling beyond popularity lists.	3 / F
[78]	Botacin (2021); 2021 / Computers & Security	Malware research suffers recurring methodological pitfalls that limit impact and reproducibility.	Literature review identifying 20 pitfalls (threat model, closed datasets, ground truth from closed-source AV, etc.)	Systematic review of 491 malware papers (2000-2018)	N/A	20 pitfalls identified across 491 papers (abstract). (Full text not accessed; numbers restricted to the abstract/landing page.)	Dataset and ground-truth practices (private datasets, AV-label ground truth, non-deterministic analysis) limit validity and reproducibility.	**Authors' stated:** Malware focus; through 2018. **Our caveats:** Secondary relevance; complements Arp et al. with reproducibility pitfalls.	N/A	2 / A
[16]	Hannousse (2021); 2021 / Engineering Applications of Artificial Intelligence	Findings in phishing-detection ML are dataset dependent and non-replicable; no appropriate benchmark dataset exists for fair evaluation.	Reproducible dataset-construction scheme with guidelines (G1: collect from several sources; do not use Alexa domain lists directly because they lack sub-domains/paths); feature-im…	New balanced dataset of 11,430 URLs with 87 features (56 URL, 24 content, 7 external services); phishing from PhishTank/OpenPhish, legitimate from Alexa-derive…	Random stratified splits on the new dataset; feature relevance via several selectors	Random forest reaches about 96.6% accuracy on the full feature set (Sec. 6, Table 9 of preprint); hybrid URL+content features outperform URL-only.	States that findings in the literature 'are dataset dependent and are far away from generalization' because of unreplicable datasets and the Alexa-domain-only sampling problem.	**Authors' stated:** Dataset is small and balanced; external-service features are time-varying. **Our caveats:** Even with their fix for paths, legitimate URLs still come from popular domains, so pop…	Large, time-stamped, source-diverse URL benchmark with documented artefact statistics.	4 / F
[48]	Apruzzese (2022); 2022 / Proceedings of the 38th Annual Computer Security Applicatio…	It is unclear where (problem space vs feature space) realistic, cheap adversarial perturbations against ML phishing detectors can be introduced and how damagin…	Formalises the 'evasion-space' (problem-space vs feature-space perturbations) and a realistic threat model; evaluates 12 cheap evasion attacks	Two public phishing website datasets (Zenodo and delta-phish style collections; see Sec. 5 of PDF), 18 ML-PWD configurations	Statistical significance testing of detection-rate degradation across 18 detectors and 12 attacks, repeated trials	Realistic cheap evasions induce a statistically significant 3-10% degradation; combined problem+feature-space perturbations can drop detection rate from 0.95 to 0 (extended version abstract).	Even trivial, cheap URL/HTML manipulations measurably reduce detection; some detectors are immune to some attacks, so robustness must be measured per feature set.	**Authors' stated:** Only two datasets; attacks are heuristic. **Our caveats:** The URL-feature detectors' sensitivity to small lexical perturbations is the same vulnerability our scheme/www artefact…	Joint analysis of artefact reliance and evasion susceptibility.	4 / F
[13]	Arp (2022); 2022 / 31st USENIX Security Symposium	Subtle design/evaluation pitfalls make ML security results unrealistic; how prevalent are they and how much do they inflate results?	Taxonomy of ten pitfalls: P1 sampling bias, P2 label inaccuracy, P3 data snooping, P4 spurious correlations, P5 biased parameter selection, P6 inappropriate baseline, P7 inappropr…	Literature study of 30 top-tier security papers (2010-2020) plus four impact case studies (mobile malware, vulnerability discovery, source-code authorship, net…	Prevalence analysis across 30 papers; controlled experiments showing each pitfall's inflation effect	Each pitfall is present in a majority of surveyed papers (e.g., sampling bias and data snooping are marked present in most; see Fig. 2/Table 1). Base-rate example: a 1% FPR at a 1:100 ratio yields ro…	Spurious correlations ('artifacts unrelated to the security problem create shortcut patterns for separating classes') and sampling bias are endemic; recommends explanation-based artefact checks, temp…	**Authors' stated:** Survey limited to top venues and 30 papers; pitfall boundaries overlap. **Our caveats:** Does not include a phishing-URL case study, but P1/P4/P8 map one-to-one to our situation.	A phishing-URL instantiation of the pitfall analysis with concrete artefact ablations.	5 / F
[17]	Sabir (2022); 2022 / IEEE Transactions on Dependable and	The robustness and reliability of ML phishing-URL detectors against cheap	Methodology to test reliability and robustness: adversarial URL generator (URLBUG),	Reproduces 50 ML phishing-URL detectors; phishing URLs from	Baseline (balanced, in-distribution) vs adversarial URLs; MCC	Median MCC falls from 0.92 (baseline) to 0.02 on Advdata; adversarial domains registrable	State-of-the-art URL detectors are brittle; the training vocabulary is biased toward	**Authors' stated:** Adversarial generation is heuristic; reproduction may not match	Combining robustness testing with artefact	5 / F

Ref.	Author (year); venue	Research objective	Method	Dataset	Evaluation protocol	Metrics reported	Main findings	Limitations	Research gap	Rel. / access
	Secure Computing	adversarial URLs is largely unknown.	reproduction of 50 traditional-ML and DL detectors, evaluation on Advdata	PhishTank/OpenPhish/ISCXURL2016/PhishStorm (96,693); legitimate URLs crawled from 1,500 Alexa domai…	as main metric; registration-cost and real-world-use analysis of adversarial …	at a median $11.99/yr; 63.94% of already-registered ones were used maliciously (abstract).	phishing target brand tokens. Explicitly notes that using Alexa index pages as the legitimate class 'creates a bia…	original hyper-parameters. **Our caveats:** Baseline evaluation remains balanced and random-split; no domain group…	ablation and domain-grouped splits.	
[42]	Apruzzese (2023); 2023 / 2023 IEEE 8th European Symposium on Security and Privacy	Practitioners distrust ML for intrusion detection because papers omit information (variance, costs, cross-dataset behaviour) needed to judge real value.	Defines 'pragmatic assessment': report what practitioners need (hardware, data pre-processing, costs, multiple datasets, statistical variance over many trials); hundreds of thousa…	Systematisation of NID papers plus large experimental campaign on public NID datasets (Sec. 5-6) and a practitioner survey	Multiple random seeds/splits, multiple datasets, reporting of runtime and resources; shows how single-split results mislead	Demonstrates run-to-run variance large enough to change rankings of methods (Sec. 6, full text).	Single-dataset, single-run evaluations are unreliable; cross-dataset and repeated-trial reporting should be standard.	**Authors' stated:** NID-specific datasets. **Our caveats:** The methodology (repeat trials, cross-dataset, cost reporting) transfers directly to URL classification.	Pragmatic assessment has not been applied to phishing URL detectors.	3 / F
[50]	Apruzzese (2023); 2023 / 2023 IEEE Conference on Secure and Trustworthy Machine Lear…	Adversarial-ML research threat models (gradient attacks) diverge from what operational ML security systems actually face.	Position paper contrasting research threat models with operational reality; case study (b) reports lack of evidence of gradient-based adversarial examples against a commercial phi…	Three industrial case studies (an online social network ML system; a commercial ML phishing-webpage detector; an ML evasion competition) plus analysis of recen…	Qualitative; literature analysis	No benchmark numbers; the phishing case study is qualitative (Sec. III).	Operational ML security problems are dominated by data drift, cheap problem-space manipulations and pipeline effects rather than gradient attacks; evaluation should reflect realistic attacker economi…	**Authors' stated:** Case studies are anecdotal by design. **Our caveats:** Useful for scoping our robustness analysis toward cheap URL manipulations (scheme/www changes, path padding) rather than gr…	Realistic, cheap perturbation benchmarks for URL detectors.	3 / F
[79]	Dambra (2023); 2023 / Proceedings of the 2023 ACM SIGSAC Conference on Computer a…	Malware-classification results depend on dataset construction, feature extraction and family definitions in poorly understood ways.	Controlled study of how dataset composition (families, samples per family, benign/malware balance), feature type and classifier drive reported performance	67k Windows PE samples from 670 families (100 each) plus benign set; static and dynamic features	Varies dataset properties systematically; family-aware splits	Shows performance strongly tied to dataset nature/distribution (Sec. 5-6).	Near-perfect numbers in prior work are tied to dataset construction choices; family-level grouping and composition matter.	**Authors' stated:** Windows PE only. **Our caveats:** Analogue: family-aware splits in malware correspond to registered-domain/campaign-grouped splits for URLs.	Equivalent composition study for URL datasets.	3 / F
[80]	Kulkarni (2024); 2024 / arXiv preprint	Dataset assembly biases (imbalance, staleness, narrow sources) skew phishing-webpage classifiers.	Identifies dataset-assembly biases (imbalanced, non-diverse, stale datasets; zero-day gap) and proposes a tool to synthesise phishing pages for any set of legitimate URLs to debia…	Position paper; proposes generating synthetic phishing pages from legitimate URLs	None reported (short paper)	None.	Argues that classifiers 'learn distinctive features from training data' and that biased dataset composition skews decision rules; proposes synthetic augmentation.	**Authors' stated:** No evaluation. **Our caveats:** Short preprint, limited evidence; include only as a recent explicit statement of dataset bias in phishing detection.	Empirical measurement of how much dataset composition bias changes learned rules.	2 / F
[45]	Dalton (2025); 2025 / arXiv preprint	Public phishing datasets are small, noisy, leak between train and test, and use unrealistic base rates, inflating reported performance.	Dataset construction pipeline: temporal train/test split, pruning of near-duplicate test points via locality-sensitive hashing, diversity/difficulty filters, base-rate adjustment;…	New large phishing-website dataset (HTML+URL) crawled with a real browser, quality-checked by heuristics and human annotation; hosted on Hugging Face; benchmar…	Temporal split + leakage pruning + five base rates; precision reported as a function of recall, FPR and base rate	Benchmarks span base rates 0.05%-5%; states existing public datasets 'suffer from leakage and unrealistic base rates, leading to overly optimistic performance results' (Sec. 1); baseline tables in Se…	Train/test leakage (same kit/template, near-duplicate pages) and balanced base rates inflate published results; realistic benchmarks need temporal splits, near-duplicate pruning and low prevalence.	**Authors' stated:** Preprint; industrial collection pipeline not fully reproducible; HTML-centric. **Our caveats:** URL-only baselines are secondary; still the most explicit public treatment of leak…	A URL-only benchmark with the same leakage/base-rate discipline.	4 / F
[46]	Giri (2026); 2026 / Zenodo preprint	Offline accuracy of lexical phishing-URL classifiers does not transfer to independently sourced traffic; formatting shortcuts (https://, www.) inflate results.	Diagnoses 'representation shift' (models keying on https:// scheme and leading www.), introduces component-aware URL canonicalisation and hybrid rule fusion, then tests a frozen m…	Lexical URL classifier trained on conventional public phishing/benign URL sets; post-freeze external snapshots: OpenPhish (phishing) and a Cisco Umbrella Top-1…	Pre-registration-style freeze protocol; domain-disjoint external test; bootstrap 95% CIs; variant-stability tests (same URL re-formatted)	Trivial formatting shortcuts (https://, www.) inflate offline accuracy and cause near-100% prediction instability across URL variants; after canonicalisation FPR on secondary benchmarks 0.6%; on the …	Removing representation shortcuts is necessary but not sufficient: cross-source benign traffic still breaks a lexical model; the main driver was vocabulary/score distribution shift on unseen benign s…	**Authors' stated:** Single model family; specific source composition; preprint. **Our caveats:** Not peer reviewed and single-authored; numbers should be cited as a preprint observation. Nonetheless…	Peer-reviewed replication of the https/www shortcut effect across datasets.	5 / F
[27]	Chiew (2019); 2019 / Information Sciences, 484, pp. 153-166	Select a compact, robust feature subset for ML phishing detection.	Two-phase ensemble feature selection; evaluates with RF, SVM, NB, C4.5, JRip, PART.	Authors' dataset with 48 features (URL + HTML) and the UCI phishing-websites dataset	Not in abstract	94.6% accuracy with RF using 20.8% of features (10 of 48) (abstract). (Only abstract/publisher-page information was available; no additional numbers should be attributed to this paper without checkin…	Small feature subsets with RF outperform full feature sets with other learners.	- Stated by authors: Not available. - Our own caveats: Includes HTML features, so not URL-only; UCI dataset is pre-extracted and dated.	Feature-selection stability across datasets.	3 / A
[26]	Rao (2020); 2020 / Journal of Ambient Intelligence and Humanized Computing, 11…	Lightweight URL-only phishing detection without visiting the site or third-party services.	Hand-crafted hostname/URL features + TF-IDF features + phishing hint words; RF classifier.	Authors' dataset plus two benchmark datasets (names/sizes not on abstract page)	Not stated on abstract page	93.25% (TF-IDF only) and 94.26% (TF-IDF + hand-crafted) on own data; 98.25% and 97.49% on two benchmark datasets (Springer abstract). (Only abstract/publisher-page information was available; no addit…	Combining TF-IDF n-grams with hand-crafted lexical features improves RF accuracy.	- Stated by authors: Not available. - Our own caveats: Benchmark names unclear from abstract; no temporal test.	Cross-dataset evaluation.	3 / A
[81]	Sameen (2020); 2020 / IEEE Access, 8, pp. 83425-83443	Detect both human-crafted and AI-generated (DeepPhish) phishing URLs in real time.	Lexical feature extraction with HTML-encoding decoding; ensemble of ML models with unbiased voting; multi-threaded	100,000 URLs: 50,000 Alexa benign + 50,000 DeepPhish AI-generated phishing (Sec. VI, Table	5:5 split ratio train/test (Sec. VI); theoretical analysis	98.00% accuracy and F1 about 98% on the 100k set; 97% TPR, 99.17% TNR, 0.8% FPR (Sec. VII, Tables 7-9).	Lexical ensembles can separate DeepPhish-generated URLs from Alexa benign URLs with high accuracy.	- Stated by authors: Tiny-URL handling needs expansion service; '100% on future AI URLs' claim is theoretical (Sec.	Realistic benign class and temporal validation.	3 / F

Ref.	Author (year); venue	Research objective	Method	Dataset	Evaluation protocol	Metrics reported	Main findings	Limitations	Research gap	Rel. / access
			execution; URL-Hit expands shortened URLs.	4); plus 50,000 PhishTank 'simple' phishing (collected by 4 Sep 2019)		(Numbers taken from the full text; section/table references given where identified.)		V). - Our own caveats: Benign = Alexa homepages vs long generated URLs: a strong le…		
[12]	Sanchez-Paniagua (2022); 2022 / IEEE Access, 10, pp. 42949-42960	Show that standard datasets (legitimate = homepages) overstate performance; legitimate login pages look like phishing URLs.	Builds PILU-90K; compares 8 classical learners on handcrafted features, TF-IDF+LR, and two char-CNNs; measures accuracy decay over time and clusters phishing domains.	PILU-90K (public): 30K legitimate homepage URLs (Quantcast top sites), 30K legitimate login URLs, 30K phishing URLs (PhishTank); plus older sets (Ebbu2017, PWD…	Train on homepage-vs-phishing then test on legitimate login URLs (FPR stress test); temporal: train on 2016 data, test on 2020 data (Table …	Models trained on PIU-60K (homepages) lose 27-39% accuracy on legitimate login URLs (best 69.50%, SVM 54.46%) (Sec. V-A, Fig. 4); TF-IDF + LR reaches 96.50% in the login scenario (abstract, Table 6);…	Benign-class composition is a major shortcut; URL length separability largely disappears when login URLs are used (Fig. 3); models age quickly.	- Stated by authors: Dataset limited to top-site login pages; deep models not exhaustively tuned. - Our own caveats: Still a random split within year; no domain-grouped split.	Domain-grouped and calibrated evaluation; artifact quantification (https/www).	5 / F
[82]	Jalil (2023); 2023 / Journal of Ambient Intelligence and Humanized Computing, 14…	Fast, third-party-free URL phishing detection with domain-informed feature selection and broader evaluation.	Feature set from full URL/protocol/host/path incl. entropy and brand TF-IDF; compares 8 learners across 6 datasets.	Six datasets: two Kaggle URL datasets and four benchmark datasets (names/sizes not on abstract page)	Multiple datasets; protocol not on abstract page	RF accuracy 96.25% and 94.65% (Kaggle sets); 92.2%, 91.63%, 94.80%, 96.85% (benchmark sets) (Springer abstract). (Only abstract/publisher-page information was available; no additional numbers should …	Accuracy varies noticeably across datasets (91.6-96.9%), illustrating dataset dependence.	- Stated by authors: Not available. - Our own caveats: No cross-dataset (train A / test B) protocol visible from abstract.	Cross-dataset transfer and calibration.	3 / A
[25]	Prasad (2024); 2024 / Computers & Security, 136, 103545	Phishing URL detection with similarity-based features and incremental model updates; release of a large feature dataset.	Computes URLSimilarityIndex, CharContinuationRate, TLDLegitimateProb and other URL/HTML-derived features; incremental learning to adapt to new data.	PhiUSIIL dataset (UCI #967, CC BY 4.0): 235,795 URLs (134,850 legitimate, 100,945 phishing), 54 features from URL and page source	Not retrievable (publisher page blocked); UCI page documents dataset only	Not retrieved. (Only abstract/publisher-page information was available; no additional numbers should be attributed to this paper without checking the full text.)	Dataset is now a common benchmark (used by Yi 2026 and others).	- Stated by authors: Not retrieved. - Our own caveats: Dataset mixes URL and page-source features and some derived features (e.g., TLDLegitimateProb) risk label leakage; collection windows undocument…	Leakage audit of derived features.	3 / A
[7]	Sankaranarayanan (2024); 2024 / PLOS ONE, 19(5), e0302196	Multi-class malicious URL classification on a larger dataset with a stacking ensemble.	Lexical feature extraction; individual boosted/bagged trees; stacking meta-learner; hyperparameter tuning.	Kaggle 'malicious-urls-dataset' (sid321axn): 651,191 URLs; 4 classes benign/defacement/phishing/malware (Sec. 3)	Stratified 80/20 train/test split (Sec. 3.2.3); no cross-dataset	Individual accuracies 93.6% (RF), 95.2% (XGB), 95.7% (LGBM), 94.8% (CatBoost); stacking average accuracy 96.8%; per-class precision 91-99%, recall 87-99% (abstract; Sec. 4). (Numbers taken from the f…	Stacking adds about 1 point over the best single booster on lexical features.	- Stated by authors: Lexical features only; short-lived URLs (Sec. 2). - Our own caveats: Kaggle aggregate dataset mixes sources (ISCX, PhishStorm, PhishTank...) so class is partly predictable from s…	Source-aware / cross-dataset evaluation and calibration.	3 / F
[8]	Khan (2026); 2026 / Frontiers in Artificial Intelligence, 9, 1854934	Explainable, statistically validated stacking ensemble for phishing URL detection.	12-stage pipeline: EDA, SHAP feature selection, benchmarking seven algorithm families, Bayesian optimisation, stacking, threshold tuning, SHAP explanations.	Public Kaggle benchmark (Harisudhan 2024): 247,950 URLs with 41 pre-extracted structural/lexical features; raw URL strings not released (Sec. 3.1)	Stratified 80/20 split, 5-fold CV inside training; no cross-dataset	Top tier: Extra Trees Acc 0.9671 / AUC 0.9948, RF Acc 0.9667 / AUC 0.9930; stacking ensemble leads on Acc/F1/MCC/Kappa with AUC 0.9947 vs 0.9948 for Extra Trees; confusion matrix 24,977 TN, 731 FP, 8…	Stacking gives marginal gains over tuned Extra Trees; SHAP-selected features retain performance.	- Stated by authors: Dataset lacks raw URLs, undocumented collection windows/sources; single dataset; no temporal validation (Sec. 3.1 and limitations). - Our own caveats: About 248k pre-featurised K…	Cross-dataset / temporal evaluation, artifact audit, calibration measurement.	4 / F
[83]	Yi (2026); 2026 / Security and Privacy	Assess whether a fixed URL feature set generalises across datasets and explain decisions.	Trains three tree ensembles; evaluates on secondary datasets; SHAP for feature attribution.	Primary email-URL dataset plus secondary/test datasets (four test sets in total; names not in abstract)	Cross-dataset: train on primary set, evaluate on four secondary test sets	Across four test sets: average classification error 4.03%, average accuracy 94% (abstract). (Only abstract/publisher-page information was available; no additional numbers should be attributed to this…	Tree ensembles with a common lexical feature set transfer reasonably across datasets; SHAP highlights consistent key features.	- Stated by authors: Not available. - Our own caveats: Abstract-level only; cannot verify whether test sets share sources with training data.	Domain-grouped splits, calibration, artifact analysis.	4 / A
[29]	Bahnsen (2017); 2017 / 2017 APWG Symposium on Electronic Crime Research	Detect phishing from URL alone without content download.	Compare engineered lexical features + RF against character LSTM reading the URL string.	2,000,000 URLs: 1M PhishTank phishing, 1M Common Crawl benign (paper Sec. IV); collection period not stated; no license.	3-fold cross-validation; accuracy; learning curves with 200k test URLs	LSTM accuracy 98.7%, 'beating by 5%' the RF (abstract; Table II in paper).	Character RNN outperforms hand-crafted features at scale.	- Authors' stated limitations: No evaluation on temporal splits. - My assessment: Alexa membership feature leaks list-based priors; PhishTank/Common Crawl distribution gap makes task easy.	Robustness and interpretability of the RNN not studied (motivating DeepPhish follow-up).	4 / F
[84]	Aljofey (2022); 2022 / Scientific Reports	Detect zero-hour phishing websites without third-party services and validate on more than one dataset.	XGBoost on URL character sequences, hyperlink features and TF-IDF of noisy HTML/plaintext	Self-built dataset of 60,252 webpages (32,972 benign, 27,280 phishing) plus a benchmark dataset for external validation	Evaluation on own dataset and on a second benchmark dataset (cross-dataset check)	96.76% accuracy with 1.39% FPR on own data; 98.48% accuracy with 2.09% FPR on the benchmark dataset (abstract). (Full text not accessed; numbers restricted to the abstract/landing page.)	Reports that a model built for zero-hour detection transfers to a second dataset; one of the few mainstream papers to include an external dataset.	**Authors' stated:** Lack of large public datasets motivated self-collection. **Our caveats:** Content-heavy; cross-dataset result involves retraining on the benchmark rather than a frozen-model tran…	Frozen-model external validation is still rare.	2 / A
[9]	Mia (2024); 2024 / Proceedings of the 11th International	Are the features that make phishing-URL classifiers accurate on one dataset	Train/test within and across datasets (Exp-1..6); SHAP global/local explanations to	Dataset-1: Vrbancic et al. (88,647 instances, 58,000 benign); Dataset-2:	Random splits within dataset; cross-dataset transfer on common	Within-dataset accuracy is high, but cross-dataset accuracy 'drops drastically to 51%' (train	Features are dataset dependent; near-chance cross-dataset accuracy from models that look	**Authors' stated:** Only two datasets; only overlapping features. **Our caveats:** Datasets	Cross-dataset tests on raw URLs with controllable	5 / F

Ref.	Author (year); venue	Research objective	Method	Dataset	Evaluation protocol	Metrics reported	Main findings	Limitations	Research gap	Rel. / access
	Conference on Network…	trustworthy on another?	compare feature contribution	Kaggle set with 19,431 rows (9,716 benign); 20 overlapping numeric URL features	features; merged training	D1, test D2) and 59% (train D2, test D1) (Sec. 3.5); SHAP shows the same feature (e.g., url_google_index,…	excellent in-distribution.	are pre-extracted feature tables, so raw-URL artefacts (scheme/www) cannot be ablated.	artefacts.	
[11]	Rashid (2024); 2024 / Computer Networks	URL-based phishing detectors work only when the test set is split from the same dataset; they do not generalise across datasets collected at different vantage …	Unsupervised domain adaptation framework to transfer URL classifiers between datasets	Three phishing-URL datasets (abstract); pairwise cross-dataset experiments	Train on one dataset, test on another (cross-dataset F1), with and without adaptation	Cross-dataset F1 improves by 0.06 on average and up to about 0.2 with adaptation (abstract). Secondary reports (not verified from full text) attribute part of the baseline drop to URL-length shift an…	Existing ML/DL URL detectors 'work effectively only when the test sets are split from the same training dataset'; format differences between collection vantage points (how URLs are recorded) are an e…	**Authors' stated:** Not visible from abstract. **Our caveats:** Full text not accessed; the 'www' token claim must be verified before citing it as the paper's finding.	Explains cross-dataset drops mechanistically (format artefacts) rather than only reporting them.	5 / A
[47]	Bhuiyan (2026); 2026 / Big Data and Cognitive Computing	Benchmark accuracy of phishing (email) detectors hides failures under cross-corpus transfer, prevalence shift and artefact-driven learning.	TF-IDF logistic regression and linear SVC; pooled baseline, single-corpus cross-dataset transfer, leave-one-corpus-out, prevalence-shift simulation, training-prevalence manipulati…	Six public phishing/spam email corpora: CEAS_08, Enron, Ling, Nazario, Nigerian Fraud, SpamAssassin	Cross-corpus and leave-one-out; prevalence shift with fixed thresholds vs alert budgets; artefact ablations	Leave-one-corpus-out F1 0.8201-0.8994 (LR) and 0.7607-0.8910 (SVC); dataset-identification accuracy 0.9722 (LR) / 0.9806 (SVC), i.e., corpus identity is highly learnable (abstract).	Models partly learn corpus markers (date tokens, headers, URL/domain terms) rather than phishing semantics; prevalence shift failure is asymmetric and threshold dependent; recommends cross-corpus val…	**Authors' stated:** Linear models only; email modality. **Our caveats:** Email, not URL; but the 'dataset-identification probe' and artefact-removal ablation design transfer directly to URL features.	Same protocol for URL datasets.	4 / F
[10]	Henry (2026); 2026 / Franklin Open	Phishing classifiers degrade sharply when deployed on a dataset different from the training one.	Cross-Domain Ensemble Probability Fusion: PCA harmonisation into a 20-dim space plus information-theoretic weighted fusion	UCI Phishing Websites dataset and PhiUSIIL phishing URL dataset (zero feature overlap)	Cross-dataset accuracy before/after the framework	Cross-dataset accuracy 57.4% baseline vs 94.4% with the framework (abstract). (Full text not accessed; numbers restricted to the abstract/landing page.)	Reports a 57.4% cross-dataset baseline, i.e., near chance, between two popular phishing datasets.	**Authors' stated:** Not visible from abstract. **Our caveats:** PCA-harmonising non-overlapping feature sets is hard to interpret; the 94.4% figure needs scrutiny for leakage. Use only the baseline …	Interpretable cross-dataset adaptation for URL detectors.	2 / A
[28]	Saxe (2017); 2017 / arXiv preprint arXiv:1702.08568	Replace hand-engineered features with a generic character-level CNN for short security strings.	Embeds characters (dim 32), parallel convolutions of several kernel sizes, sum pooling, dense layers; trained on raw URL strings.	Invincea/VirusTotal-derived URLs (counts in Table 1; URLs with 1-4 AV detections discarded), plus file paths and registry keys	Held-out split with low-FPR ROC analysis (FPR 1e-4 to 1e-2) (Sec. 4)	5-10% detection-rate gain over manual-feature baselines at 0.1% FPR (abstract); about 6% higher at FPR 1e-3 for URLs (Sec. 4, Fig. 3). (Numbers taken from the full text; section/table references give…	Char-CNN beats hashed n-gram baselines on URLs at low FPR.	- Stated by authors: Labels derived from AV engine counts (noisy); not peer reviewed. - Our own caveats: Dataset proprietary; no cross-dataset test.	Public reproducibility.	4 / F
[30]	Le (2018); 2018 / arXiv preprint arXiv:1802.03162	Learn URL embeddings end-to-end to capture semantics/sequence and handle unseen tokens.	Character CNN branch + word CNN branch (words embedded via character CNNs to handle rare words); jointly trained; ablations on branches and embedding modes.	VirusTotal crawl: 5M URLs for training (sampled from first 60%, time-ordered), 10M for testing (from last 40%); malicious = flagged by >=5 blacklists, benign =…	Temporal split (train earlier 60%, test later 40%); ROC/AUC and TPR at FPR 1e-4..1e-1 (Table 3)	URLNet(Full) achieves the best AUC and higher TPR at FPR 1e-3..1e-1 vs BoW baselines; slightly worse than word-level at FPR 1e-4 (Table 3, Fig. 4); feature vectors about 14x smaller than BoW for 2k U…	Joint char+word CNN generalises to unseen words better than BoW.	- Stated by authors: Hyperparameter sensitivity; large compute. - Our own caveats: Labels from blacklist counts; multi-class malware vs phishing collapsed into 'malicious'.	Explainability of learned features; robustness.	5 / F
[85]	Huang (2019); 2019 / 2019 18th IEEE International Conference on Trust, Security …	Zero-day phishing URL detection without manual features.	Character CNN for spatial features, word-level hierarchical RNN with attention for temporal features, CNN fusion, classifier.	'Verified dataset collected from the Internet' (abstract); sizes not available	Not available; abstract emphasises generalisation to newly emerging URLs	Not in abstract ('competitive performance'). (Only abstract/publisher-page information was available; no additional numbers should be attributed to this paper without checking the full text.)	Fused char/word representations improve generalisation on new URLs (authors' claim).	- Stated by authors: Not available. - Our own caveats: No public code/data located.	Reproducibility.	3 / A
[86]	Wang (2019); 2019 / Security and Communication Networks, 2019, Article 2595794,…	Fast URL-only phishing detection without page content or third-party services.	URL encoded as 2-D tensor; BiLSTM extracts global features; CNN selects key characters; fixed-length vector -> classifier.	Nearly 500,000 URLs from Alexa (benign) and PhishTank (phishing) (abstract)	Not in abstract	Accuracy 97%, AUC 99%, 0.4 ms per URL (abstract). (Only abstract/publisher-page information was available; no additional numbers should be attributed to this paper without checking the full text.)	RNN+CNN hybrid beats either alone.	- Stated by authors: Not available. - Our own caveats: Alexa-vs-PhishTank construction (homepage shortcut).	Realistic benign class.	3 / A
[32]	Aljofey (2020); 2020 / Electronics, 9(9), 1514	Fast URL-only phishing detection without third-party services or manual feature engineering.	Char-CNN (embedding + conv + pooling); baselines include RF/LR/NB/MNB with hand-crafted features across URL/host/path/file/query parts.	D1: 318,642 URLs (Alexa, OpenPhish, Spamhaus, techhelplist, SANS ISC, PhishTank); D2 Ebbu2017 (Yandex/PhishTank); D3, D4 benchmark sets (Common Crawl+Alexa vs …	Random splits per dataset; five experiments; metrics accuracy/precision/recall/F1/AUC/time	Accuracy 95.02% on D1; 98.58%, 95.46%, 95.22% on D2-D4 (abstract, Sec. 5); common prefixes 'http://', 'https://', 'www.' are removed before modelling (Sec. 4.1, Table 3). (Numbers taken from the full…	Char-CNN outperforms classical ML with hand-crafted or TF-IDF features; URL-length distributions differ by class (Fig. 10).	- Stated by authors: Cannot handle shortened URLs; may misclassify unusual legitimate URLs (Sec. 6). - Our own caveats: Explicitly removes scheme/www prefixes - evidence that authors recognised these…	Cross-dataset generalisation and artifact quantification.	4 / F
[31]	Tajaddodianfar (2020); 2020 / ICASSP 2020 - IEEE International Conference on	Internet-scale phishing URL detection operating at extremely low FPR.	Character path and word path with parallel convolution blocks (Inception-like), concatenated, dense output; compared with a traditional text	Microsoft production telemetry: six weeks of data; first two weeks -> 1.7M training samples (benign down-sampled to	Temporal: train on first 2 weeks, test on later production data; TPR at fixed 0.01% FPR (Table 2)	TPR increased by 126.7% relative to a traditional text classifier at 0.01% FPR (abstract; Table 2 lists TP and error rates at fixed 0.01% FPR,	Char/word parallel-CNN generalises better to new URLs than classical text classification at production FPRs.	- Stated by authors: Proprietary data; production labels. - Our own caveats: Not reproducible; absolute TPRs are low at 0.01% FPR, showing how hard low-	Public low-FPR benchmarks.	4 / F

Ref.	Author (year); venue	Research objective	Method	Dataset	Evaluation protocol	Metrics reported	Main findings	Limitations	Research gap	Rel. / access
	Acoustics, S…		classifier and a URLNet baseline.	5% positives); test set 20M instances sampl…		e.g. Baseline_URLNet 12.3% TP). (Numbers taken from the …		FPR operation is.		
[87]	Wei (2020); 2020 / Computer Networks, 178, 107275	Lightweight, fast URL-text-only phishing detector for mobile.	Char-CNN sized for mobile inference.	Not in abstract (phishing/legitimate URL text)	Not in abstract	'Almost 100% accuracy' (abstract) - no precise figures retrieved. (Only abstract/publisher-page information was available; no additional numbers should be attributed to this paper without checking th…	Char-CNN sufficient and fast.	- Stated by authors: Not available. - Our own caveats: Near-perfect accuracy claims suggest easy/biased data.	Realistic evaluation.	2 / A
[88]	Alshingiti (2023); 2023 / Electronics, 12(1), 232	Compare deep architectures for phishing website detection.	Applies 1D-CNN, LSTM and stacked LSTM-CNN to tabular URL features.	Pre-extracted feature dataset of 20,000 URLs (9,800 phishing) with 80 features, reduced to 30 via SelectKBest (Sec. 3.2)	80/20 random split (Sec. 3.2)	Accuracy 99.2% (CNN), 97.6% (LSTM-CNN), 96.8% (LSTM) (abstract; Sec. 4). (Numbers taken from the full text; section/table references given where identified.)	CNN on tabular features best.	- Stated by authors: Small dataset. - Our own caveats: Despite the title it is not character-level; 20k pre-featurised samples; random split - results not comparable to raw-URL models.	Raw-URL evaluation.	2 / F
[33]	Bozkir (2023); 2023 / Computers & Security, 124, 102964	URL-only phishing detection with n-gram embeddings; release of a large, time-spanning dataset.	Three n-gram granularities embedded in parallel; each channel CNN -> LSTM -> attention; concatenated for classification.	GramBeddings dataset (public via request form): 800,000 URLs, 400K phishing + 400K legitimate (custom crawler), collected May 2019-June 2021 (dataset page; Tra…	Not retrieved (publisher blocked)	Not retrieved from the paper (TransURL Fig. 5 reproduces GramBeddings results on its dataset). (Only abstract/publisher-page information was available; no additional numbers should be attributed to t…	Multi-granular n-gram embeddings with attention are competitive; dataset spans 2+ years with weekly collection to reduce duplicates.	- Stated by authors: Not retrieved. - Our own caveats: Legitimate URLs crawled from top sites; dataset access via form.	Cross-dataset tests appear only in follow-up work (TransURL).	4 / A
[89]	Ozcan (2023); 2023 / Neural Computing and Applications, 35(7), pp. 4957-4973	Combine char embeddings and NLP features in one deep model.	DNN branch over NLP features + LSTM over char embeddings, fused.	Phishing URL datasets (sizes not in abstract)	Not in abstract	Not in abstract ('outperform other models in accuracy'). (Only abstract/publisher-page information was available; no additional numbers should be attributed to this paper without checking the full te…	Hybrid feature+sequence model beats single-view models (authors).	- Stated by authors: Not available. - Our own caveats: No numbers verified.	n/a	3 / A
[90]	Ghalechyan (2024); 2024 / Scientific Reports, 14, 25134	Does a model trained on open + private data work in production over time?	Feature engineering without third-party dependencies; deterministic vs probabilistic NNs; daily re-validation.	Combination of open sources (Alexa, PhishTank, OpenPhish) and EasyDMARC production data; daily validation feed	Daily validation on newly reported URLs (temporal, production)	About 97% average accuracy on daily validation data labelled by PhishTank/OpenPhish/EasyDMARC (abstract); high accuracy on short (<50 chars) and long URLs. (Only abstract/publisher-page information w…	Model trained on open+private data succeeds in production; probabilistic NN improves accuracy.	- Stated by authors: Validation labels may contain mislabelled data that could not be checked (abstract). - Our own caveats: Private data not shareable; exact architectures unknown here.	Public temporal benchmark.	4 / A
[34]	Sahingoz (2024); 2024 / IEEE Access, 12, pp. 8052-8070	Large-scale deep-learning URL phishing detection and release of a multi-million URL dataset.	Character vectorisation of URLs; five architectures compared; extensive related-work table of datasets.	About 5.1M URLs (shared on GitHub ebubekirbbr/dephides): 2,320,893 phishing (PhishTank up to Aug 2018) + legitimate URLs crawled (about 30 per domain) from top…	Random train/test splits on small and full datasets	CNN best with 98.74% accuracy (abstract; Sec. V). (Numbers taken from the full text; section/table references given where identified.)	CNN > RNN/attention variants at this scale; dataset released.	- Stated by authors: Phishing data up to 2018; legitimate URLs crawled from top domains. - Our own caveats: Random splits with about 30 URLs per legitimate domain almost guarantee domain overlap betw…	Domain-grouped and temporal evaluation.	4 / F
[56]	Niculescu-Mizil (2005); 2005 / Proceedings of the 22nd International Conference on Machine…	Which learning algorithms produce calibrated probabilities and how to fix those that do not?	Empirical comparison across ten learners; Platt scaling vs isotonic regression on held-out calibration data.	Eight binary classification problems (paper Sec. 3).	Squared error (Brier), log-loss, reliability diagrams	Boosted trees and SVMs push predictions away from 0/1 (sigmoid-shaped reliability diagrams); RF/bagging are better calibrated; isotonic needs >~1000 calibration points (paper Sec. 4-5; numbers not re…	Boosting (max-margin) methods are systematically miscalibrated and benefit most from Platt/isotonic; RF and bagged trees nearly calibrated.	- Authors' stated limitations: 2005-era learners; no gradient boosting with log-loss. - My assessment: Modern XGBoost/LightGBM with logistic loss are better calibrated than AdaBoost, so findings need…	Calibration behaviour of soft-voting/stacked ensembles on security data.	5 / A
[57]	Guo (2017); 2017 / Proceedings of the 34th International Conference on Machine…	Modern deep nets are over-confident; probabilities do not match empirical accuracy.	Define/popularise ECE and reliability diagrams; evaluate post-hoc calibration methods; propose temperature scaling.	Standard vision/NLP benchmarks (paper Sec. 5).	Expected Calibration Error (ECE), Maximum Calibration Error, NLL, reliability diagrams	Temperature scaling achieves lowest ECE on most datasets (paper Table 1).	Depth/width/BatchNorm/weight decay affect calibration; one-parameter temperature scaling is surprisingly effective.	- Authors' stated limitations: Post-hoc methods assume calibration set from same distribution. - My assessment: Our models are tree ensembles; temperature scaling applies to logits/scores but Platt/i…	Calibration of security classifiers under dataset shift and adversarial mutation.	5 / F
[55]	Li (2021); 2021 / Annual Computer Security Applications	Can confidence/uncertainty flag dataset shift and adversarial examples for malware detectors?	Re-build four Android malware detectors with six calibration methods; measure nine uncertainty metrics incl.	Android malware datasets with temporal split (paper Sec. 5); not URL data.	Calibration (ECE, Brier, NLL) and uncertainty quality under temporal dataset shift and under	Main findings (abstract): predictive uncertainty helps under dataset shift but cannot cope with adversarial evasion;	Vanilla detectors poorly calibrated; calibration degrades under shift and collapses under adversarial examples.	- Authors' stated limitations: Android-only; specific attack set. - My assessment: Different domain, but the protocol	No equivalent study for phishing URL classifiers.	4 / F

Ref.	Author (year); venue	Research objective	Method	Dataset	Evaluation protocol	Metrics reported	Main findings	Limitations	Research gap	Rel. / access
	Conference		imbalance-aware ones; test on shifted and adversarial data.		adversarial evasion	approximate Bayesian methods most promising; calibration methods can fail under at…		(calibration x shift x adversarial) maps directly onto our plan.		
[36]	Rudd (2020); 2020 / arXiv preprint arXiv:2011.03040	How should transformers be trained on URL strings for security tasks?	Auto-regressive pre-training on 20M unlabeled URLs does not transfer; adding an auxiliary auto-regressive loss while training from scratch (balanced dynamically) helps; compared w…	FireEye URL corpus: 1,007,451 training, 111,930 validation (20,013 malicious, 91,813 benign), 279,874 test URLs; 20M unlabeled URLs for pre-training (Sec. 3)	Held-out split; ROC-AUC comparison (Fig. 2-3)	Mixed-objective transformer on par with the best baseline (1D CNN) by ROC-AUC (Fig. 3); numbers in figure legends. (Numbers taken from the full text; section/table references given where identified.)	Generic NLP pre-training recipes do not transfer directly to URLs; char-CNNs remain a strong baseline.	- Stated by authors: Proprietary data; limited pre-training scale. - Our own caveats: Not peer reviewed.	Public pre-training corpora for URLs.	3 / F
[91]	Haynes (2021); 2021 / Procedia Computer Science, 191, pp. 127-134	Mobile-deployable phishing detection from URLs only.	Baseline ANNs on features; deep ANN on URL-only features (poor); pre-trained transformers fine-tuned on URL text.	URL and HTML feature datasets (sizes not in abstract)	Not in abstract	ANNs on URL+HTML features >96%; deep ANN on URL-only features max 86.2%; pre-trained transformers outperform custom-vocabulary model; about 8 min training (abstract). (Only abstract/publisher-page in…	Pre-trained text transformers transfer to URLs better than custom-vocab models in this setting.	- Stated by authors: Not available. - Our own caveats: Abstract omits transformer accuracy numbers.	n/a	3 / A
[35]	Maneriker (2021); 2021 / MILCOM 2021 - IEEE Military Communications Conference, pp. …	Transformer models for browser-scale phishing URL detection at very low FPR, with robustness to black-box attacks.	Compares URLTran_BERT, URLTran_RoBERTa, URLTran_CustVoc against URLNet and Texception; adversarial data augmentation and fine-tuning.	Microsoft browser telemetry: 6 weeks; train 1,039,413, validation 259,854, test 1,784,155 URLs (8,742 phishing) after 1:20 down-sampling (Sec. 3)	Temporal split (4 weeks train / 1 validation / 1 test); TPR at FPR 0.01%; adversarial fine-tuning tests	Table 2: URLTran_BERT Acc 99.67%, TPR@0.01%FPR 86.80%, F1 0.9971, AUC 0.9993; URLNet TPR@0.01% 71.20%; Texception 52.15%; relative TPR improvement over 21.9% (abstract). (Numbers taken from the full …	Fine-tuned BERT beats char/word CNNs at low FPR; adversarial fine-tuning preserves low FPR under homoglyph/compound-split attacks.	- Stated by authors: Proprietary data; attack coverage limited. - Our own caveats: Accuracy about 99.6% for all models shows accuracy is uninformative at this imbalance; TPR@low-FPR is the meaningful…	Public reproducible low-FPR benchmark; explainability of transformer URL models.	5 / F
[37]	Wang (2023); 2023 / ICASSP 2023 - IEEE International Conference on Acoustics, S…	Avoid naive transfer of text-pretrained models by pre-training a URL-specific transformer at scale.	URL-specific pre-training (3B+ URLs), adversarial fine-tuning for classification.	More than 3 billion unlabeled URLs for pre-training; labelled benign/malicious URL benchmarks (sizes not in abstract)	Not in abstract ('extensive and rigorous benchmark studies')	None in abstract ('significantly superior ... efficiency, robustness and accuracy'). (Only abstract/publisher-page information was available; no additional numbers should be attributed to this paper …	URL-domain pre-training beats text-pretrained transfer (authors).	- Stated by authors: Not available. - Our own caveats: No public weights located; not reproducible for us.	Open URL foundation models.	4 / A
[39]	Liu (2024); 2024 / Computer Networks, 253, 110707	Transformers miss local/character/structural cues in URLs; improve robustness and generalisation.	Co-trains char-aware transformer with pyramid/multi-scale modules; baselines URLNet, GramBeddings, URLTran, TextCNN etc.; code released (github.com/Vul-det/TransURL).	GramBeddings (800k balanced, 2019-2021), Mendeley/MalCrawler (1,561,934; 35,315 malicious), Kaggle-1 (632,503 balanced), Kaggle-2 (4-class, about 651k) (Table …	Random splits; class-imbalance subsampling; multi-class; cross-dataset (Table 6); adversarial evasion-character attacks; 30 live-URL case s…	Peak F1 improvement 40% in class-imbalanced setting and +14.13% accuracy under adversarial attack vs best baseline (abstract); cross-dataset example: TransURL Acc 0.9023 / F1 0.8846 / AUC 0.9839 vs G…	Cross-dataset performance drops sharply for all models (GramBeddings to about 70% accuracy), TransURL degrades least; adversarial character insertion hurts baselines.	- Stated by authors: Compute cost; evasion types limited. - Our own caveats: Cross-dataset results still far from in-distribution, confirming our generalisation concern; Table 2 shows TLD/length dist…	Artifact-aware evaluation; explainability.	5 / F
[92]	Trad (2024); 2024 / Machine Learning and Knowledge Extraction, 6(1), pp. 367-384	Can prompted general LLMs replace task-specific URL classifiers?	Multiple prompt strategies on chat models; fine-tuning of small base LLMs for binary URL classification; comparison with prior results on the same dataset (Table 3).	Hannousse & Yahiouche Mendeley dataset: 11,430 URLs (balanced); 1,000-sample balanced test subset (500/500); remaining 10,430 for fine-tuning (Sec. 4)	Fixed 1,000 test subset; also varying phishing ratio experiments (Fig. 8-10)	Best prompting F1 92.74% (Claude 2); fine-tuned gpt2-medium F1 97.29%, AUC 99.56% (abstract; Sec. 5); at low phishing ratios GPT-2 F1 76.19%/86.73% vs Claude 2 59.15%/74.58% (Sec. 5.3). (Numbers take…	Fine-tuned small LLMs beat prompted frontier chat models; prompted models degrade under class imbalance.	- Stated by authors: Small test set (1,000); single dataset (Sec. 6). - Our own caveats: Dataset is small; results may not transfer to 250k-scale data.	Larger, temporally split LLM evaluations.	4 / F
[93]	Ali (2025); 2025 / Proceedings of the 26th International Conference on Distrib…	Parameter-efficient LLM fine-tuning for phishing URLs.	LoRA adapters on RoBERTa/GPT-2.	Proprietary corpus of 573,880 phishing + benign URLs (abstract)	Not in abstract	99.86% accuracy (abstract). (Only abstract/publisher-page information was available; no additional numbers should be attributed to this paper without checking the full text.)	LoRA matches full fine-tuning at a fraction of parameters.	- Stated by authors: Not available. - Our own caveats: 2-page poster; proprietary data; accuracy-only reporting.	Public evaluation.	2 / A
[94]	Kibriya (2025); 2025 / Scientific Reports, 15, 43044	Lightweight, explainable 4-class malicious URL classification using LLM embeddings.	Extract URL embeddings with pre-trained LLMs; train small recurrent head; LIME for token-level attributions; latency measurement.	Kaggle malicious-urls dataset: 651,191 URLs (428,103 benign, 96,457 defacement, 94,111 phishing, 32,520 malware), aggregated from ISCX-URL-2016, PhishTank, Mal…	60:40 random train/test split; ablations; per-class PR/ROC curves	97.5% accuracy with BERT and T5 embeddings (avg precision/recall/F1 0.97/0.97/0.96); GPT-2 97%; 0.119 ms per sample; per-class AUC 0.99-1.0 (Results; Table 2, Figs. 6-8). (Numbers taken from the full…	Frozen LLM embeddings + tiny head reach boosted-tree-level accuracy on the Kaggle set; LIME gives readable token attributions.	- Stated by authors: Single dataset; obfuscation robustness tested only partly. - Our own caveats: Same aggregated Kaggle dataset as Sankaranarayanan 2024 - source-formatting shortcuts likely; random…	Cross-dataset/temporal validation.	3 / F
[38]	Li (2025); 2025 / Computer Networks, 270, 111513	URL-specific pre-trained encoder usable across URL tasks.	Custom URL tokenizer; multi-task pre-training with contrastive and adversarial objectives; evaluation on phishing detection, ad-URL	Pre-training: 3.28B unique URLs (Common Crawl 2.85B, Open PageRank 338.71M, expired domains 538.2M,	60/40 train/test for phishing and ad tasks; 80/20 for topic task; data-scale experiments; TPR at FPR 0.001	Phishing detection AUC > 0.99; with 60% of training data accuracy 0.9720 and TPR about 0.8 at FPR 0.001; outperforms BERT (0.9156) and RoBERTa	URL-domain pre-training helps most in low-data regimes; classical lexical models remain strong on in-distribution benchmarks.	- Stated by authors: Compute; multi-task interference partially addressed. - Our own caveats: Removed URLs longer than 150 chars from the phishing	Cross-dataset and temporal evaluation of pre-trained URL encoders.	4 / F

Ref.	Author (year); venue	Research objective	Method	Dataset	Evaluation protocol	Metrics reported	Main findings	Limitations	Research gap	Rel. / access
			detection, webpage topic classification; code g…	malicious sources 8.26M, ODP 3.86M); phishing task: GramBed…		(0.9206) in a low-data setting at 0.9506 (Sec. 5); tradi…		benchmark (Table 1) - a preprocessing choice that in…		
[95]	Liu (2025); 2025 / Information Fusion, 113, 102638	Domain adaptation of pre-trained LMs to URLs with multi-level feature fusion.	Post-training (masked token, noisy-text recognition, domain discrimination) then attention-based fusion network.	Public URL datasets (specifics not in abstract)	Small-data, class-imbalance and adversarial scenarios; 20-URL live case study (abstract)	AUC 0.9941; all 20 malicious URLs in case study detected (abstract). (Only abstract/publisher-page information was available; no additional numbers should be attributed to this paper without checking…	Post-training + multi-level attention improves robustness.	- Stated by authors: Not available. - Our own caveats: Same group as TransURL/URLBERT; heavy models.	Explainability, cost.	3 / A
[96]	Tian (2025); 2025 / arXiv preprint arXiv:2504.16449	Update the survey literature with LLM/transformer defenses, datasets and code.	Reviews blacklists -> classical ML -> deep learning -> LLM prompting/agents; tabulates datasets and reported accuracies.	Survey: catalogues public datasets 2016-2024 and open-source implementations 2013-2025 (GitHub list)	n/a (survey)	n/a (Numbers taken from the full text; section/table references given where identified.)	Notes lack of proper baselines and standardised benchmarks; many works report >99% on small sets.	- Stated by authors: Preprint under review. - Our own caveats: Accuracy tables are copied from papers without protocol normalisation.	Standard benchmarks and protocol reporting.	4 / F
[97]	Chen (2026); 2026 / arXiv preprint arXiv:2602.21394	Adaptive, multimodal phishing detection beyond static URL heuristics.	Tool-using agent (page crawl, screenshot, memory retrieval) with episodic memory; ablations of tools/memory.	TR-OP (5,000 phishing + 5,000 benign, Jul-Dec 2023), DynaPD, and new SocPhish (2,765 URLs: 516 phishing, 2,249 benign from five social platforms) (Sec. 4, Tabl…	Held-out evaluation on public sets plus real-world SocPhish; production deployment (about 60k URLs/week); prompt-injection robustness test	Recall +13.6% over SOTA on public sets, +20% on SocPhish; production 91.44% recall, 90.34% F1 (abstract; Tables 1, 5); 1.07 s/URL latency. (Numbers taken from the full text; section/table references …	Agentic multimodal reasoning improves recall on novel threats; URL-only URLTran degrades on real-world social-media URLs.	- Stated by authors: Latency/cost; prompt-injection sensitivity (Appendix A.3). - Our own caveats: Not URL-only; heavy; included to bound the design space.	n/a	2 / F
[98]	Trikilis (2026); 2026 / arXiv preprint arXiv:2601.20270	Improve LLM reasoning for phishing URL classification with minimal labelled data.	Decomposes URL judgement into sub-questions, iterates, uses answer-sensitivity to trigger re-reasoning.	HP, EBBU (Ebbu2017) and ISCX-URL-2016; random balanced 1,000-URL test subsets, 5 repetitions (Sec. 3)	Mean F1 over 5 random 1,000-URL draws per dataset (Table 1)	Least-to-Most average F1 0.9040 vs 0.8726 one-shot across models/datasets; Gemini best at 0.9621 average F1; URLTran supervised about 0.99 F1 (Table 1). (Numbers taken from the full text; section/tab…	Prompted LLMs approach but do not reach supervised transformers; reasoning prompts help smaller models most.	- Stated by authors: Small test subsets; API cost. - Our own caveats: Not peer reviewed; 1,000-URL subsets give wide confidence intervals.	Cost-aware comparison with lightweight models.	3 / F
[99]	taruntiwarihp) (2020); 2020 / Kaggle dataset	n/a	n/a	Download: Kaggle page (phishing_site_urls.csv). License: not confirmed. Timestamps: none. Labels: 'good'/'bad'. Known problems: undocumented provenance; heavy …	n/a	n/a	n/a	- Authors' stated limitations: n/a - My assessment: Provenance unknown - weak as an external benchmark.	n/a	3 / F
[58]	sid321axn) (2021); 2021 / Kaggle dataset	n/a (dataset)	Concatenation of public lists keeping only URL and class.	Download: Kaggle page above (malicious_phish.csv; requires Kaggle login). License: not confirmed (Kaggle pages commonly show CC0/unknown - verify on page). Tim…	n/a	n/a	Most widely used multi-class URL corpus on Kaggle; our ~250k binary dataset likely derives from or overlaps with it.	- Authors' stated limitations: n/a - My assessment: Provenance only partially documented; no per-URL dates; label noise unknown.	A documented, timestamped, de-duplicated version with source column.	5 / F
[100]	Internet) (2022); 2022 / Alexa.com end-of-service notice; press coverage	n/a	n/a	n/a	n/a	n/a	Alexa top-1M, the benign source of ISCX-URL-2016 and many phishing datasets, is no longer maintained; snapshots are frozen as of 2022.	- Authors' stated limitations: n/a - My assessment: n/a	n/a	3 / A
[101]	Foundation (2026); 2026 / https://commoncrawl.org/overview	n/a	n/a	Download: s3://commoncrawl/ (free), URL index https://index.commoncrawl.org/ . License: Common Crawl Terms of Use (crawl data free for use; respect source copy…	n/a	n/a	n/a	- Authors' stated limitations: n/a - My assessment: Benign-by-assumption; needs screening (e.g. Google Safe Browsing/VirusTotal) as PhiShark-style papers do.	n/a	4 / F
[5]	Mamun (2016); 2016 / Network and System Security	Lightweight categorisation of malicious URLs by attack type using lexical cues.	Extract lexical features, select sufficient feature subsets, classify with tree-based and kNN methods; study obfuscation techniques per URL type.	ISCX-URL-2016 (see Dataset section). Download: http://cicresearch.ca/CICDataset/ISCX-URL-2016/ (UNB CIC page). License: none stated (UNB copyright).	Per-class and multi-class accuracy; feature selection	~99% accuracy for single-type detection and 93-99% as a group with Random Forest (abstract/Sec. 5; Table 2a: >97% on single-class datasets).	Lexical analysis is effective and efficient; RF best; obfuscation types differ by attack class.	- Authors' stated limitations: Class labels inherited from sources; dataset era 2016. - My assessment: Source-per-class design means classifiers can learn source artefacts (Arp et al. pitfall); benig…	Temporal metadata absent; URL strings in the public release are partially feature-only.	5 / F

Ref.	Author (year); venue	Research objective	Method	Dataset	Evaluation protocol	Metrics reported	Main findings	Limitations	Research gap	Rel. / access
				Timestamps…						
[102]	OpenPhish (2026); 2026 / https://openphish.com/phishing_feeds.html	n/a	n/a	Download: https://raw.githubusercontent.com/openphish/public_feed/refs/heads/main/feed.txt . License: OpenPhish Terms of Use (community feed free, 'limited'). …	n/a	n/a	n/a	- Authors' stated limitations: n/a - My assessment: n/a	n/a	4 / F
[103]	(community) (2026); 2026 / https://phishtank.org/developer_info.php	n/a	n/a	Download: http://data.phishtank.com/data/online-valid.{csv,json,xml}[.gz/.bz2] (optionally with app key). License: Cisco end-user licence terms (research use p…	n/a	n/a	Only major public phishing source with per-URL timestamps.	- Authors' stated limitations: n/a - My assessment: Benign counterpart must be sourced separately (Tranco/Common Crawl), creating distribution artefacts.	n/a	5 / F
[104]	Spamhaus (2026); 2026 / https://urlhaus.abuse.ch/api/	n/a	n/a	Download: https://urlhaus-api.abuse.ch/v2/files/exports/<AUTH-KEY>/recent.csv (and full/active variants). License: abuse.ch terms of use, free under fair use; …	n/a	n/a	n/a	- Authors' stated limitations: n/a - My assessment: Malware != phishing; mixing shifts the task definition.	n/a	3 / F
[105]	Axelsson (2000); 2000 / ACM Transactions on Information and System Security	Low attack prevalence makes even small false-positive rates dominate alarms, limiting the usefulness of detectors.	Bayesian analysis of detector alarms under low attack prevalence	Analytical; no dataset	Shows that with realistic base rates the false-alarm rate dominates the Bayesian detection rate (positive predictive value)	Classic result: to achieve a useful PPV at low prevalence the FPR must be on the order of 1e-5 (Sec. 4-5 of original). (Full text not accessed; numbers restricted to the abstract/landing page.)	Accuracy/TPR on balanced data say little about operational usefulness; precision collapses at low prevalence.	**Authors' stated:** Intrusion-detection framing. **Our caveats:** Prevalence of phishing among URLs a user visits is far below 50%, so our balanced-set precision is not meaningful operationally.	Base-rate-adjusted reporting is still rare in phishing-URL papers.	3 / A
[106]	Sommer (2010); 2010 / 2010 IEEE Symposium on Security and Privacy	Why is ML-based anomaly detection, despite strong papers, almost never deployed operationally?	Analysis of why ML anomaly detection is rarely deployed: high cost of errors, semantic gap, diversity of traffic, difficulty of sound evaluation, adversarial setting	Position paper; no dataset	Recommends evaluation with realistic data, understanding of what the system learns, and attention to false-positive cost	None. (Full text not accessed; numbers restricted to the abstract/landing page.)	Closed-world assumptions and lack of insight into learned features produce systems that look good in the lab and fail in deployment.	**Authors' stated:** NID-specific. **Our caveats:** The 'semantic gap' maps to our model learning 'looks like a popular www/https site' instead of 'is legitimate'.	N/A	3 / P
[15]	Aassal (2020); 2020 / IEEE Access	Phishing detection methods are evaluated on inconsistent, often balanced datasets with few metrics; how do they fare under realistic imbalance and on diverse d…	PhishBench benchmarking framework; systematic feature taxonomies for URLs, websites, emails; evaluation of published methods under varying imbalance and dataset size; feature-impo…	New URL, website and email datasets with varying legitimate:phishing ratios; legitimate URLs from diverse sources rather than Alexa domain lists	Balanced vs imbalanced ratios, scaling experiments, retraining on new attacks	Shows that increasing legitimate:phishing ratio degrades the performance of published methods and that 'retraining alone is not enough to defeat new attacks' (abstract; Sec. VII tables).	Explicitly criticises prior work for using balanced datasets and Alexa domain-only legitimate sets; argues for imbalance-aware evaluation, proper metrics and representative datasets.	**Authors' stated:** Feature reproduction of prior work is approximate. **Our caveats:** 2020-era datasets; still no domain-grouped splitting.	Combining imbalance-aware benchmarking with temporal and domain-grouped splits.	5 / F
[107]	Zeng (2020); 2020 / Proceedings of the Sixth International Workshop on Security…	Lack of diverse, shared datasets and a common benchmarking framework for phishing detection.	Benchmarking framework (PhishBench) for features and classifiers	Diverse phishing email, website and URL datasets released with PhishBench	Framework supports metrics beyond accuracy and imbalanced evaluation	None in abstract. (Full text not accessed; numbers restricted to the abstract/landing page.)	Researchers 'usually test their proposed methods with limited metrics, datasets, and parameters'; a common benchmark is needed.	**Authors' stated:** Workshop paper. **Our caveats:** Datasets are 2019-2020 and website-heavy.	A maintained URL benchmark with leakage controls.	2 / A
[40]	Roy (2024); 2024 / arXiv:2408.05667	Phishing detection usually relies on server-side blocklists/heavy models; need private, real-time client-side detection.	Fine-tune MobileBERT on URL and parsed source code; ensemble; ship as browser extension.	Author-crawled datasets (paper Sec. 3); sizes in paper.	Accuracy/precision/recall/F1 of nine language models; median inference time and memory (Table 2); comparison with anti-phishing tools; zero…	MobileBERT accuracy 0.96 with precision 0.95 / recall 0.96, fastest among SLMs; Llama 2 F1 0.96 but much slower; memory per model ranges from 74 MB to 7,352 MB (paper Table 2).	Small transformer achieves near-LLM accuracy with far lower memory/latency; fully local operation is feasible.	- Authors' stated limitations: Transformer still heavier than classical models; relies on page source for best accuracy. - My assessment: No comparison with lexical gradient-boosting baselines on the…	Head-to-head latency/model-size of lexical ensembles vs. small transformers on identical URL sets.	4 / F
[108]	Verma (2017); 2017 / Proceedings of the 3rd ACM International Workshop on Securi…	Content-based detectors are slow; can URL-only features be extracted and classified fast?	Design lexical/character n-gram features computable in linear time; evaluate several classifiers.	Not verified (abstract unavailable).	Classification accuracy and feature-extraction speed	Not verified.	URL-only features allow fast, accurate detection (per title/abstract framing).	- Authors' stated limitations: n/a - My assessment: Older datasets; no robustness analysis.	n/a	3 / A
[14]	Marchal (2018); 2018 / 11th USENIX Workshop on Cyber Security Experimentation and …	Phishing detectors that score highly in papers rarely work in deployment; dataset composition and evaluation choices are the cause.	Guidelines for ground-truth composition, evaluation metrics and temporal-resilience testing of phishing webpage detectors	Position/guidelines paper; no new dataset	Recommends: legitimate set must include unpopular/long-tail sites, URLs with paths (not just FQDNs from Alexa/DMOZ), real-	No experiments; prescriptive. States that balanced (1/1) datasets 'suffer from base rate fallacy' and that a 1:100 distribution is needed for a meaningful precision (Sec. 3-	Academic detectors look excellent on non-representative data; representativity of the legitimate class (popularity, URL structure) and temporal evaluation are the main missing	**Authors' stated:** Guidelines not empirically validated in the paper. **Our caveats:** Written for webpage (content) detectors, but every data-composition guideline applies	Few later URL papers follow these guidelines; our paper can operationalise them.	5 / F

Ref.	Author (year); venue	Research objective	Method	Dataset	Evaluation protocol	Metrics reported	Main findings	Limitations	Research gap	Rel. / access
					world class distr…	4).	pieces.	verbatim to URL-only det…		
[43]	Das (2020); 2020 / IEEE Communications Surveys & Tutorials	Phishing research ignores security-specific challenges (base rate, active attacker, dataset quality, generalisation) in evaluation.	Critical survey with dataset-property, feature, algorithm and metric analysis	Systematisation of phishing detection literature (email, website, URL) against four 'security challenges': real-time detection, active attacker, dataset qualit…	Reviews evaluation practice and finds: use of balanced datasets and inappropriate metrics, unreported timings, lack of generalisation studi…	Qualitative; tables of dataset sizes/ratios per paper (Sec. V-VII).	Base-rate fallacy, active attacker, dataset diversity and generalisation studies 'are rarely mentioned, let alone emphasized' in prior surveys and papers.	**Authors' stated:** Coverage through 2019. **Our caveats:** None substantial.	Empirical generalisation studies for URL detectors (which later Mia 2024 / Rashid 2024 begin to provide).	4 / F
[109]	Tang (2021); 2021 / Machine Learning and Knowledge Extraction	Compare ML phishing-website solutions, their datasets and reported results.	Comparative review of ML solutions, features, datasets and reported accuracies	Survey; tabulates datasets (UCI, Mendeley, ISCXURL-2016, PhishTank/OpenPhish + Alexa/Common Crawl crawls)	Notes that many solutions report >99% accuracy on single datasets; Table 5 lists 'did not use multiple different datasets to train the mode…	Qualitative; reported accuracies 98-99.6% across surveyed works (Table 5).	Single-dataset evaluation and reliance on PhishTank + Alexa/Common Crawl sources are the norm.	**Authors' stated:** Breadth over depth. **Our caveats:** Does not analyse artefacts or base rates.	N/A	2 / F
[110]	Do (2022); 2022 / IEEE Access	DL phishing detection lacks a taxonomy and faces dataset, metric and drift challenges.	Taxonomy of DL approaches; analysis of datasets, features, metrics, challenges	Systematic review of DL phishing-detection papers	Reports that many works train on old, non-diverse or highly imbalanced data and use accuracy on imbalanced sets	Qualitative; states that 'datasets comprising of old phishing attacks do not represent real attack scenarios', that accuracy/error rate are unsuitable for imbalanced data, and that batch learning ada…	Dataset recency/diversity/imbalance and metric choice are listed as open challenges; recommends online learning for drift.	**Authors' stated:** Review scope; recommends a 'recent and balanced dataset', which conflicts with base-rate advice. **Our caveats:** Their 'balanced dataset' recommendation illustrates the field's …	Clear guidance on realistic prevalence in evaluation.	3 / F
[111]	Zieni (2023); 2023 / IEEE Access	Survey phishing-website detection methods and their data sources and metrics.	Structured review of features, selection, algorithms, metrics	Survey of list-based, similarity-based and ML-based detection; dataset sources (PhishTank, OpenPhish, Alexa, DMOZ, Google queries)	Notes that Alexa retired in May 2022 and DMOZ was discontinued, that datasets 'often share the data sources' yet differ in content, and hig…	Qualitative.	Benign data provenance (Alexa/DMOZ) is fragile and the imbalanced nature of the problem is under-addressed.	**Authors' stated:** Survey. **Our caveats:** No explicit artefact discussion.	N/A	3 / F
[44]	Alam (2026); 2026 / 35th USENIX Security Symposium	AI-based phishing-website detection studies report high accuracy, but it is unclear whether they meet real-world requirements (drift, benign diversity, imbalan…	LLM-assisted extraction plus expert validation against four functionality metrics (incl. F3 robustness on diverse benign pages, F4 evaluation transparency under class imbalance) a…	55 academic AI-based phishing-website detection approaches assessed	Rubric scoring of each paper; taxonomy of detection strategies	No study fulfils all requirements; most risk elevated false alarms due to limited testing on diverse benign pages (long-tail/low-rank domains); many assume static tactics (abstract, Sec. 4-5).	Accuracy-driven evaluation hides blind spots: insufficient benign diversity, no temporally disjoint testing, metrics unsuited to class imbalance (recommends AUC-PR, MCC, class-specific metrics).	**Authors' stated:** LLM extraction validated by experts but still subjective; website-level focus. **Our caveats:** Prepublication version; URL-only works are a subset of the 55.	Same rubric applied specifically to URL-only detectors.	4 / F
[51]	Lundberg (2017); 2017 / Advances in Neural Information Processing Systems 30	Many feature-attribution methods exist with no principled way to choose among them.	Defines the class of additive feature attribution methods, shows a unique solution (SHAP values) satisfying three desirable properties, and gives model-agnostic (KernelSHAP) and m…	None specific to security; experiments on standard ML benchmarks and MNIST-style tasks (see paper Sec. 5).	Theoretical axioms (local accuracy, missingness, consistency) + user-study agreement and sample-efficiency experiments	No security metrics; qualitative/user-study results only (paper Sec. 5, Fig. 4-5).	SHAP unifies LIME, DeepLIFT, layer-wise relevance propagation and Shapley sampling; KernelSHAP is more sample-efficient than LIME-style sampling.	- Authors' stated limitations: Exact Shapley computation is exponential; KernelSHAP is an approximation and assumes feature independence under the background distribution. - My assessment: Explanatio…	Little guidance on explanation stability under input perturbation, which we need for mutation-robustness tests.	5 / F
[52]	Lundberg (2020); 2020 / Nature Machine Intelligence 2(1):56-67	Tree ensembles (RF, GBDT) are accurate but opaque; heuristic feature importances are inconsistent.	Exact SHAP computation for trees (TreeExplainer), SHAP interaction values, and aggregation of local explanations into global importance, dependence and clustering plots.	Three medical datasets (NHANES mortality, chronic kidney disease, hospital procedure duration); not security data.	Benchmarks of 13 explanation methods on consistency/accuracy metrics; medical mortality/kidney/hospital datasets	Method-comparison benchmark scores reported in the paper's Fig. 2/3; no classification accuracy relevant to us.	TreeExplainer is exact and fast; gain/split-count importances can be inconsistent; local explanations reveal interaction effects and dataset shift.	- Authors' stated limitations: Interventional vs. observational conditional expectation choice affects attributions; limited to tree models. - My assessment: Our ensemble (if it mixes non-tree models…	No study of explanation drift between training and external datasets for security tasks.	5 / F
[53]	Al-Fayoumi (2024); 2024 / International Journal of Online and Biomedical Engineering …	ML phishing-URL detectors lack transparency and trust.	Compute SHAP values, retain most influential features, train a lightweight gradient boosting classifier.	Not specified in the abstract (full text not retrieved).	Hold-out classification; accuracy, F1, per-record prediction time	Abstract reports 99.8% accuracy and 99% F1 and 'very low' per-record prediction time (abstract only; tables not seen).	SHAP-driven feature pruning keeps accuracy while reducing cost.	- Authors' stated limitations: Not available (abstract only). - My assessment: Venue is low-tier; 99.8% accuracy suggests possible leakage/within-dataset optimism; no external test.	No robustness or calibration analysis; no cross-dataset validation.	3 / A
[54]	Kehkashan (2025); 2025 / Scientific Reports 15, article 41751	Phishing detectors are resource-heavy and opaque.	Rank features with SHAP, retain top features, train five classical classifiers; ablation with/without SHAP selection.	UCI Phishing Websites dataset (Mohammad et al.), 11,055 records, 30 engineered features (the abstract instead mentions a Kaggle dataset of >11,000 URLs; the pa…	Hold-out; accuracy, precision, recall, F1; ablation without SHAP	RF with SHAP selection: accuracy 97.0%, precision 97.0%, recall 97.6%, F1 97.3%; RF without SHAP 90.0% (ablation); other models 93.3-96.0% (results section/tables as read on PMC).	SHAP-guided feature selection improved all classifiers; RF best.	- Authors' stated limitations: Single small dataset; classical models only. - My assessment: UCI dataset is 2015-era with pre-extracted features, so no raw URLs and no mutation tests are possible; in…	No raw-URL lexical pipeline, no latency or calibration reporting.	3 / F

Appendix B. Literature Review

Version 1, 2026-10-11. Scope, sources and screening are in §1; the per-paper evidence is in `reading_notes/<citekey>.md`; the comparison table is `LITERATURE_SURVEY_TABLE.md`; metadata in `paper_metadata.csv`; bibliography in `references.bib`.

Reading conventions. Numbers quoted below are the numbers *reported by the cited authors*; none was reproduced by us unless explicitly marked "(ours)". Scores from different papers are **not comparable** with each other: datasets, label definitions, class ratios, splits and metrics all differ. Where a note is based on the abstract or publisher page only, the paper is marked †. Preprints are marked ‡.

1. Method of the review

1.1 Search and screening

Three parallel searches (2026-10-11) covered (L1) detection methods, (L2) evaluation realism, and (L3) explainability, robustness, calibration, efficiency and datasets. Engines and indexes: web search, arXiv API, Crossref, Semantic Scholar, OpenAlex, USENIX/NDSS proceedings pages, IEEE Xplore/ACM/Springer/ScienceDirect/MDPI landing pages, UCI, Mendeley Data, Zenodo. Query strings and inclusion/exclusion decisions are logged in `search_log_L1.md`, `search_log_L2.md`, `search_log_L3.md`.

Inclusion: peer-reviewed or clearly labelled preprint work on (a) URL-based malicious/phishing detection, (b) evaluation methodology for security ML that applies to it, (c) datasets and feeds used in this study, (d) explainability, calibration, robustness or efficiency relevant to URL classifiers. Exclusion: papers whose identity (title, authors, venue, DOI/arXiv id) could not be confirmed on an authoritative page; content/visual-only phishing detectors unless they inform evaluation; vendor marketing.

1.2 Outcome

	Count
Unique verified sources	111 after removing 3 duplicate records of the same DOI (104 fully verified; 7 partially verified, e.g. no DOI or venue page available)
Full text read: 70 legal open-access PDFs in `papers/`, plus dataset/feed pages read as primary sources	75
Abstract / publisher page only (marked †)	35
Paywalled, metadata only	1
Candidate items rejected as unverifiable	9 (listed at the end of each search log; e.g. a guessed DOI for Marchal et al. *Off-the-Hook* that resolved to another paper)

Corrections found during verification and propagated to the bibliography: URLTran was published at **MILCOM 2021**, not IEEE BigData [35]; URLBERT appeared in *Computer Networks* 2025 under a new title [38]; arXiv 2411.09813 is first-authored by **Maraz Mia** [9]; the DeepPhish work is verifiable only as a Black Hat talk [69].

This review is systematic and traceable but **not exhaustive**: ACM/IEEE full texts behind paywalls were not read, and Kaggle-hosted notebooks and grey literature were excluded.

2. Lexical and handcrafted URL features

The field's foundation is the observation that the URL string alone carries strong signal. McGrath and Gupta measured, on 2007–08 PhishTank/MarkMonitor feeds against DMOZ, much longer URLs (modal length 67–107 vs 22 characters), brand tokens in 50–75 % of phishing URLs, and frequent subdomains [18]. Garera et al. built a taxonomy of URL obfuscation (IP hosts, redirection, brand in host/path) [19†]. Ma et al. showed that tens of thousands of automatically generated bag-of-token features plus host properties reach 95–99 % accuracy, and — importantly for us — that **training on one data source and testing on another raised error to roughly 14–21 %** (their Table 4) [3]; their online-learning follow-up used a 100-day, day-ahead protocol on ~2 M web-mail URLs and showed that stale batch models

degrade [20, 21†]. Subsequent lexical work added obfuscation-resistant features and online AROW learning [4], online confidence-weighted learning [22†], character-distribution statistics [23†, 108†], word-level NLP features [6†] and search-engine word relatedness [24]. ISCX-URL-2016 released a five-class lexical benchmark [5]. The survey by Sahoo et al. consolidated the area and listed the absence of standardised benchmarks, concept drift and adversarial evasion as open challenges [68].

Common thread relevant to us. Almost every foundational dataset pairs a phishing feed (PhishTank, MarkMonitor) with a benign *directory or popularity list* (DMOZ, Yahoo random, Alexa). Several authors note in passing that the benign side is homepage-heavy [18, 24], which makes length, path depth and token counts separable for reasons unrelated to maliciousness.

3. Classical classifiers, ensembles and hybrids

Tree ensembles dominate tabular URL work. Random forests were best in ISCX-URL-2016 (≈99 % per class) [5], CatchPhish (TF-IDF + handcrafted, 94–98 %) [26†], hybrid feature selection (94.6 % with 10 of 48 features) [27†] and multi-dataset studies (91.6–96.9 % across six datasets) [82†]. Ensembles and stacking add little: on the 651k Kaggle corpus, stacking reached 96.8 % against 95.7 % for LightGBM [7]; on a 248k Kaggle feature table, a Bayesian-tuned stacking ensemble matched but did not exceed Extra Trees in AUC (0.9947 vs 0.9948) [8]; PhishHaven's voting ensemble reported 98 % on Alexa homepages vs generated URLs [81]. PhiUSIIL combined URL similarity indices with incremental learning and released the 235,795-URL dataset we use as Corpus A [25†]. Noisy feature-subset voting ensembles were proposed as a robustness device [71].

Assessment. The marginal gain of hybrid ensembles over a single well-configured tree ensemble is small, and is reported almost exclusively on random splits of a single dataset. The audited repository repeats this pattern: its 1:2:2 soft vote does not beat its own random forest (`REPOSITORY_AUDIT.md` §5).

4. Character-level and sequence models

Character CNNs (eXpose: +5–10 % detection rate at 0.1 % FPR over manual features on proprietary data [28‡]), character LSTMs (98.7 % vs 93.5 % for RF on 2 M PhishTank/Common Crawl URLs with 3-fold CV [29]) and joint character+word CNNs (URLNet, with a time-ordered 60/40 split and TPR at FPR 10^{-4}–10^{-1} [30‡]) established learned representations. Later variants: CNN+attention RNN [85†], BiLSTM+CNN [86†], Inception-style char/word CNN evaluated at 0.01 % FPR on production telemetry [31], char-CNN with stripped `http://`, `https://` and `www.` prefixes [32], mobile-sized CNN [87†], DNN-LSTM [89†], n-gram embeddings with attention (GramBeddings, 800k URLs collected 2019–2021) [33†], and a 5.1 M-URL CNN study [34]. Alshingiti et al. use CNN/LSTM on 30 pre-extracted features rather than raw characters [88].

Two observations matter for our design. First, Aljofey et al. **explicitly remove the scheme and `www.` prefix** before modelling [32] — an implicit acknowledgement that these tokens are recording artifacts — yet they do not measure the effect. Second, the production studies with temporal splits and low-FPR operating points [30, 31, 35] report far lower true-positive rates at realistic FPRs than the 97–99 % accuracies typical of public-benchmark papers, which illustrates how much evaluation design changes the picture.

5. Transformers, pre-trained URL encoders and LLMs

URLTran fine-tuned BERT/RoBERTa on six weeks of Microsoft telemetry (temporal split, 1:20 class ratio) and reached TPR 86.8 % at FPR 0.01 % vs 71.2 % for URLNet [35]. Generic NLP pre-training did not transfer straightforwardly in Rudd and Abdallah's study, where a mixed-objective transformer only matched a 1-D CNN [36‡]. URL-specific pre-training at billion-URL scale followed (PhishBERT [37†]; URLBERT, with TPR ≈ 0.8 at FPR 10^{-3} and the observation that classical KNN/RF/NB still exceed 96 % accuracy on the same benchmark [38]; TransURL [39]; PMANet [95†]). Lightweight transformers for

client-side use report 74 MB–7.3 GB memory per model and MobileBERT accuracy 0.96 [40‡, 91†]. LLM work spans prompting vs fine-tuning on the 11k Hannousse set (fine-tuned GPT-2 F1 97.3 %, prompted models degrade under class imbalance) [92], least-to-most prompting (average F1 0.904) [98‡], frozen LLM embeddings with a small head on the 651k Kaggle set (97.5 %) [94], LoRA fine-tuning [93†] and multimodal agents [97‡]. A 2025 survey notes the lack of proper baselines and standard benchmarks [96‡].

Assessment. TransURL is the only learned-representation paper in our set with a substantial cross-dataset experiment: all models drop sharply (GramBeddings to ≈70 % accuracy), TransURL least (90.2 %) [39]. Its Table 2 also documents TLD and length distribution differences between datasets — candidate artifacts that were not analysed further.

6. Evaluation realism: drift, bias, leakage and base rates

This is the body of work our study builds on most directly.

General security-ML pitfalls. Arp et al. catalogue ten pitfalls in 30 top-venue papers; sampling bias (P1), spurious correlations — "artifacts unrelated to the security problem create shortcut patterns for separating classes" (P4) — and the base-rate fallacy (P8) are each present in most surveyed work [13]. Sommer and Paxson's closed-world critique [106†] and Axelsson's base-rate analysis [105†] are the classical antecedents. Apruzzese et al. show that single-dataset, single-run results can change method rankings [42] and that real attackers use cheap problem-space manipulations [50]. Malware research reached similar conclusions about dataset composition [79, 78†].

Temporal bias and drift. TESSERACT formalised temporal and spatial bias and showed that time-aware evaluation lowers F1 well below published 10-fold numbers (AUT(F1, 24 m) 0.32–0.64) [41, 63‡]. Transcend and TRANSCENDENT detect drift and reject unreliable predictions [59, 60]; continuous learning reduces but does not remove decay [61]. For phishing specifically: content-based detectors lost 20.65 points over 2018–2020 without continual learning [62]; URL models trained on 2016 data degrade on 2020 URLs [12]; drift studies remain largely e-mail-centred [65‡] or report near-perfect drift-aware accuracy without cross-source validation [64†]. Phishing sites live hours to days [1], and blacklist-derived labels are delayed and biased towards non-evasive phish [2]. Pendlebury et al. themselves note that public URL corpora lack first-seen timestamps, which prevents TESSERACT-style evaluation [41].

Dataset bias and benign-class composition. This is where the evidence is strongest and most specific:

- Using Alexa index pages as the benign class "creates a bias in the data" because phishing feeds contain full URLs [17].
- Benchmarks are "dataset dependent and far away from generalization"; Alexa lists should not be used directly because they lack sub-domains and paths [16].
- Prior work uses balanced datasets and Alexa domain-only benign sets; performance degrades as the legitimate:phishing ratio grows [15]; generalisation studies "are rarely mentioned" [43].
- The benign set must include unpopular sites and URLs with paths, at about 1:100 prevalence, with temporal tests [14].
- Models trained on homepage-vs-phishing data **lose 27–39 % accuracy on legitimate login URLs** (best 69.5 %), and URL-length separability largely disappears [12].
- Top lists are unstable, manipulable and differ structurally; any list is a proxy for popularity, not legitimacy [77, 76]; Alexa itself was retired in 2022 [100, 111].
- Most of 55 surveyed phishing detectors risk high false alarms because of limited benign diversity [44‡].
- Public datasets "suffer from leakage and unrealistic base rates" [45‡]; synthetic debiasing has been proposed [80‡]; benchmarking frameworks exist [107†].

- Surveys repeatedly flag single-dataset evaluation, old data and accuracy on imbalanced sets [110, 109, 111].

Cross-dataset generalisation. Cross-dataset accuracy falls to 51 % and 59 % between two public URL feature datasets, with SHAP showing the same features changing sign [9]; detectors "work effectively only when the test sets are split from the same training dataset", with URL recording format named as a cause [11†]; a 57.4 % cross-dataset baseline between UCI and PhiUSIIL [10†]; tree ensembles averaging 94 % over four secondary test sets [83†] (sources of the test sets could not be checked); and an external benign test with FPR 86.84 % (95 % CI 83.6–90.0) after canonicalisation in a single-author preprint that names the `https://` and `www.` shortcut directly [46‡]. For phishing e-mail, a dataset-identification probe reached 97–98 % accuracy and artifact-removal ablations were used to expose corpus markers [47].

7. Explainability

SHAP provides additive attributions with consistency guarantees [51]; TreeExplainer computes them exactly for tree ensembles and supports global summaries [52]. In phishing work, SHAP is used mostly for feature selection (XAI-PhD, 99.8 % accuracy reported [53†]; Kehkashan et al., RF 97.0 % on the UCI set [54]; Khan et al. [8]) or for post-hoc justification [83†]. Only Mia et al. use SHAP *diagnostically*, to show feature effects differ across datasets [9]; Arp et al. recommend explanation-based artifact checks in general [13]. Lundberg and Lee note that KernelSHAP assumes feature independence, which is violated by correlated lexical counts [51] — the audited repository uses KernelSHAP on the full soft-vote, at high cost.

8. Adversarial robustness and obfuscation

Fifty reproduced URL detectors fell from median MCC 0.92 to 0.02 on cheaply registrable adversarial URLs (URLBUG: homoglyphs, hyphenation, subdomain insertion, character swaps) [17]. SpacePhish distinguishes problem-space from feature-space perturbations and finds statistically significant 3–10 % degradation from cheap evasions [48]. GAN- and LSTM-generated URLs evade detectors [49†, 69], as do black-box attacks on ad-URL classifiers [73‡]. Measurement studies describe the realistic threat model: IDN homographs still bypass browser defences [72], squatting phishing domains evade blacklists for weeks [75], combosquatting is long-lived [74], and phishing kits cloak against crawlers [70]. URLTran's adversarial fine-tuning preserves low-FPR behaviour against homoglyph and compound-split attacks [35].

Gap relevant to us. Robustness studies measure *adversarial* manipulations; none measures sensitivity to *label-irrelevant recording variants* (scheme present or absent, `www.` present or absent, default port, fragments), which our audit shows is where lexical models are most fragile.

9. Calibration and operating points

Boosting pushes probabilities away from 0 and 1 while bagged trees are near-calibrated; Platt and isotonic scaling correct this given enough held-out data [56]. Guo et al. popularised ECE and reliability diagrams and warned that post-hoc calibration assumes the calibration set matches deployment [57]. For Android malware, calibration degrades under dataset shift and fails under adversarial examples [55]. Production URL papers report TPR at fixed low FPR [28, 30, 31, 35, 38]; most public-benchmark papers report accuracy at 0.5. **No study in our set measures calibration of phishing-URL classifiers or whether validation-chosen FPR thresholds hold under cross-dataset shift** (confirmed by targeted searches logged in `search_log_L3.md`).

10. Efficiency and deployability

Lexical tree models are fast and small; char-LSTMs were ≈3× slower than RF in Bahnsen et al. (281 vs 942 URLs/s) [29]; PDRCNN reports 0.4 ms/URL [86†]; frozen-LLM embeddings 0.119 ms/sample excluding embedding cost [94]; MobileBERT-class models are the lightest transformers but still 74 MB+

[40‡]; agents need ≈1 s/URL [97‡]. External lookups (WHOIS, search engines) cost seconds per URL [4, 24]. Head-to-head latency and size comparisons on identical hardware and data are rare.

11. Datasets

Dataset	Size / classes	Benign source	Timestamps	Known issues (literature + our audit)
ISCX-URL-2016 [5]	~110k, 5 classes	Alexa-derived crawl	no	source-per-class; our audit: benign from ≈300 sites, deep paths
PhishStorm [24]	96,018 balanced	DMOZ	no	homepage-heavy benign (paper); ours: 70 % `www.` benign vs 35 % phish
Ebbu2017 [6]	73,575	Yandex search results	no	many duplicates (ours: 26 % removed)
Hannousse 2021 [16]	11,430 balanced	Alexa crawl *with paths*	no (May 2020)	small, balanced
PhiUSIIL [25]	235,795	—	no	ours: 100 % of benign are `https://www.<domain>` homepages
GramBeddings [33]	800k balanced	top sites crawl	collection window 2019–21	access by form (not obtained)
PILU-90K [12]	90k incl. 30k login URLs	Quantcast	no	not obtained in this study
DEPHIDES [34]	5.1 M	≈30 URLs per top domain	no	random splits leak domains
Kaggle sid321axn [58]	651,191, 4 classes	aggregated	no	mixed sources; scheme often absent
PhreshPhish [45‡]	large, HTML+URL	browser crawl	temporal split	not used (HTML-centred)
Feeds: PhishTank, OpenPhish, URLhaus [103, 102, 104]; Tranco [77]; Common Crawl [101]	live	—	PhishTank/URLhaus yes	positive-only (feeds) or not labels (lists)

12. Synthesis: what is established, what is not

Established by prior work.

6. Lexical URL features and simple tree ensembles reach very high in-distribution scores on public benchmarks.
7. Cross-dataset performance of URL classifiers collapses, often to near chance (A: [3, 9, 11, 10, 39]).
8. Benign-class sampling from popularity lists produces biased, homepage-heavy benign data (A: [17, 16, 14, 15, 12]).
9. Random splits, balanced classes and accuracy over-state deployability; temporal and low-FPR evaluation is needed (A: [13, 41, 14, 35]).
10. URL detectors are fragile to cheap adversarial manipulation (A: [17, 48]).

Not established (gaps; developed in `RESEARCH_GAPS.md`).

- (I) A *quantified, multi-corpus, raw-URL* account of which specific recording artifacts carry in-distribution performance, whether their direction is consistent across corpora, and how much registered-domain memorisation adds. Prior cross-dataset studies used pre-computed feature tables [9, 10] or abstracts do not report the mechanism [11]; the only direct treatment is a non-peer-reviewed single-model preprint [46].
- (I) Whether removing artifacts at the *feature* level suffices, versus canonicalising the input, and whether either restores external performance.
- (A for malware, I for URLs) Calibration and threshold transfer of phishing-URL classifiers under dataset shift [55].
- (I) Sensitivity of URL classifiers to label-irrelevant recording variants (scheme/`www.`/port/fragment), as distinct from adversarial perturbations.
- (A) Public URL corpora lack timestamps; long-horizon temporal evaluation of URL models on public data is missing [41].

13. How this project differs

Closest work	What it does	What we add

Closest work	What it does	What we add
[9]	2 datasets, pre-computed features, random splits, SHAP	raw URLs from 6 corpora + 3 positive-only feeds; ablation of named artifacts; grouped splits; probe; calibration
[11]	domain adaptation between 3 datasets	mechanism (artifact direction) rather than adaptation
[46] ‡	one model, canonicalisation, external benign set	multi-model, multi-seed, paired statistics, 6-way leave-one-corpus-out, probe, calibration, temporal recall
[12]	homepage vs login benign shortcut; 2016→2020 decay	generalises the shortcut analysis across corpora and recording formats
[47]	probe + artifact ablation for e-mail	same diagnostic design for URLs
[8], [7]	stacking on one Kaggle set, random split	shows why such results do not transfer

Appendix C. Research Gaps, Hypotheses and Selected Direction

Version 1 — 2026-10-11, written after the literature collection (`paper_metadata_L1–L3.csv`, `reading_notes/`) and the repository and dataset audits. Citekeys refer to `references.bib`. Each gap states whether it is **acknowledged by prior authors** (A) or **inferred by us from comparing studies** (I).

0. What the audits established (our own evidence, not literature)

- The baseline ensemble's two most influential features are `https_usage` and a `www.`-driven subdomain count; it labels `https://google.com/` malicious with 88 % confidence (`REPOSITORY_AUDIT.md` §7).
- In PhiUSIIL, the closest public analogue of the project's data, **100 %** of benign URLs are `https://www.<domain>` homepages with an empty path, while phishing URLs are 48 % https / 42 % `www.` / 72 % homepage (`DATASET_AUDIT.md` §3). The `www.` and length artifacts point in opposite directions in different public corpora (AUC 0.21 vs 0.71 for `www.`).
- A 20k-row smoke experiment: lexical models reach ROC-AUC 0.999 on a grouped PhiUSIIL split and then flag **98–99 % of legitimate Hannousse-2021 URLs** as phishing (`results/runs` smoke, to be replaced by the full runs).

1. Candidate gap G1 — Collection artifacts and shortcut learning in URL benchmarks (A + I)

Supporting literature. General security-ML pitfalls: sampling bias → spurious correlations → base-rate fallacy chain [13]; closed-world evaluation [106]; base rates [105]. Phishing-specific: Alexa index pages vs full phishing URLs "creates a bias in the data" [17]; findings "dataset dependent and far away from generalization", do not use Alexa lists directly [16]; balanced data and domain-only benign sets criticised [15, 43, 14]; cross-dataset accuracy near chance with SHAP sign flips [9]; detectors "work effectively only when the test sets are split from the same training dataset", with URL recording format named as a cause [11]; a 2026 single-author preprint reports the `https://www.` shortcut and near-100 % prediction instability under reformatting [46]; corpus-identification probes and artifact-removal ablations exist for phishing *e-mail* [47]; insufficient benign diversity in 55 detectors [44]; leakage and base rates in web-page benchmarks [45].

What earlier work accomplishes. Documents cross-dataset drops (Mia, Rashid, Henry), prescribes guidelines (Marchal, El Aassal, Arp), and — in one unreviewed preprint — names the scheme/`www.` artifact.

Unresolved limitation. No peer-reviewed study (a) quantifies, across several public URL corpora, how much of the in-distribution performance of standard lexical detectors is carried by a small set of *nameable* collection artifacts; (b) shows that the *sign* of those artifacts differs between corpora (which mechanistically explains cross-dataset collapse); (c) measures how much domain memorisation (random vs registered-domain-grouped splits) adds on top; or (d) tests whether removing the artifacts at feature level is sufficient to generalise. Prior cross-dataset work used pre-computed feature tables rather than raw URLs, so artifacts could not be isolated.

Why it matters. Dozens of papers report > 97 % accuracy with such features; deployment on real traffic (where benign URLs have paths and phishing URLs are https) gives the FPR observed in our smoke test, which would make a product unusable.

Repository shortfall. The baseline is a textbook instance: a random split, one-hot TLDs, no external test, scheme/`www.` features on top.

Technical solution. Artifact-aware evaluation protocol: per-class artifact statistics; artifact-feature ablation; registered-domain-grouped splits; dataset-identification probe; cross-corpus transfer matrix; models re-trained with artifact-neutral features.

Hypotheses.

- H1a: Removing `{scheme, www, label-count}` features reduces in-distribution AUC on PhiUSIIL only marginally (< 0.01) but changes external-set FPR by a large margin (> 20 points) for tree ensembles.
- H1b: Random splits over-estimate grouped-split performance, with the gap largest on corpora with few domains per URL (ISCX-2016, Kaggle lineage).
- H1c: A linear probe can identify the source corpus of a benign URL from lexical features far above chance (≥ 2× chance), and removing artifact features reduces but does not eliminate this.

Experiment. Runs `A_b28_*`, `A_v2_*`, `*_noart_*`, `B_*` with external tests (Sections 4–6 of `EXPERIMENT_PROTOCOL.md`); `analysis_dataset_probe.py`; `analysis_dataset_artifacts.py`.

Measurable outcomes. ΔAUC / ΔFPR / ΔTPR@FPR=1 % between (random, grouped) and (with, without artifacts), with paired bootstrap CIs over 5 seeds; probe accuracy vs chance.

Risks / competing approaches. Unsupervised domain adaptation [11] and transfer learning [10] attack the symptom (shift) rather than the cause; canonicalisation [46] is a special case of our artifact-neutral features. Risk: artifact removal may expose that *nothing* transfers, in which case the paper's contribution is diagnostic rather than a better detector — still valid and reportable.

2. Candidate gap G2 — Operating-point reliability: calibration and FPR-constrained thresholds under shift (A + I)

Supporting literature. Boosted trees are systematically mis-calibrated and Platt/isotonic correct this [56]; calibration of modern models [57]; in security, calibration degrades under dataset shift and collapses under adversarial inputs for Android malware [55] — the authors and our searches found **no equivalent for phishing URLs**; prevalence shift makes fixed thresholds fail asymmetrically [47]; realistic prevalence 1:100 or lower [14, 45, 41]; metrics for imbalance [44].

Unresolved limitation. Phishing-URL papers report accuracy/F1 at threshold 0.5 on balanced data. Whether a threshold chosen for a 1 % FPR budget on validation *holds* on external data, and whether post-hoc calibration helps or hurts under the artifact-induced shift, is unmeasured.

Repository shortfall. Threshold 0.5, uncalibrated soft-vote probabilities presented as "confidence".

Solution. Validation-only threshold policies (max-F1; FPR ≤ 1 %, 0.1 %), isotonic/Platt calibration fitted on validation, ECE/Brier/reliability diagrams, in-distribution vs external.

Hypotheses. H2a: Thresholds set at 1 % validation FPR produce external FPR an order of magnitude higher for artifact-bearing models and closer to target for artifact-neutral ones. H2b: Isotonic calibration lowers in-distribution ECE but does not repair external calibration (shift dominates).

Experiment / outcome. `analysis_calibration.py` on every run; report realised FPR at the validation-chosen threshold on each external set; ECE before/after.

Risks. Positive-only external sets (PhishTank) support recall but not FPR; FPR claims rest on Hannousse, Ebbu2017, PhishStorm, ISCX.

3. Candidate gap G3 — Temporal recall of lexical detectors over a 15-year phishing stream (I)

Supporting literature. Temporal bias and time-decay metrics [41, 63]; concept-drift detection [59, 60]; phishing life cycle and blacklist latency [1, 2]; concept drift in phishing detectors [65, 62, 64]; public URL corpora lack timestamps (noted by [41] for malware and by us for URLs).

Unresolved limitation. No study scores a frozen URL classifier against a long, timestamped phishing stream to show how recall decays with the move to https phishing (PhishTank 2026: 92 % https). Existing drift papers use synthetic or short windows.

Solution. Score frozen models (trained on A/B) on the 67k PhishTank URLs grouped by submission year/month; recall curves per feature set.

Hypothesis. H3: Recall of artifact-bearing models decreases monotonically with phishing year (as https adoption rises), while artifact-neutral models are flatter.

Risks. PhishTank's 2011–2024 portion is small (most entries are recent); it is positive-only, so this is a *recall-over-time* study, not a full temporal benchmark; PhishTank itself is a source for several training corpora (lineage overlap for B).

4. Candidate gap G4 — Lightweight character model vs handcrafted features under the grouped/external protocol (I)

Supporting literature. Char-level CNNs and transformers outperform handcrafted features in-distribution (collected by L1: URLNet, eXpose, Texception, URLTran, GramBeddings, PhishLang [40]); but these are evaluated on random splits; GramBeddings and PhishLang discuss efficiency. Cross-dataset for deep URL models: [11] (partial), [10].

Unresolved limitation. Whether learned character representations are *more* or *less* artifact-dependent than handcrafted features under grouped and external evaluation is not established with controlled artifact ablation (char models see the scheme and `www.` tokens directly).

Solution. Small char-CNN and a hybrid char+lexical model under the identical protocol, including Tier-B mutation tests (toggle scheme/`www.`).

Hypothesis. H4: The char-CNN matches tree ensembles in-distribution but shows higher flip rates under scheme/`www.` toggles unless trained on canonicalised input.

Risks. CPU-only budget limits model size and seeds (3); no transformer baseline — stated as a limitation.

5. Direction not pursued

- *Adversarial attacks with gradient-based or GAN-generated URLs* [49, 69, 73]: well covered by [17, 48]; our budget is better spent on label-preserving, deployment-realistic variants.
- *Content/HTML or host-based enrichment*: changes the problem (online lookups, latency, privacy); out of scope for an offline URL detector.
- *New ensemble architectures*: the audit shows the existing soft-vote does not beat RF; novelty by combination is not warranted.

6. Selection

Criterion	G1 artifacts	G2 calibration	G3 temporal	G4 char models
Problem importance	very high	high	high	medium
Evidence unresolved	A + I, strong	A (no URL study)	I	I
Difference from prior work	first multi-corpus, raw-URL, ablation-based quantification	first for URLs	first long-horizon recall curve	incremental
Feasibility (CPU, data in hand)	yes	yes	yes (positive-only)	yes, limited seeds
Fair measurability	yes (paired, grouped)	yes	partial	yes
Practical usefulness	high (explains deployment FPR)	high	medium	medium

Primary direction: G1, with **G2** as the operational complement (together: an *artifact-aware evaluation and training protocol*). **G3** and **G4** are secondary analyses executed under the same runs. The contribution is therefore: (1) a diagnostic showing that reported performance of lexical URL detectors is carried by nameable collection artifacts whose direction is corpus-specific; (2) a protocol (grouped splits, artifact-neutral features, validation-only FPR thresholds, calibration, external and temporal tests)

implemented as reusable, tested code; (3) empirical answers on whether artifact-neutral training recovers external performance. All three remain **provisional until the runs in `results/runs/` are complete**.

7. Novelty check (closest prior work and how we differ)

Work	Overlap	Difference
[46] (Zenodo preprint, single author)	names https/www shortcut; external benign test	we: peer-review-grade protocol, 6 corpora, 5 seeds, paired statistics, artifact *direction* analysis, probe, grouped splits, calibration, temporal recall, three model families
[9]	cross-dataset + SHAP	they use pre-computed feature tables (cannot ablate artifacts); two datasets; no grouped split
[11]	names recording-format shift	proposes domain adaptation; no artifact ablation; abstract-only to us
[47]	probe + ablation design	e-mail text, linear models
[17]	Alexa index-page bias noted	focus on adversarial URLs, not on evaluation protocol
[16]	dataset-dependence	provides a dataset + guidelines; no artifact quantification

Appendix D. Repository Audit of the Original Implementation

Audit date: 2026-10-11. Audited commit: `65364b4` ("Initial commit - Malicious URL Detection project"), the only commit in the repository. Research work proceeds on branch `research-v2`; the original code is preserved on `main`.

Method: every file in the repository was read in full. Notebook execution history (stored outputs, execution counts, kernel metadata) was extracted programmatically and compared with the Python modules. No stored output was taken as proof of reproducibility; see Section 6.

1. Repository map

Path	Size	Status	Notes
`app.py`	1.4 KB	Functional but fragile	Flask API; loads `weighted_soft_voting_ensemble.joblib` from the working directory (not `models/`); serves `index.html` which does not exist (the file is named `idex.html`). Bare `except:`. Reports a "confidence" and labels a URL "Legitimate".
`feature_extractor.py` (root)	5.9 KB	Functional	Byte-identical duplicate of `src/features/feature_extractor.py`.
`idex.html`	4.6 KB	Functional UI	Calls `/api/predict`; shows a green check for "Legitimate". Misnamed.
`src/config.py`	0 B	Empty	
`src/utils.py`	0 B	Empty	
`src/data/load_data.py`	0 B	Empty	
`src/data/data_cleaning.py`	1.7 KB	Implemented, **not used by notebooks**	`normalize_url` lowercases scheme and netloc and rebuilds the URL. Notebook 02 only strips whitespace and lowercases labels, so training data never went through this module. Also `clean_dataset` triggers a pandas `SettingWithCopy` pattern (slice then assign).
`src/data/data_splitting.py`	1.0 KB	Implemented, used by NB 05	Stratified random 60/20/20 split, `random_state=42`. Maps `legitimate→0`, `phishing→1`. No grouping by domain, no temporal option.
`src/features/feature_extractor.py`	5.9 KB	Implemented, used by NB 03 (copied inline) and NB 09	28 features; see Section 3.
`src/features/url_features.py`, `domain_features.py`, `security_features.py`	0 B	Empty placeholders	
`src/models/logistic_regression.py`, `random_forest.py`, `xgboost.py`, `ensemble.py`	0.5–0.6 KB	Implemented, used by NB 06	Factory functions; fixed hyperparameters (no tuning anywhere).
`src/evaluation/metrics.py`, `confusion_matrix.py`	0.7–1.0 KB	Implemented, used by NB 07	Accuracy/precision/recall/F1 + confusion counts only. No ROC-AUC/PR-AUC/FPR/FNR/specificity.
`src/evaluation/model_comparison.py`	108 B	Trivial	Wraps `pd.DataFrame`.
`src/explainability/shap_analysis.py`	4.9 KB	Implemented, partially broken	`SHAPAnalyzer` uses `KernelExplainer` on the full ensemble; `plot_shap_summary` indexes `shap_values[1]` which is wrong for the 3-D array returned by recent SHAP versions (NB 08 works around this inline with `shap_values[:, :, 1]`). Output filename differs from the one actually used in NB 08.
`src/prediction/predict_url.py`	0 B	Empty	Prediction logic lives only inline in NB 09 and `app.py`.
`tests/test_*.py` (4 files)	0 B	**Empty — no tests exist**	
`notebooks/01–09`	—	Executed (outputs stored)	Kernel Python 3.13.9, Anaconda; absolute paths `C:\Users\Jiya\Documents\Malicious URL Detection\…`.
`results/**`	~5 MB	Present	Metrics CSVs, figures, SHAP CSV, 6 example predictions.
`.gitignore`	51 B		Ignores `*.joblib`, `__pycache__/`, and (malformed line) `.ipynb_checkpoints/data/ `. The intention was presumably to ignore `data/`; the `data/` and `models/` directories are absent from the repository.
`requirements.txt`, `README.md`, `setup.py`, config files	—	**Absent**	No dependency specification, no documentation, no CLI.

Dependencies inferred from imports and notebook pip output: pandas 2.3.3, numpy 2.3.5, scikit-learn 1.7.2, xgboost 3.4.1, shap 0.52.0, matplotlib, seaborn, joblib, flask.

2. Current architecture (as executed)

```
data/raw/raw_data.csv (253,098 rows: url,label)         [NOT IN REPO]
   └─ NB02: strip whitespace, lowercase label           → data/cleaned/cleaned_urls.csv
   └─ NB03: extract_features (28 lexical features)      → data/processed/url_features.csv
   └─ NB05: one-hot tld_type (521 values → 548 cols);
            stratified 60/20/20 split (seed 42);
            fit LR / RF / XGB pipelines (median imputer [+scaler])
   └─ NB06: VotingClassifier(soft, weights 1:2:2) over fresh LR/RF/XGB → models/*.joblib
   └─ NB07: test metrics, confusion matrices, ROC/PR figures
   └─ NB08: SHAP (KernelExplainer, 500 validation rows, 100 background rows)
   └─ NB09 / app.py: single-URL prediction, reindex to 548 training columns
```

Label encoding: `phishing = 1` (positive class), `legitimate = 0`. The dataset label is "phishing", but the app and notebooks describe outputs as "Malicious"; the two terms are used interchangeably in the project.

3. Features (28 defined, 548 after one-hot)

The 28 features are purely lexical (no host/WHOIS/content lookups), which is good for offline deployment. Observations from reading the code:

- `num_subdomains = max(len(host.split('.')) - 2, 0)`: counts labels, so `www.example.com` → 1 and `example.com` → 0; `example.co.uk` → 1 (wrong). Uses no public-suffix list.
- `tld_type` is the last hostname label, then one-hot encoded into 521 columns at training time; at prediction time unseen TLDs silently become all-zero rows (`reindex(fill_value=0)`). This makes the feature matrix sparse, inflates dimensionality and lets rare TLDs memorise individual sources.
- `hexadecimal_characters` counts every two-character `[0-9a-f]{2}` run anywhere in the URL, so it fires on ordinary words ("ca", "de", "fa", "ce"...). It is a noisy proxy for URL length rather than hex encoding.
- `https_usage`: scheme == https. In the dataset, 93.2% of legitimate URLs but only 50.2% of phishing URLs are https (NB04 `feature_means_by_label`). This is the single most important SHAP feature of the ensemble (mean |SHAP| 0.156, nearly twice the next feature). See Section 7.
- `brand_name_presence` and `suspicious_keywords` are substring matches (e.g. "apple" matches "pineapple", "account" matches "accountant").
- `url_shortening_service`, `suspicious_tld`: small hard-coded lists (14 and 20 entries).
- `random_looking_domain`: hand-set thresholds (length ≥ 12, ≥ 3 digits, entropy ≥ 3.5) with no justification.
- **Scheme-less URLs break every host feature (verified 2026-10-11).** `urlsplit("visitcanada.com/")` returns an empty hostname, so the original extractor sets `domain_length = 0`, `num_subdomains = 0`, `tld_type = "ip"` and treats the whole string as the path. Input `paypal.com.secure-login.xyz/webscr` gets the same host features as any other scheme-less URL. Many public corpora (Kaggle/HuggingFace lineage: 99.8 % scheme-less) store URLs this way, and users often paste URLs without a scheme, so the deployed app silently degrades on such input. Re-running the unmodified pipeline on the scheme-less corpus B produced only 6 TLD columns and 84.8 % test accuracy (`results/reproduction/lineage253k/`). Regression test: `tests/test_feature_extractor.py::test_original_extractor_scheme_less_bug`.
- No handling of: ports, userinfo, fragments, IDN decoding (only literal `xn--`), percent-decoding before analysis, path depth, longest token, token statistics, registered-domain extraction, digit runs in hostname, or character n-grams.
- `urlsplit` on malformed input (e.g. `http://[::1`) raises `ValueError`; there is no guard, so `app.py` returns HTTP 400 with the exception text.

4. Models and training configuration

Model	Config	Preprocessing
Logistic Regression	`max_iter=1000, class_weight='balanced', seed 42`	median imputer + StandardScaler
Random Forest	`n_estimators=200, class_weight='balanced', seed 42`	median imputer
XGBoost	`n_estimators=200, max_depth=6, lr=0.1, subsample=0.8, colsample_bytree=0.8`	median imputer
Weighted soft voting	LR:RF:XGB = 1:2:2	each member's own pipeline

- No hyperparameter search; no validation-based model selection; the validation split is used only to print metrics.
- The ensemble weights (1,2,2) are hard-coded and never justified or tuned.
- Decision threshold is the default 0.5 everywhere.
- The imputer is a no-op (no missing values exist), so it is harmless but misleading.
- The ensemble is trained in NB06 on fresh pipelines; the standalone LR/RF/XGB evaluated in NB07 were trained in NB05 with identical settings, so the comparison is fair but duplicates the work.

5. Stored results (origin: notebook outputs, test split of seed-42 random split)

Test set n = 50,620 (25,884 legitimate / 24,736 phishing).

Model	Acc	Prec	Rec	F1	FP	FN
Logistic Regression	0.8864	0.9030	0.8598	0.8809	2,285	3,467
Random Forest	0.9679	0.9771	0.9568	0.9668	555	1,068
XGBoost	0.9671	0.9828	0.9492	0.9657	410	1,257
Weighted soft voting	0.9678	0.9824	0.9512	0.9665	422	1,207

Findings about these numbers:

- The ensemble does **not** outperform Random Forest on accuracy, recall or F1; the differences between RF, XGB and the ensemble are ≤ 0.001 in F1 and have no uncertainty estimates. The "hybrid" contribution of the original project is therefore not supported by its own results.
- ROC-AUC / PR-AUC values exist only inside the figure legends of `roc_curves.png` / `pr_curves.png`; they were never saved as data.
- All numbers come from a single random split with a single seed.

6. Reproducibility status

Item	Status
Raw dataset `data/raw/raw_data.csv`	**Not in repository, not documented.** Origin, licence, collection period unknown. Needed to reproduce anything.
Trained models `models/*.joblib`	Not in repository (gitignored).
Split files `data/processed/splits/*.csv`	Not in repository, but the split is deterministic (seed 42, stratified) given the feature file.
Environment	No `requirements.txt`; versions only inferable from notebook pip logs.
Paths	Absolute Windows paths in NB outputs; notebooks use `Path.cwd().parent`, which works only when the kernel is started inside `notebooks/`.
Tests	None (four empty files).
Random seeds	`random_state=42` for split and models; `np.random.choice` in NB09 unseeded; KernelExplainer sampling unseeded.
Can the stored metrics be reproduced today?	**Not without the raw CSV.** The *code path* was re-executed unchanged on two substitute corpora (`scripts/reproduce_original_pipeline.py`): it runs end to end (PhiUSIIL: soft vote test accuracy 0.9970, ROC-AUC 0.9988; corpus B: 0.8484 / 0.9317, degraded by the scheme-less parsing bug in §3).

Conclusion: at present nothing in `results/` can be independently regenerated from the repository alone. The first blocker for Phase 1.2 is obtaining `raw_data.csv` (or an equivalent documented dataset).

7. Evaluation and leakage concerns

11. **Source/collection artifacts dominate the model.** From the author's own feature means by label: `https_usage` 0.93 vs 0.50; `num_subdomains` 1.20 vs 0.80 (i.e. legitimate URLs mostly carry `www.`);

`excessive_hyphens` higher for legitimate; `ip_address` is exactly 0.0 for legitimate. SHAP ranks `https_usage` (0.156) and `num_subdomains` (0.091) far above all other features. These are properties of *how the two classes were collected* (e.g. a curated benign list normalised to `https://www.` vs. a phishing feed) rather than properties of maliciousness. The saved `results/url_predictions.csv` confirms the consequence: `https://google.com/` is predicted MALICIOUS with 88% confidence, `https://www.goggle.com` (a typosquat) LEGITIMATE with 82%, and `https://www.wikipedia.org` only 62% legitimate. The model has learned "no www. → phishing".

12. **Random split with URL-level deduplication only.** Many URLs share a registered domain (e.g. many `docs.google.com` phishing pages, many pages from the same university site). A random split places pages of the same site in train and test, so the test score partly measures memorisation of domains. Domain overlap was never quantified.
13. **No temporal information** exists in the dataset, so temporal generalisation cannot be assessed with it.
14. **One-hot TLD over 521 values** fitted on the full dataset before splitting (`pd.get_dummies` on `df_features` in NB05 before `create_data_splits`). This is a mild leak of the TLD vocabulary (test-only TLDs get columns); it does not leak labels, but it is not a train-only fit.
15. **Threshold and ensemble weights** were never selected on validation data; the validation split is effectively unused.
16. **Single seed, no confidence intervals, no statistical tests.**
17. **No external dataset** evaluation.

8. Consistency between training, prediction and app

- Feature code is consistent (root file, `src` file and NB03/NB09 inline copies are identical), but it exists in four copies, which will diverge.
- Prediction reconstructs the 548-column schema by reading `X_test.csv` at inference time (NB09) — the schema is not persisted with the model. `app.py` does not do even this: it passes a 28-column frame (with a string `tld_type` column) to a model trained on 548 numeric columns, which will raise inside the imputer. **`app.py` cannot work as committed.**
- `app.py` reads `index.html`, but the file is `idex.html`.
- The app presents "Legitimate" with a confidence percentage derived from an uncalibrated soft-vote probability; no uncertainty or limitation statement is shown.

9. Code quality summary

Strengths: the notebooks are clean and linear; the feature list is explicit and documented in `results/logs/feature_definitions.csv`; pipelines wrap preprocessing so no scaler leakage occurs inside a split; the ensemble is built from reusable factories.

Weaknesses (prioritised):

18. Dataset not available / undocumented (blocks everything).
19. Shortcut features (`https_usage`, `www`-driven `num_subdomains`) dominate and produce absurd predictions on well-known domains.
20. No grouped or external evaluation; no uncertainty.
21. No tests; empty modules; duplicated feature code; broken app.
22. No ROC/PR-AUC, FPR/FNR, calibration, latency, or model-size measurements saved as data.
23. No hyperparameter tuning, no validation-based threshold or weight selection.
24. No requirements/README/CLI; absolute paths.

10. Prioritised improvement plan

Priority	Action	Phase
P0	Obtain/document the raw dataset; otherwise adopt documented public datasets (see	1, 4

Priority	Action	Phase
	`research/DATASET_SOURCES.md`).	
P0	Re-implement the pipeline as importable, tested modules with a config file, deterministic splits saved to disk, and a CLI (`scripts/`).	5
P0	Quantify registered-domain overlap between splits; implement domain-grouped splitting with `tldextract`.	4
P0	Reproduce the baseline (28 features, LR/RF/XGB/soft vote, seed 42 random split) with the new code and save predictions, probabilities, AUCs.	1
P1	Artifact analysis: measure how much performance depends on `https_usage`/`www` and how the models behave on random vs grouped vs external splits; ablate artifact features.	6
P1	Improved lexical feature set (public-suffix-aware, token statistics, encoding/IDN handling) and a char-level model; fair comparison.	5, 6
P1	Multi-seed runs, bootstrap CIs, paired tests.	6
P2	Calibration (reliability diagrams, ECE), threshold selection at fixed FPR on validation, latency/model size, SHAP with TreeExplainer, error analysis, controlled mutation robustness.	6, 7
P2	Fix app: load persisted schema, present estimates with uncertainty, no "safe" claim.	5
P3	Manuscript, README, requirements, reproducibility checks.	8, 9

Appendix E. Dataset Audit

Date: 2026-10-11. Scripts: `scripts/prepare_external.py`, `scripts/build_canonical.py`, `scripts/analysis_dataset_artifacts.py`. Machine-readable outputs: `results/logs/dataset_report_<name>.json`, `results/tables/dataset_artifact_stats.csv`, `results/figures/dataset_artifacts.png`. Provenance of every file: `research/DATASET_SOURCES.md` and `data/external/<name>/SOURCE.md`.

1. The repository's own dataset (data/raw/raw_data.csv)

Item	Finding
Availability	**Not in the repository** (data directory was intended to be gitignored); not supplied with the project.
Known facts (from notebook outputs)	253,098 rows; `url,label`; legitimate 129,420 / phishing 123,678 (51.1 / 48.9 %); 0 duplicate URLs; 0 conflicting labels; https 72.2 %, http 27.8 %, ftp 8 URLs.
Origin	No published dataset with these counts exists on Kaggle, HuggingFace, Mendeley, Zenodo, UCI or IEEE DataPort (searched by counts, file name and first-row URLs). Four of the five first rows exist, *without scheme*, in the Kaggle `sid321axn` / `taruntiwarihp` lineage (ISCX-URL-2016 + PhishTank + PhishStorm) as re-uploaded on HuggingFace; the fifth (a 2020s Google-Docs phishing URL) exists in none of the 2014–2019 sets. Conclusion: a derived compilation — scheme-normalised, `www.`-prefixed URLs from the Kaggle lineage merged with a newer phishing feed. Collection dates unknown. (Details and confidence levels in `DATASET_SOURCES.md` §1.)
Label semantics	`phishing` vs `legitimate`; the project calls positives "malicious". Phishing is the only malicious category evidenced.
Class-conditional artifacts (from the author's own `feature_means_by_label.csv`)	https: 93.2 % (legit) vs 50.2 % (phish); mean subdomain count 1.20 vs 0.80 (i.e. `www.` far more common in legit); IP hosts 0.0 % vs 1.8 %; `excessive_hyphens` 14 % vs 3 % (legit higher — a sign of long crawled benign paths).
Timestamps	None. Temporal evaluation impossible on this file.
Action	The authors should supply `raw_data.csv` (and the originating link). Until then `phiusiil` is used as the main stand-in and `lineage253k` as a second training corpus with identical size and class balance. The commented-out `main` entry in `configs/datasets.yaml` re-enables the original the moment it is available.

2. Canonical corpora built for this study

All corpora pass the same pipeline (§2 of `EXPERIMENT_PROTOCOL.md`). `kept` = rows after dropping missing/unmappable labels, normalisation-based deduplication and removal of conflicting-label URLs. `domains` = distinct registered domains (public-suffix aware) or IP literals.

Corpus	Role	Raw rows	Kept	Malicious	Benign	Domains	Conflicts removed	Timestamps
`phiusiil` (UCI 967, CC BY 4.0)	**Corpus A** (main)	235,795	234,687	99,838	134,849	175,508	1	no
`lineage253k` (HF Mitake lineage, CC BY-SA 4.0; sample sized 129,420 / 123,678 like the original)	**Corpus B**	253,098	252,394	123,095	129,299	130,453	1	no
`hannousse2021` (Mendeley, CC BY 4.0)	external	11,430	11,416	5,701	5,715	6,634	0	no (May 2020)
`ebbu2017` (GitHub ebubekirbbr/pdd)	external	73,575	54,721	24,811	29,910	21,606	0	no (2017)
`phishstorm` (mirror)	external	96,018	95,962	47,972	47,990	30,894	0	no (2014)
`iscx2016` (mirror: benign + phishing lists)	external	45,343	45,324	9,946	35,378	4,304	0	no (2016)
`kaggle651k_bp` (sid321axn mirror, benign + phishing only)	external, **shares lineage with B and the original**	522,214	514,031	90,015	424,016	145,482	3,940	no
`phishtank` (verified online, 2026-10-11 dump)	positive-only, temporal	67,621	67,326	67,326	0	12,971	0	**2011-02 → 2026-10**
`urlhaus` (abuse.ch, malware URLs)	positive-only, out-of-task check	72,849	72,847	72,847	0	17,585	0	**2018-06 → 2026-10**
`openphish` (community feed, 2026-10-11)	positive-only sanity	300	300	300	0	144	0	no

Note on PhiUSIIL labels: the UCI file encodes `label = 1` for **legitimate**; the canonical file inverts this so that 1 = phishing (checked: 100,945 raw phishing rows → 99,838 after dedup).

Ebbu2017 lost 26 % of rows to deduplication (12,322 duplicate phishing URLs in the upstream file). kaggle651k_bp had 3,940 normalised URLs carrying both labels; all were removed.

3. Collection artifacts, per class (from dataset_artifact_stats.csv)

Corpus	Class	https	leading `www.`	homepage (empty path, no query)	IP host	median length	domains / URL
phiusiil	benign	**100.0 %**	**100.0 %**	**100.0 %**	0.0 %	27	0.98
phiusiil	phishing	48.4 %	42.0 %	71.8 %	0.6 %	34	0.44
lineage253k	benign	0 % (no scheme stored)	8.2 %	34.5 %	0.0 %	33	0.46
lineage253k	phishing	0 %	23.4 %	38.0 %	2.1 %	33	0.58
hannousse2021	benign	44.5 %	67.0 %	38.4 %	0.0 %	41	0.70
hannousse2021	phishing	33.3 %	20.9 %	18.4 %	1.7 %	55	0.47
ebbu2017	benign	36.1 %	59.1 %	4.7 %	0.0 %	55	0.43
ebbu2017	phishing	2.0 %	26.2 %	2.0 %	0.6 %	58	0.36
phishstorm	benign	2.2 %	70.1 %	0.0 %	0.0 %	44	0.36
phishstorm	phishing	1.4 %	34.6 %	0.0 %	0.6 %	62	0.29
iscx2016	benign	5.5 %	**0.4 %**	0.0 %	0.0 %	101	**0.009**
iscx2016	phishing	1.7 %	42.3 %	3.4 %	1.5 %	68	0.40
kaggle651k_bp	benign	0.5 %	3.0 %	11.2 %	0.0 %	46	0.25
kaggle651k_bp	phishing	7.6 %	40.1 %	25.9 %	0.4 %	34	0.49
phishtank (2011–2026)	phishing	91.9 %	4.2 %	34.3 %	0.1 %	39	0.19
urlhaus	malware	37.9 %	0.5 %	0.3 %	58.7 %	35	0.24

Single-feature ROC-AUC of the artifacts (orientation: AUC < 0.5 means the feature is *more common in benign* URLs):

Corpus	https	`www.`	homepage	length	IP host
phiusiil	0.242	0.210	0.359	0.720	0.503
lineage253k	0.500	0.576	0.517	0.538	0.511
hannousse2021	0.444	0.269	0.400	0.665	0.509
ebbu2017	0.329	0.335	0.486	0.547	0.503
phishstorm	0.496	0.323	0.500	0.730	0.503
iscx2016	0.481	**0.710**	0.517	**0.244**	0.508
kaggle651k_bp	0.536	**0.685**	0.574	0.355	0.502

Interpretation

25. **The direction of the `www.` and length artifacts flips between corpora.** In PhiUSIIL, Hannousse, Ebbu2017 and PhishStorm, `www.` indicates *benign* (AUC 0.21–0.34); in ISCX-2016 and the Kaggle lineage it indicates *phishing* (AUC 0.69–0.71). Long URLs are phishing in PhiUSIIL/PhishStorm (AUC 0.72–0.73) but benign in ISCX-2016 (0.24), whose benign list consists of deep crawled paths from only ~300 sites (0.009 domains per URL). A classifier that learns either sign will fail on the corpora of the opposite sign — this is the mechanism behind cross-dataset collapse reported in the literature and is testable here (RQ1/RQ2).
26. **PhiUSIIL's benign class is a degenerate population**: every benign URL is `https://www.<domain>` with no path. Any model trained on it learns "path present ⇒ suspicious" and "no `www.` ⇒ suspicious". The project's original dataset, built with the same `https://www.` normalisation of benign homepages, shares this property (93 % https, `www.` prevalent), which explains the audit's `https://google.com/ → malicious` failure.
27. **ISCX-2016 benign URLs come from ~300 domains** (35,378 URLs, 4,304 registered domains overall and only 0.4 % `www.`), so random splits leak domains massively; grouped splitting is mandatory there.
28. **The present-day phishing population (PhishTank 2026) is 92 % https with 4 % `www.`**, i.e. it looks like PhiUSIIL's *benign* class on the scheme feature and unlike PhiUSIIL's phishing class. A

model that relies on `https` as a benign indicator should lose recall on current phishing; this is measured by scoring `phishtank` by submission month (temporal recall).

29. `kaggle651k_bp` and `lineage253k` store URLs without scheme, so scheme features are constant there; models trained on scheme-bearing corpora see an out-of-vocabulary condition on these sets (`had_explicit_scheme = 0`), which is itself a realistic deployment variant (URLs copied from logs or e-mail text without scheme).

4. Leakage and quality checks performed

Check	Result
Exact / normalised duplicates across partitions	Impossible by construction (dedup before splitting; asserted in tests).
Conflicting labels	Removed (counts above).
Registered-domain overlap between train and test	Quantified for every split in `results/runs/<run>/split_<seed>.json` → `meta.overlap`; zero for grouped splits by construction.
Unavailable / dead URLs	Not checked — this is an offline lexical study; liveness is irrelevant to the label definition used by the sources.
Mislabeled records	Not individually audited; known-noisy sources (Kaggle lineage, PhishStorm mirror) are used only as external tests, never as the primary test for claims.
Source leakage inside a corpus	Yes — documented above (per-class scheme/`www`/path artifacts). Addressed by artifact-ablation runs and external tests rather than by filtering, because filtering would hide the phenomenon under study.
Near-duplicates / URL variants	Grouped splits remove the dominant variant family (same registered domain). Template-level near-duplicates across domains (phishing kits) remain and are a known limitation.
Timestamps	Only PhishTank and URLhaus; both positive-only. Their timestamps are submission/report times, which are trustworthy for ordering.

5. Consequences for the experimental design

- Primary claims are made on **grouped** splits of Corpus A and B, with **external** evaluation on Hannousse 2021 (independent, balanced, 2020), Ebbu2017, PhishStorm and ISCX-2016, and **positive-only temporal recall** on PhishTank by month.
- `kaggle651k_bp` is reported but flagged as lineage-overlapping with Corpus B and (probably) the original data.
- Precision at realistic prevalence is derived analytically from TPR/FPR at the chosen thresholds (base rates 1 %, 0.1 %) rather than by subsampling, following the literature's base-rate guidance.

Appendix F. Experiment Protocol

> ***Change record.*** *v1 was written before any experiment. v1.1 adds the* `lexical_v2_canon` *and* `charcnn_canon` *representations and the leave-one-corpus-out design. These were added after (a) the dataset-identification probe and (b) inspecting the external results of the first baseline run (*`A_b28_random`*). They are therefore post hoc additions and are reported as such. No hyper-parameter, threshold or feature choice was tuned on test or external data; all models keep the fixed configurations below.*

Any change to this protocol is logged in `DECISIONS.md` with the reason and date.

1. Task and label definition

Binary classification. Positive class = malicious/phishing URL (label 1); negative = benign/legitimate (label 0). All security metrics (recall, FNR, FPR, precision, TPR@FPR) use this encoding. Datasets with finer labels (e.g. malware/defacement) are mapped to positive and this is recorded in the dataset report.

2. Data preparation (identical for every dataset)

30. Load raw CSV; drop rows with missing URL or label; strip whitespace. Record SHA-256 of the raw file.
31. Map labels; drop unmappable rows (count logged).
32. Normalisation-based duplicate detection (lowercase scheme and host, drop fragment, drop trailing slash on empty path, drop default ports). Keep first occurrence. **Remove** every normalised URL carrying conflicting labels.
33. Compute the grouping key = registered domain (public suffix list snapshot bundled in tldextract) or the IP literal.
34. Write `data/processed/canonical/<name>.parquet` and `results/logs/dataset_report_<name>.json` (row counts at every step, class balance, scheme/`www`/IP shares, timestamp range).

Raw features are extracted from the *original* URL string, not the normalised one.

3. Feature sets

Name	Columns	Purpose
`baseline28`	28 features of the original repository, code imported unchanged; `tld_type` one-hot encoded with the encoder fitted on the training partition only (`min_frequency=20`, infrequent bucket)	Faithful reproduction
`baseline28 – artifacts`	`baseline28` without `https_usage`, `num_subdomains`	Artifact ablation (RQ1)
`lexical_v2`	91 numeric offline features (public-suffix aware, encoding/IDN aware, token statistics, brand/keyword placement)	Improved handcrafted set
`lexical_v2 – artifacts`	without `https_scheme`, `had_explicit_scheme`, `has_www_prefix`, `host_num_labels`, `num_subdomain_labels_psl`	Artifact ablation
`lexical_v2_canon`	86 features: the URL string is first canonicalised (scheme and a leading `www.`/`wwwN.` label removed), then `lexical_v2` is computed and the five artifact columns are dropped. Invariant to scheme and `www.` by construction (unit-tested).	Artifact-neutral representation (added after the probe showed column dropping leaks scheme through counts; see DECISIONS.md)
`char` / `char canon`	raw characters (≤ 160, ASCII vocabulary), optionally canonicalised	Learned representation
`char + lexical_v2`	both	Hybrid

No feature uses network access.

4. Splitting strategies

Strategy	Definition	Used for
`random`	Stratified 60/20/20 (train/val/test), seed s	Reproduction; upper bound with domain leakage
`grouped`	StratifiedGroupKFold (10 folds, shuffle, seed s) on registered domain; folds 0–1 → test, 2–3 → val, rest → train. No	Main protocol

Strategy	Definition	Used for
	registered domain in two partitions.	
`temporal`	Sort by timestamp; oldest 60 % train, next 20 % val, newest 20 % test	Only where trustworthy timestamps exist (URLhaus-based set)
`external`	Train on dataset A (grouped split), evaluate on the entire dataset B	Cross-collection generalisation
`LOCO`	Leave-one-corpus-out: pool the other five labelled corpora (each capped at 50k rows, stratified), grouped split of the pool for train/val, evaluate on the whole held-out corpus	Generalisation to an unseen collection when training data are diverse

For every split we store the index arrays, the class balance per partition, and the domain-overlap report (`share_test_rows_with_group_in_train`).

5. Models and hyper-parameters

Fixed configurations (no test-set tuning). Hyper-parameters of LR/RF/XGB are the original repository's values so that reproduction is faithful; LightGBM, stacking and the char-CNN use conventional defaults listed in `src/urlresearch/models/`. Any hyper-parameter search (if performed) uses 5-fold *grouped* CV inside the training partition only and is reported separately.

Key	Model
`lr`	Logistic regression, standardised inputs, balanced class weights, C = 1
`rf`	Random forest, 200 trees, balanced class weights
`xgb`	XGBoost, 200 rounds, depth 6, η = 0.1, subsample/colsample 0.8
`lgbm`	LightGBM, 400 rounds, 63 leaves, η = 0.05
`softvote`	Original hybrid: soft vote LR:RF:XGB with weights 1:2:2
`stack`	Stacking of LR/RF/XGB with logistic meta-learner (5-fold internal CV)
`charcnn`	Char-CNN: emb 32, 3 parallel convs (k = 3,5,7; 128 filters), global max-pool, dense 128, dropout 0.3, Adam 1e-3, batch 256, ≤ 5 epochs or 20 min, early stopping on validation AUC (patience 2), class-balanced BCE; 115,329 parameters
`charcnn_canon`	Same model on canonicalised strings
`charcnn_hybrid`	Char-CNN with standardised `lexical_v2` features concatenated before the dense head

6. Threshold policies (chosen on validation only)

`t0.5` (default), `val_maxf1` (maximises F1 on validation), `val_fpr0.01` and `val_fpr0.001` (largest-recall threshold whose validation FPR ≤ 1 % / 0.1 %). The same threshold is then applied unchanged to test and external sets.

7. Metrics

Per model × seed × evaluation set × policy: accuracy, precision, recall, specificity, F1, FPR, FNR, balanced accuracy, MCC, ROC-AUC, PR-AUC (average precision), TPR@FPR = 1 % and 0.1 %, log-loss, Brier score, ECE (15 bins), confusion counts. Resources: fit time, per-URL prediction latency, pickled model size, parameter count.

8. Repetition and statistics

- Random and grouped protocols: 5 seeds (42–46) → mean ± sd. LOCO: 3 seeds. Char models: 3 seeds on corpus A, 1 seed on corpus B (CPU budget).
- Within a seed, paired comparisons on identical test examples: paired bootstrap (1 000 resamples) of ΔAUC / ΔPR-AUC / ΔF1 with 95 % percentile CI and a two-sided bootstrap p-value; McNemar's exact test on paired correctness at the policy threshold.
- Point estimates carry 95 % percentile-bootstrap CIs (1 000 resamples).
- A difference is called "significant" only when the paired 95 % CI excludes 0 **and** the effect is consistent in sign across seeds.

9. Leakage controls checklist

- [ ] No normalised URL appears in two partitions (asserted in `load_dataset` + split tests).
- [ ] Grouped split: zero registered-domain overlap (asserted by `overlap_report`).

- [ ] Encoders/scalers are inside sklearn Pipelines → fitted on training rows only.
- [ ] Thresholds selected on validation; test used once per configuration.
- [ ] Char-CNN early stopping monitors validation, never test.
- [ ] External datasets are never used for any fitting or selection.
- [ ] Feature lists are fixed before experiments (this document).

10. Robustness and diagnostics (P2)

- Mutation tests (`evaluation/mutations.py`): Tier A RFC-equivalent transformations (label preserved by construction); Tier B presentation variants (`www` toggle, scheme toggle, tracking parameter) with the stated assumption that intent is unchanged. Report flip rate, mean |Δp|, AUC/recall/FPR before vs after.
- SHAP TreeExplainer on XGBoost (3 000 test URLs) for each feature set / split.
- Reliability diagrams and ECE; optional post-hoc calibration (isotonic/Platt) fitted on validation.
- Error analysis: FPs and FNs stratified by registered-domain novelty, length, IP host, `www`/scheme, encoding, and free-hosting suffix.

Appendix G. Complete Result Tables

Generated by scripts/make_paper_tables.py from results/runs/*. Mean ± standard deviation over seeds. Rows marked pending correspond to experiments that had not finished when this document was generated.

TABLE T1. Corpora after cleaning.

corpus	raw	kept	malicious	benign	reg_domains	https	www	timestamps
ebbu2017	73575	54721	24811	29910	21606	0.207	0.442	no
hannousse2021	11430	11416	5701	5715	6634	0.389	0.44	no
iscx2016	45343	45324	9946	35378	4304	0.047	0.096	no
kaggle651k_bp	522214	514031	90015	424016	145482	0.017	0.095	no
lineage253k	253098	252394	123095	129299	130453	0	0.156	no
openphish	300	300	300	0	144	0.713	0.19	no
phishstorm	96018	95962	47972	47990	30894	0.018	0.523	no
phishtank	67621	67326	67326	0	12971	0.919	0.042	2011-02→2026-10
phiusiil	235795	234687	99838	134849	175508	0.781	0.753	no
urlhaus	72849	72847	72847	0	17585	0.379	0.005	2018-06→2026-10

TABLE T2. Original pipeline re-execution vs stored original results.

data	model	accuracy	precision	recall	f1	roc_auc
original raw_data.csv (stored result, not re-run)	Logistic Regression	0.8864	0.903	0.8598	0.8809	not stored
original raw_data.csv (stored result, not re-run)	Random Forest	0.9679	0.9771	0.9568	0.9668	not stored
original raw_data.csv (stored result, not re-run)	XGBoost	0.9671	0.9828	0.9492	0.9657	not stored
original raw_data.csv (stored result, not re-run)	Weighted Soft-Voting Ensemble	0.9678	0.9824	0.9512	0.9665	not stored
phiusiil (original code re-run)	Logistic Regression	0.9952	0.9985	0.9903	0.9944	0.9983
phiusiil (original code re-run)	Random Forest	0.9968	0.9984	0.994	0.9962	0.9984
phiusiil (original code re-run)	XGBoost	0.9969	0.9992	0.9936	0.9964	0.9986
phiusiil (original code re-run)	Weighted Soft-Voting Ensemble	0.997	0.9994	0.9935	0.9965	0.9988
lineage253k (original code re-run)	Logistic Regression	0.7025	0.7403	0.6009	0.6633	0.805
lineage253k (original code re-run)	Random Forest	0.8455	0.8465	0.8346	0.8405	0.9239
lineage253k (original code re-run)	XGBoost	0.8406	0.8542	0.8117	0.8324	0.9265
lineage253k (original code re-run)	Weighted Soft-Voting Ensemble	0.8484	0.8529	0.8328	0.8427	0.9317

TABLE T3. In-distribution test performance: random vs grouped splits.

corpus	features	split	model	seeds	test_domain_overlap	roc_auc	f1@0.5	tpr@valFPR1%	realised_fpr	status
phiusiil	b28	random	lr	5	0.264 ± 0.002	0.9984 ± 0.0003	0.9946 ± 0.0005	0.9927 ± 0.0009	0.0101 ± 0.0010	complete
phiusiil	b28	random	rf	5	0.264 ± 0.002	0.9984 ± 0.0002	0.9962 ± 0.0005	0.9956 ± 0.0007	0.0094 ± 0.0007	complete
phiusiil	b28	random	softvote	5	0.264 ± 0.002	0.9989 ± 0.0002	0.9964 ± 0.0006	0.9957 ± 0.0007	0.0098 ± 0.0010	complete
phiusiil	b28	random	xgb	5	0.264 ± 0.002	0.9987 ± 0.0003	0.9964 ± 0.0006	0.9955 ± 0.0008	0.0098 ± 0.0012	complete
phiusiil	b28	grouped	lr	5	0.000 ± 0.000	0.9981 ± 0.0004	0.9930 ± 0.0025	0.9915 ± 0.0022	0.0099 ± 0.0011	complete
phiusiil	b28	grouped	rf	5	0.000 ± 0.000	0.9982 ± 0.0003	0.9943 ± 0.0029	0.9940 ± 0.0021	0.0089 ± 0.0018	complete
phiusiil	b28	grouped	softvote	5	0.000 ± 0.000	0.9987 ± 0.0002	0.9947 ± 0.0027	0.9947 ± 0.0015	0.0086 ± 0.0021	complete
phiusiil	b28	grouped	xgb	5	0.000 ± 0.000	0.9986 ± 0.0003	0.9947 ± 0.0026	0.9945 ± 0.0013	0.0091 ± 0.0012	complete
phiusiil	v2	random	—	—	—	—	—	—	—	pending
phiusiil	v2	grouped	—	—	—	—	—	—	—	pending
lineage253k	b28	random	—	—	—	—	—	—	—	pending
lineage253k	b28	grouped	—	—	—	—	—	—	—	pending
lineage253k	v2	random	—	—	—	—	—	—	—	pending
lineage253k	v2	grouped	—	—	—	—	—	—	—	pending

TABLE T4. External corpora: ROC-AUC and FPR at the validation-chosen FPR ≤ 1 % threshold.

run	model	seeds	in-dist AUC	hannousse2021 AUC	hannousse2021 FPR	ebbu2017 AUC	ebbu2017 FPR	phishstorm AUC	phishstorm FPR	iscx2016 AUC	iscx2016 FPR	lineage253k AUC	lineage253k FPR	phishtank recall	status
A_b28_grouped	lr	5	0.998 ± 0.000	0.688 ± 0.013	0.994 ± 0.000	0.724 ± 0.025	1.000 ± 0.000	0.703 ± 0.021	1.000 ± 0.000	0.421 ± 0.024	1.000 ± 0.000	0.554 ± 0.004	0.743 ± 0.001	0.992 ± 0.001	complete
A_b28_grouped	rf	5	0.998 ± 0.000	0.557 ± 0.006	0.994 ± 0.000	0.555 ± 0.008	1.000 ± 0.000	0.621 ± 0.005	1.000 ± 0.000	0.474 ± 0.012	1.000 ± 0.000	0.410 ± 0.012	1.000 ± 0.000	0.995 ± 0.000	complete
A_b28_grouped	softvote	5	0.999 ± 0.000	0.667 ± 0.011	0.994 ± 0.000	0.738 ± 0.021	1.000 ± 0.000	0.758 ± 0.013	1.000 ± 0.000	0.425 ± 0.071	1.000 ± 0.000	0.553 ± 0.011	1.000 ± 0.000	0.995 ± 0.000	complete
A_b28_grouped	xgb	5	0.999 ± 0.000	0.679 ± 0.016	0.994 ± 0.000	0.753 ± 0.025	1.000 ± 0.000	0.752 ± 0.020	1.000 ± 0.000	0.429 ± 0.079	1.000 ± 0.000	0.493 ± 0.027	1.000 ± 0.000	0.995 ± 0.000	complete
A_b28noart_grouped	lr	1	0.987	0.662	0.943	0.648	0.997	0.578	1.000	0.504	0.994	0.525	0.472	0.983	complete
A_b28noart_grouped	rf	1	0.997	0.549	0.983	0.534	1.000	0.640	1.000	0.510	1.000	0.406	1.000	0.992	complete
A_b28noart_grouped	softvote	1	0.999	0.637	0.993	0.553	1.000	0.713	1.000	0.185	1.000	0.421	0.865	0.995	complete
A_b28noart_grouped	xgb	1	0.999	0.632	0.994	0.520	1.000	0.664	1.000	0.097	1.000	0.416	0.781	0.995	complete
A_v2_grouped	—	—	—	—	—	—	—	—	—	—	—	—	—	—	pending
A_v2noart_grouped	—	—	—	—	—	—	—	—	—	—	—	—	—	—	pending
A_v2canon_grouped	—	—	—	—	—	—	—	—	—	—	—	—	—	—	pending
A_char_grouped	—	—	—	—	—	—	—	—	—	—	—	—	—	—	pending
B_b28_grouped	—	—	—	—	—	—	—	—	—	—	—	—	—	—	pending
B_b28noart_grouped	—	—	—	—	—	—	—	—	—	—	—	—	—	—	pending
B_v2_grouped	—	—	—	—	—	—	—	—	—	—	—	—	—	—	pending
B_v2noart_grouped	—	—	—	—	—	—	—	—	—	—	—	—	—	—	pending
B_v2canon_grouped	—	—	—	—	—	—	—	—	—	—	—	—	—	—	pending
B_char_grouped	—	—	—	—	—	—	—	—	—	—	—	—	—	—	pending

TABLE T5. Leave-one-corpus-out.

held_out	representation	model	seeds	pool test AUC	held-out AUC	held-out F1@0.5	held-out recall@valFPR1%	held-out FPR@valFPR1%	status
phiusiil	v2	lr	3	0.921 ± 0.012	0.966 ± 0.006	0.750 ± 0.015	0.147 ± 0.122	0.000 ± 0.000	complete
phiusiil	v2	rf	3	0.960 ± 0.001	0.891 ± 0.027	0.762 ± 0.009	0.251 ± 0.054	0.000 ± 0.000	complete
phiusiil	v2	xgb	3	0.968 ± 0.002	0.977 ± 0.017	0.784 ± 0.004	0.422 ± 0.047	0.000 ± 0.000	complete
phiusiil	v2noart	lr	3	0.917 ± 0.016	0.888 ± 0.007	0.766 ± 0.007	0.321 ± 0.031	0.000 ± 0.000	complete
phiusiil	v2noart	rf	3	0.958 ± 0.001	0.866 ± 0.013	0.784 ± 0.010	0.321 ± 0.052	0.000 ± 0.000	complete
phiusiil	v2noart	xgb	3	0.965 ± 0.003	0.878 ± 0.013	0.800 ± 0.001	0.424 ± 0.084	0.000 ± 0.000	complete
phiusiil	v2canon	—	—	—	—	—	—	—	pending
lineage253k	v2	lr	3	0.934 ± 0.025	0.865 ± 0.004	0.660 ± 0.004	0.997 ± 0.001	0.946 ± 0.034	complete
lineage253k	v2	rf	3	0.973 ± 0.012	0.772 ± 0.003	0.755 ± 0.001	0.421 ± 0.060	0.176 ± 0.021	complete
lineage253k	v2	xgb	3	0.980 ± 0.007	0.874 ± 0.004	0.769 ± 0.003	0.709 ± 0.058	0.160 ± 0.091	complete
lineage253k	v2noart	lr	3	0.936 ± 0.009	0.883 ± 0.003	0.710 ± 0.023	0.723 ± 0.126	0.152 ± 0.113	complete
lineage253k	v2noart	rf	3	0.970 ± 0.013	0.780 ± 0.009	0.761 ± 0.003	0.377 ± 0.035	0.149 ± 0.022	complete
lineage253k	v2noart	xgb	3	0.972 ± 0.010	0.849 ± 0.005	0.779 ± 0.005	0.694 ± 0.050	0.188 ± 0.027	complete
lineage253k	v2canon	—	—	—	—	—	—	—	pending
hannousse2021	v2	—	—	—	—	—	—	—	pending
hannousse2021	v2noart	—	—	—	—	—	—	—	pending
hannousse2021	v2canon	—	—	—	—	—	—	—	pending

held_out	representation	model	seeds	pool test AUC	held-out AUC	held-out F1@0.5	held-out recall@valFPR1%	held-out FPR@valFPR1%	status
ebbu2017	v2	—	—	—	—	—	—	—	pending
ebbu2017	v2noart	—	—	—	—	—	—	—	pending
ebbu2017	v2canon	—	—	—	—	—	—	—	pending
phishstorm	v2	—	—	—	—	—	—	—	pending
phishstorm	v2noart	—	—	—	—	—	—	—	pending
phishstorm	v2canon	—	—	—	—	—	—	—	pending
iscx2016	v2	—	—	—	—	—	—	—	pending
iscx2016	v2noart	—	—	—	—	—	—	—	pending
iscx2016	v2canon	—	—	—	—	—	—	—	pending

TABLE T6. Paired comparisons against XGBoost — pending (analysis not yet run).

TABLE T7. Calibration and threshold transfer — pending (analysis not yet run).

TABLE T8. Sensitivity to label-preserving URL variants — pending (analysis not yet run).

TABLE T9. Computational cost.

run_id	model	fit_s	latency_ms	size_mb	n_features
A_b28_grouped	lr	2.1296	0.0042	0.0156	28
A_b28_grouped	rf	34.8751	0.0252	58.7252	28
A_b28_grouped	softvote	7603.071	0.0342	59.0914	28
A_b28_grouped	xgb	12.7386	0.0092	0.3649	28

TABLE T10. Dataset-identification probe.

class	feature_set	drop_artifacts	n	chance	accuracy	phiusiil	lineage253k	hannousse2021	ebbu2017	phishstorm	iscx2016
benign	baseline28	False	45715	0.167	0.803	1	0.999	0.367	0.544	0.841	0.945
malicious	baseline28	False	45701	0.167	0.577	0.872	0.999	0.18	0.506	0.3	0.489
benign	lexical_v2	False	45715	0.167	0.83	1	1	0.402	0.67	0.82	0.967
malicious	lexical_v2	False	45701	0.167	0.606	0.865	0.999	0.277	0.535	0.385	0.481
benign	lexical_v2_canon	False	45715	0.167	0.617	0.981	0.184	0.319	0.553	0.703	0.874
malicious	lexical_v2_canon	False	45701	0.167	0.428	0.748	0.344	0.107	0.505	0.37	0.402
benign	lexical_v2	True	45715	0.167	0.814	1	0.99	0.383	0.661	0.766	0.961
malicious	lexical_v2	True	45701	0.167	0.578	0.839	0.972	0.179	0.544	0.369	0.45

Appendix H. Additional Figures

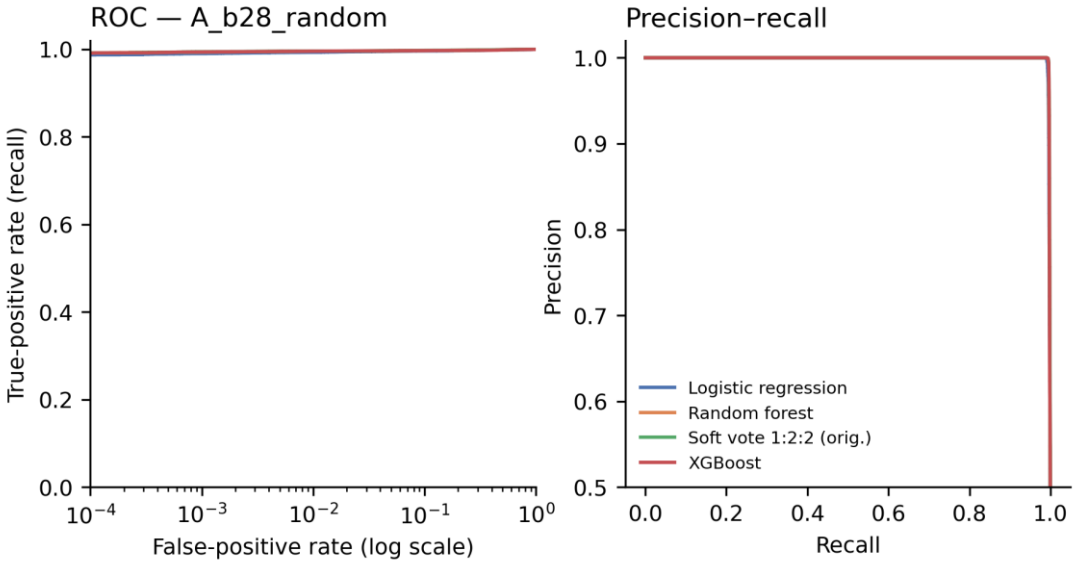


Fig. 8. ROC and precision–recall curves, corpus A, random split, original features (seed 42).

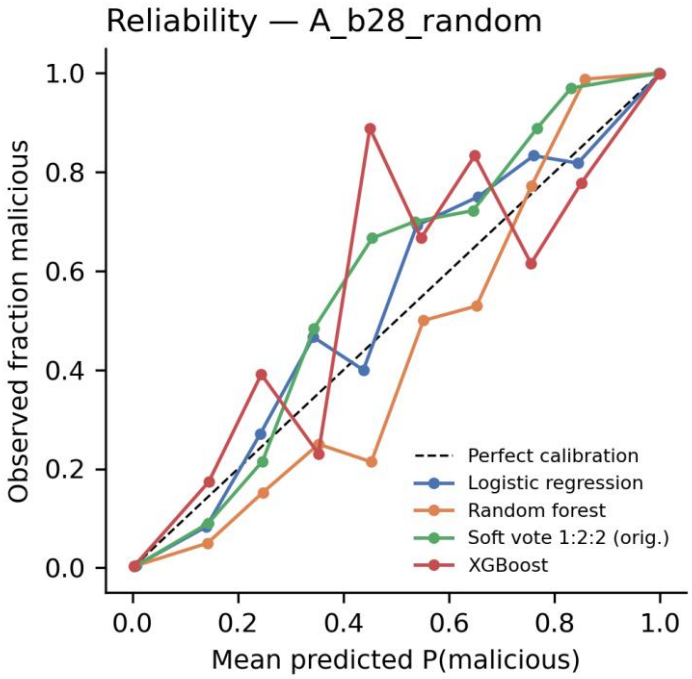


Fig. 9. Reliability diagram, corpus A, random split (seed 42).

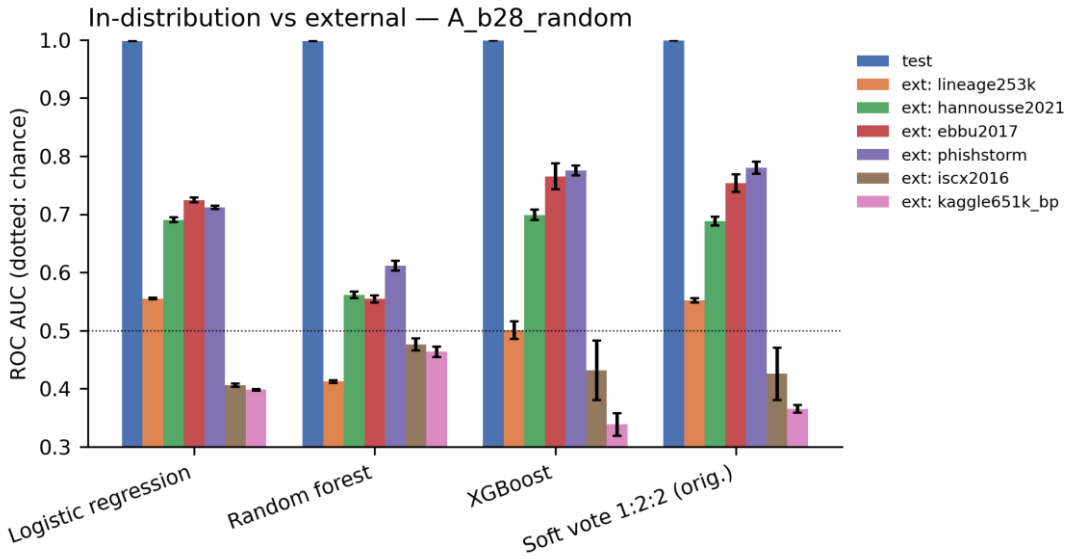


Fig. 10. In-distribution vs external ROC-AUC, corpus A, random split, original features (5 seeds).

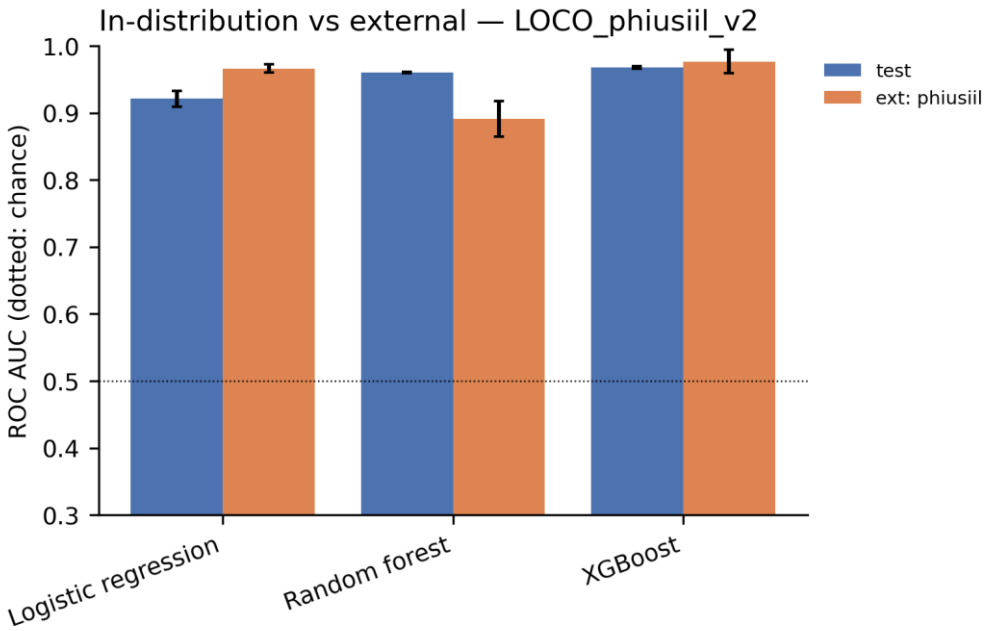


Fig. 11. Leave-one-corpus-out with PhiUSIIL held out (lexical_v2): pool test vs held-out ROC-AUC.

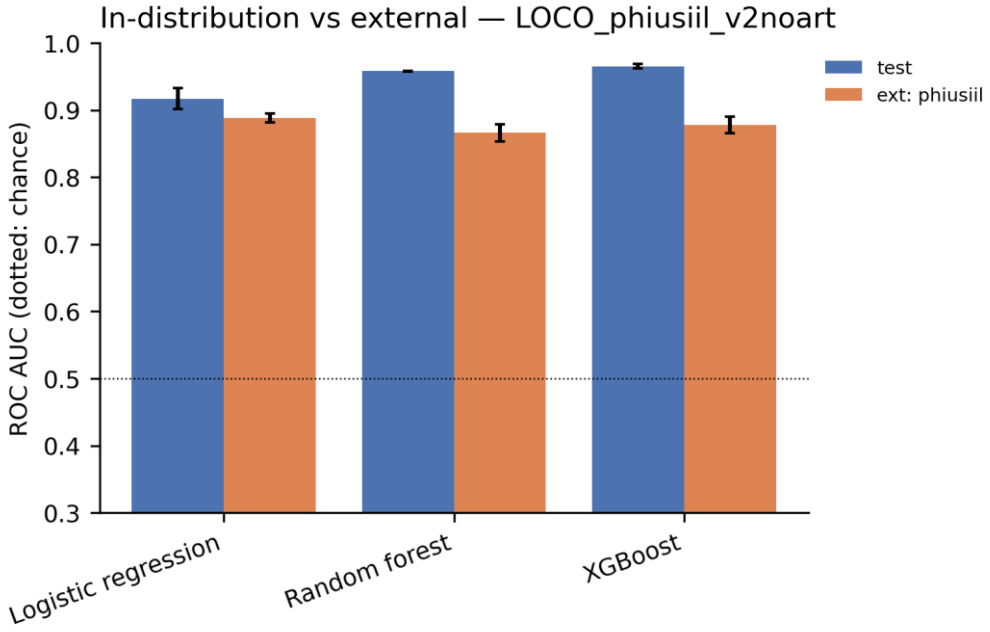


Fig. 12. Same as previous figure without the five artifact columns.

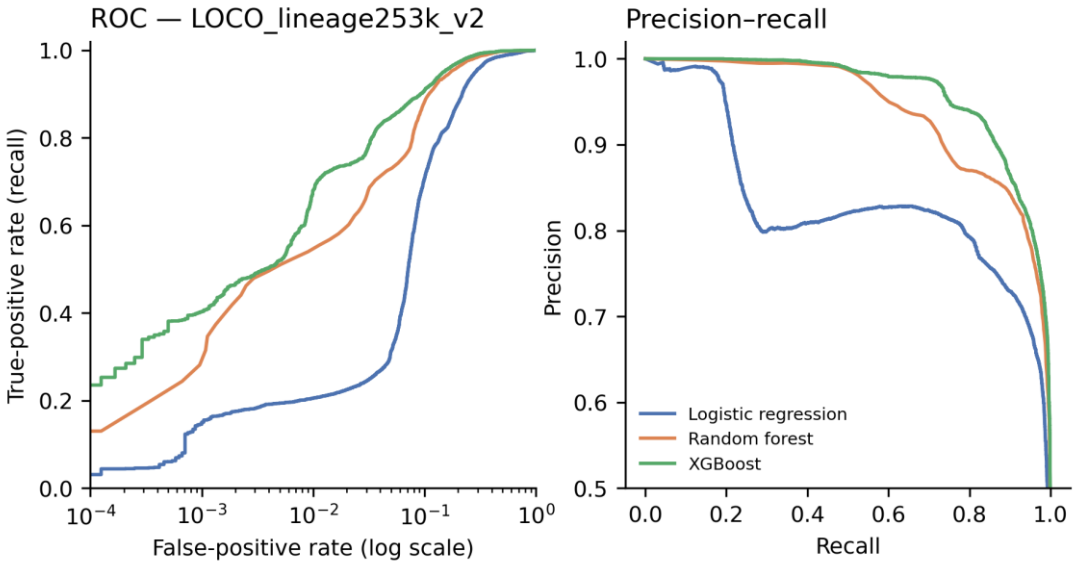


Fig. 13. ROC and precision–recall curves on the pool test split, corpus B held out (seed 42).

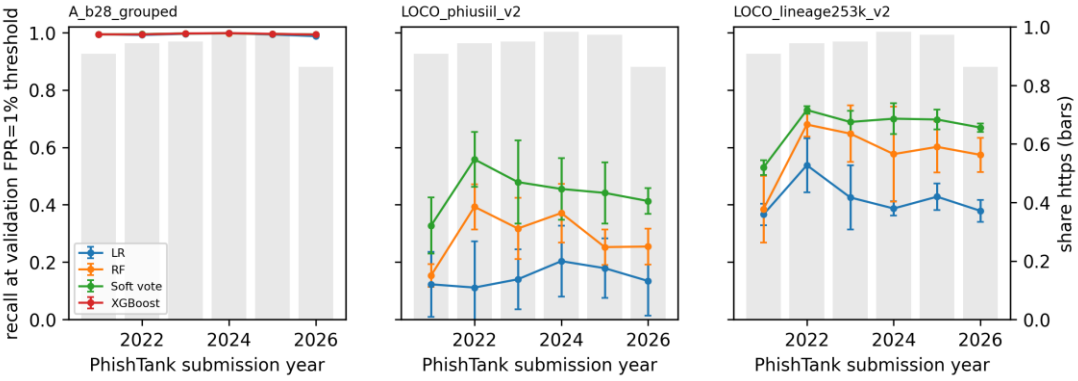


Fig. 14. Preliminary: recall on PhishTank verified phishing URLs by submission year (bars: share of https URLs) at the validation-chosen FPR ≤ 1 % threshold. Years with fewer than 100 URLs omitted; analysis to be repeated on the complete matrix.

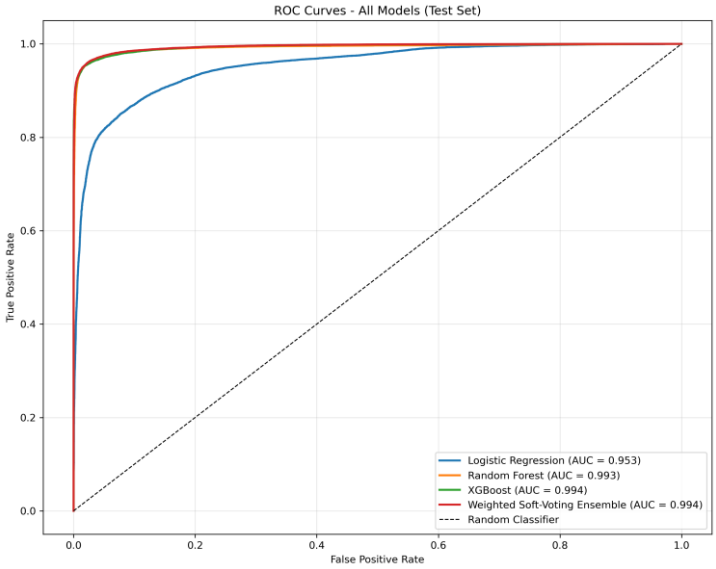


Fig. 15. Original project output (results/figures/roc_curves.png): ROC curves on the original, unavailable dataset; reproduced here unchanged for reference.

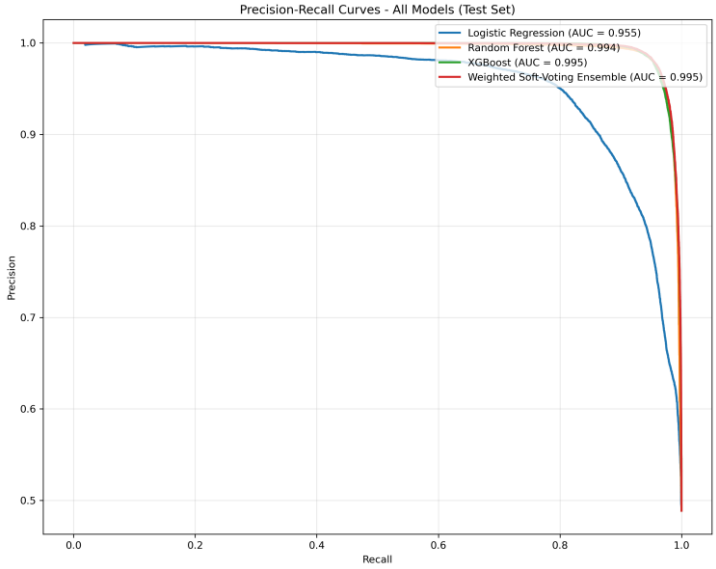


Fig. 16. Original project output: precision–recall curves on the original dataset.

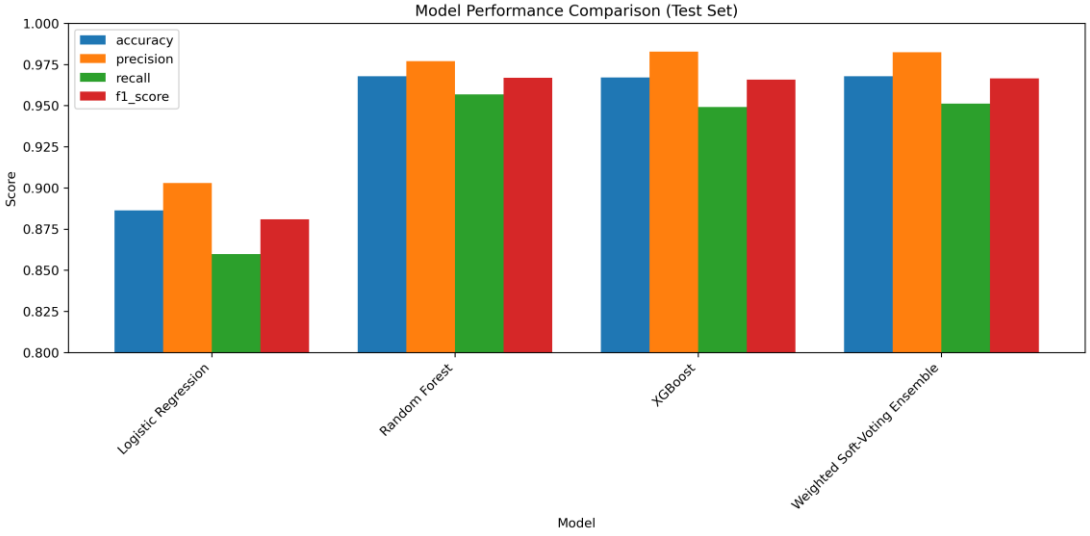


Fig. 17. Original project output: model comparison on the original dataset (test split).

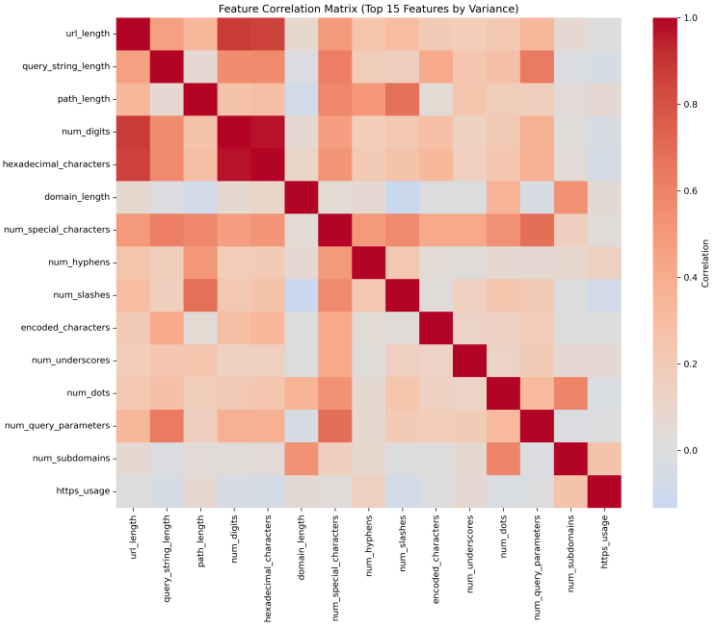


Fig. 18. Original project output: correlation of the 15 highest-variance original features.

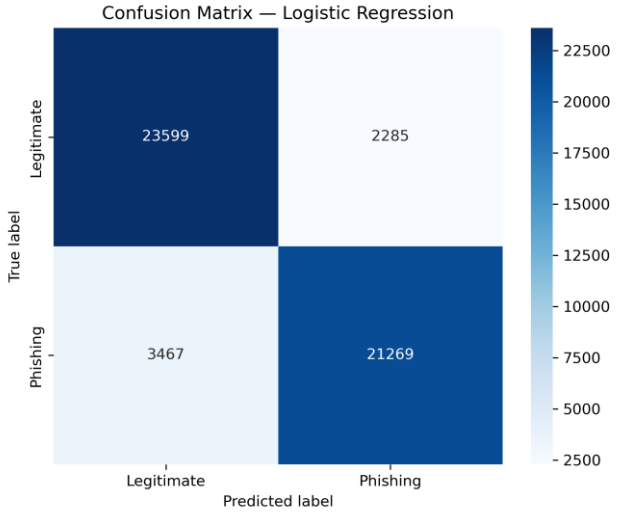


Fig. 19. Original project output: confusion matrix of logistic regression on the original test split.

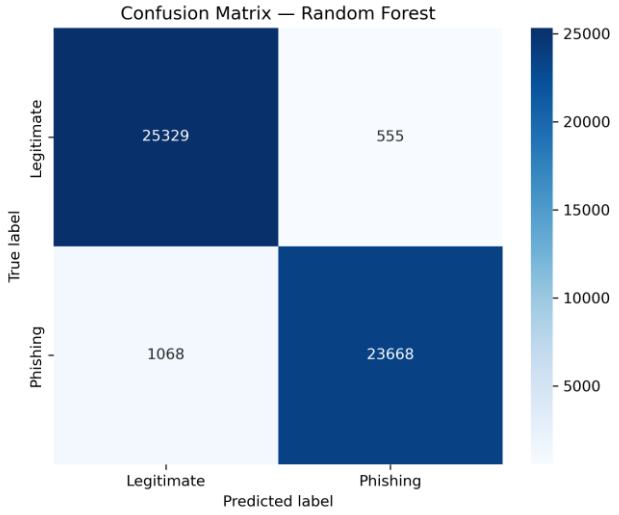


Fig. 20. Original project output: confusion matrix of random forest on the original test split.

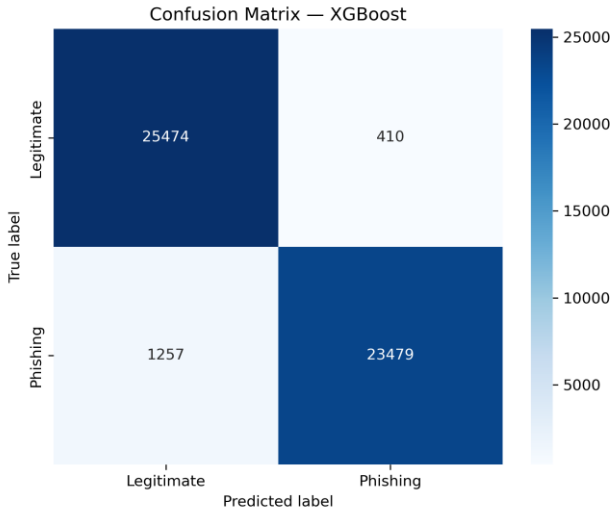


Fig. 21. Original project output: confusion matrix of xgboost on the original test split.

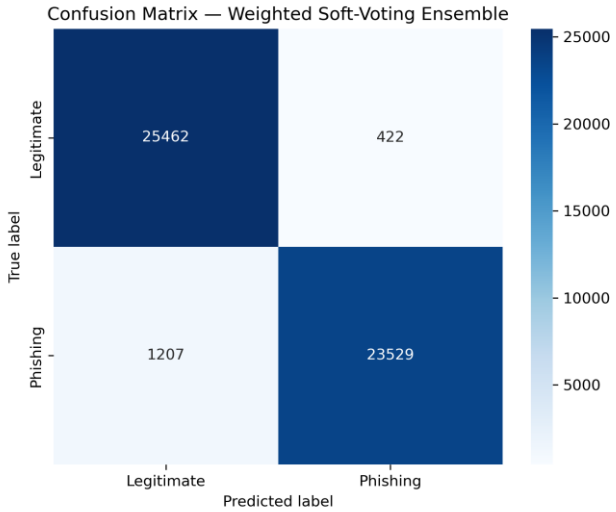


Fig. 22. Original project output: confusion matrix of weighted soft-voting ensemble on the original test split.

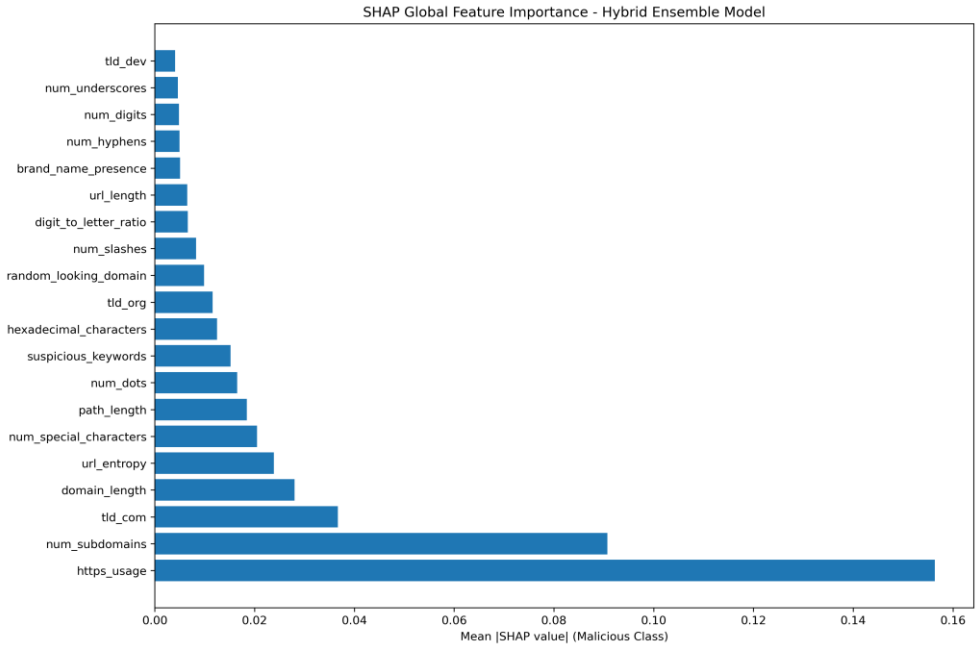


Fig. 23. Original project output: SHAP global importance of the soft-voting ensemble (KernelExplainer, 500 validation URLs). https_usage and num_subdomains dominate.

Appendix I. Decision Log

Date	Decision	Rationale	Alternatives considered
2026-10-11	Work on branch `research-v2`; original modules in `src/` untouched; new code in `src/urlresearch/`.	Preserve the baseline for fair comparison; the audit shows the original package is mostly empty files, so a clean sub-package is clearer than editing in place.	Rewrite in place (rejected: destroys baseline fidelity).
2026-10-11	Reuse the original `extract_features` verbatim for the `baseline28` feature set (wrapped to never raise).	Reproduction must use the exact original feature code.	Re-implementing (risk of silent divergence).
2026-10-11	One-hot encoding of `tld_type` is fitted on the training partition only, with `min_frequency=20` and an "infrequent" bucket.	The original fit `get_dummies` on the full dataset before splitting (521 columns). Train-only fitting is required by the leakage rules; rare TLDs would otherwise act as per-source identifiers. Documented as a deliberate deviation in the reproduction report.	Exact 521-column replication (kept as an option if the raw data arrive; difference expected to be small).
2026-10-11	Public-suffix extraction uses the tldextract bundled snapshot (no network).	Reproducibility and offline feature extraction.	Live PSL fetch (non-deterministic).
2026-10-11	Grouping key for grouped splits = registered domain (domain + public suffix), or the IP literal for IP hosts.	Pages of the same site share ownership/infrastructure; this is the standard leakage unit in URL work.	Hostname grouping (too fine: `a.example.com` vs `b.example.com` leak).
2026-10-11	Thresholds are selected on the validation split only, with three policies: 0.5, max-F1 on validation, and FPR-constrained (1 %, 0.1 %) on validation.	Security deployments operate at fixed false-positive budgets; test-set threshold tuning would be leakage.	Test-set tuning (rejected).
2026-10-11	Deep baseline = small character CNN (eXpose/URLNet style) on CPU; no transformer fine-tuning.	Machine has 12 CPU cores and no GPU; a transformer baseline would not be reproducible in reasonable time here. Documented as a limitation.	URLTran/URLBERT fine-tuning (future work).
2026-10-11	Original dataset unavailable → experiments use documented public datasets until the user supplies `raw_data.csv`.	The repository does not contain or document its data; without data nothing is reproducible.	Waiting (rejected: blocks all progress).
2026-10-11	Paper PDFs are kept locally (`research/papers/`, git-ignored) and not committed.	Many arXiv/publisher licences do not allow redistribution; the bibliography and reading notes are committed instead. While removing the PDFs from the unpushed local branch history, `git filter-branch` also removed them from the working tree; 36 were restored from download caches and all 44 remaining files were re-downloaded from the same legal sources (URLs in `research/pdf_download_urls.txt`); every expected PDF is present and starts with %PDF. No pushed history was affected.	Committing PDFs (rejected: licence risk, 168 MB).
2026-10-11	Added `lexical_v2_canon` (canonicalise string: strip scheme and leading www, then compute features) and `charcnn_canon`.	The dataset-identification probe showed that dropping artifact columns is insufficient: scheme presence leaks through slash/colon/length counts (scheme-less corpus still 99 % identifiable). Canonicalising the input removes the leak by construction (invariance test in `tests/test_feature_extractor.py`).	Column dropping only (kept as the `noart` ablation).
2026-10-11	Char-CNN budget: 5 epochs, 20-min cap, 160-char input, 3 seeds on corpus A, 1 seed on corpus B; train-set scoring skipped.	10-core laptop CPU, 16 GB RAM; full budget would take > 12 h.	GPU training (unavailable).
2026-10-11	Leave-one-corpus-out (LOCO) pools cap each source at 50k rows (stratified, seed 42).	Prevents the two ~250k corpora from dominating the pooled training data.	Uncapped pooling.

Appendix J. Reproducibility Guide

This repository started as a course project (lexical features, a logistic-regression / random-forest / XGBoost soft-vote ensemble, SHAP, a Flask demo). Branch `research-v2` turns it into a reproducible research study of **how much of the reported performance of lexical URL detectors comes from dataset collection artifacts**, and what an artifact-aware training and evaluation protocol changes.

The original code is preserved unchanged on `main` and in `src/{data,features,models,evaluation,explainability}` and `notebooks/`. New code lives in `src/urlresearch/`, `scripts/`, `configs/` and `tests/`.

*The predictor and demo return **model estimates**. A URL that is not flagged is not thereby safe.*

Repository map

Path	Content
`src/urlresearch/features/`	safe URL parser (never raises), `baseline28` (original features, original code), `lexical_v2`, `lexical_v2_canon` (artifact-neutral)
`src/urlresearch/data/`	loading, label mapping, normalised dedup, conflict removal, registered-domain grouping; random / grouped / temporal splits with overlap reports
`src/urlresearch/models/`	LR, RF, XGBoost, LightGBM, original 1:2:2 soft vote, stacking; char-CNN, canonicalised char-CNN, char+lexical hybrid
`src/urlresearch/evaluation/`	security metrics (FPR/FNR/TPR@FPR, ECE, Brier), bootstrap CIs, paired bootstrap, McNemar; label-preserving URL mutations
`src/urlresearch/experiments/runner.py`	config-driven runner (validation-only thresholds, external tests, resource measurement, saved predictions)
`src/urlresearch/prediction/predictor.py`	model bundle with schema + thresholds + calibrator; used by `app.py`
`scripts/`	data preparation, experiment queue, analyses, figures, original-pipeline reproduction
`configs/`	dataset spec and the experiment matrix (`configs/matrix/*.yaml`)
`research/`	audit, literature review, gaps, protocol, dataset audit, decisions, progress, bibliography, reading notes, PDFs
`paper/manuscript.md`	conference-paper manuscript
`results/`	metrics, tables, figures, logs (per-run predictions are git-ignored but regenerated by the scripts)

Setup

Tested on Windows 11, Python 3.10.11, CPU only (Intel i7-1355U, 16 GB RAM).

```
python -m venv .venv && .venv\Scripts\activate          # or source .venv/bin/activate
pip install torch --index-url https://download.pytorch.org/whl/cpu
pip install -r requirements.txt
python -m pytest tests -q                                 # 90+ tests
```

Data

No dataset is redistributed in this repository. `research/DATASET_SOURCES.md` gives the URL, licence, access date and SHA-256 of every file; `data/external/<name>/SOURCE.md` is written next to each download.

Corpus	Where to get it	Licence
PhiUSIIL (Corpus A)	https://archive.ics.uci.edu/static/public/967/phiusiil+phishing+url+dataset.zip	CC BY 4.0
Kaggle/HF lineage (Corpus B)	HuggingFace `Mitake/PhishingURLsANDBenignURLs` (`PhishBenignDataset.csv`)	CC BY-SA 4.0
Hannousse & Yahiouche 2021	https://data.mendeley.com/datasets/c2gw7fy2j4/3 (`dataset_B_05_2020.csv`)	CC BY 4.0
Ebbu2017	https://github.com/ebubekirbbr/pdd/tree/master/input	none stated
PhishStorm, ISCX-URL-2016	mirrors documented in `DATASET_SOURCES.md`; official pages require forms	see source
Kaggle 651k (sid321axn)	Kaggle (login) or the documented HF mirror	see source
PhishTank, URLhaus, OpenPhish	public feeds (terms of use apply; PhishTank recommends an application key)	feed terms

The original project file `data/raw/raw_data.csv` (253,098 URLs) is **not available**; its provenance analysis is in `research/DATASET_AUDIT.md` §1. If you have it, uncomment the `main` entry in `configs/datasets.yaml` and run every config with `dataset: main`.

Place downloads as described in `DATASET_SOURCES.md`, then:

```
python scripts/prepare_external.py                      # -> data/interim/*.csv (url,label[,timestamp])
python scripts/build_canonical.py configs/datasets.yaml  # -> data/processed/canonical/*.parquet +
dataset reports
python scripts/build_canonical.py configs/datasets_loco.yaml   # leave-one-corpus-out training pools
python scripts/analysis_dataset_artifacts.py            # per-class artifact statistics (DATASET_AUDIT
§3)
```

Experiments

```
python scripts/generate_configs.py                      # writes configs/matrix/*.yaml and RUN_ORDER.txt
python scripts/run_all.py                               # main matrix (resumable; skips completed runs)
set RUN_ORDER_FILE=RUN_ORDER_LOCO.txt  && python scripts/run_all.py   # leave-one-corpus-out
set RUN_ORDER_FILE=RUN_ORDER_CANON.txt && python scripts/run_all.py   # canonicalised features
python scripts/run_experiment.py configs/matrix/A_v2_grouped.yaml     # any single run
python scripts/reproduce_original_pipeline.py --dataset phiusiil      # original notebooks' code path
```

Each run writes `results/runs/<run_id>/` with `config.json`, `environment.json`, `split_<seed>.json` (indices + class balance + domain-overlap report), `predictions_<model>_<seed>[_ext_<set>].csv`, `metrics.csv`, `resources.csv` and `run.log`. Full matrix run time on the test machine: roughly one working day.

Analyses, tables and figures

```
python scripts/aggregate_results.py                     # tables: mean±sd, bootstrap CIs, paired tests
python scripts/make_figures.py                          # ROC/PR, reliability, comparisons, overlap
python scripts/analysis_dataset_probe.py --feature-set lexical_v2_canon
python scripts/analysis_calibration.py --run A_v2canon_grouped
python scripts/analysis_temporal.py                     # PhishTank recall by submission year
python scripts/analysis_mutations.py --run A_v2_grouped --models xgb lr
python scripts/analysis_shap.py --run A_v2_grouped --model xgb
python scripts/analysis_errors.py --run A_v2canon_grouped --model xgb
python scripts/make_paper_tables.py                     # tables used in paper/manuscript.md
```

Every figure and table is generated from files under `results/runs/`; nothing is edited by hand.

Protocol in one paragraph

Malicious = 1. Duplicates are removed after light normalisation and URLs with conflicting labels are dropped. The main split groups by registered domain (no domain in two partitions). Thresholds (max-F1, FPR ≤ 1 %, FPR ≤ 0.1 %) are chosen on validation only and applied unchanged to test and external sets. Five seeds for tabular models, three for char models (one for corpus B). Paired bootstrap and McNemar tests compare models on identical test URLs. Details: `research/EXPERIMENT_PROTOCOL.md`.

Demo app

```
python scripts/train_final_model.py --dataset phiusiil --feature-set lexical_v2_canon --model xgb --split
grouped
set URL_MODEL_BUNDLE=models\xgb_lexical_v2_canon_grouped_s42.joblib && python app.py
```

`POST /api/predict {"url": "...", "policy": "val_fpr0.01"}` returns the probability, threshold, a flag, a confidence band and an explicit non-guarantee statement.

Limitations

See `paper/manuscript.md` §8 and `research/RESEARCH_GAPS.md`. In short: the original dataset is unavailable; all corpora are public collections with their own artifacts; timestamped data are positive-only and mostly 2021–2026; no transformer baseline (CPU only); robustness is tested only for the listed label-preserving mutations.